

Autonomy Software Binder

Source of Truth & Operations Manual

Mars Rover Design Team

September 25, 2026

Contents

1	Quick Start and Operations Manual	8
1.1	1. Setting up the Dev Container Environment	8
1.1.1	Prerequisites	8
1.1.2	Building and Attaching	8
1.1.3	Container Environment Details	8
1.2	2. Building the Code	9
1.2.1	Standard Build (Release Mode)	9
1.2.2	Simulation Mode Build	9
1.2.3	Unit and Integration Tests Build	9
1.2.4	Cleaning the Build Directory	9
1.3	3. Running Autonomy	9
1.3.1	Running on Physical Hardware	10
1.3.2	Running in Simulation	10
1.4	4. Interactive Terminal Controls	10
1.5	5. Log Management	11
1.6	6. Quick Commands Reference	11
2	Architecture Overview	11
2.1	1. System Data Flow	11
2.1.1	Data Pipeline Breakdown	12
2.2	2. Concurrency and Threading Model	13
2.2.1	Thread Architecture	13
2.3	3. Communication Architecture (RoveComm)	13
2.3.1	Transport Protocols	13
2.3.2	Manifest Binding	13
2.4	4. Coordinate Reference Frames	14
2.4.1	1. Global / World Reference Frame (UTM / NWU)	14
2.4.2	2. Rover / Kinematics Body Frame	14
2.4.3	3. Camera / Perception Optical Frame	14
3	The State Machine	14
3.1	1. Architectural Philosophy	14
3.1.1	Design Principles	15
3.2	2. Enumerated State Definitions	15
3.3	3. Enumerated Events	16
3.4	4. State Transition Matrix	17
3.5	5. Recovery and Failsafe Subsystems	18
3.5.1	Stuck State Recovery Machine (<code>StuckState.cpp</code>)	18
3.5.2	Battery Protection Failsafe	19

3.5.3	Heading and Odometry Dynamic Realignment	19
4	URC Autonomous Navigation Mission Rules	19
4.1	1. General Mission Constraints	19
4.2	2. Mission Target Specifications (7 Total Targets)	20
4.2.1	A. GNSS-Only Locations (2 Targets)	20
4.2.2	B. AR Tag Posts (2 Targets)	20
4.2.3	C. Ground Objects (3 Targets)	20
4.3	3. Communication, Telemetry, and Lighting	21
4.3.1	Visual Status Indicators	21
4.3.2	Arrival Confirmation Protocol	21
4.4	4. Operational Protocols: Aborts, Returns, and Reprogramming	21
4.4.1	Aborts and Location Returns	21
4.4.2	Reprogramming Conditions	21
4.5	5. Technical Summary Cheat Sheet	21
5	Perception Subsystem	22
5.1	1. Algorithmic Architecture	22
5.2	2. Detection Subsystems	22
5.2.1	A. AR Tag Detection (<code>TagDetector.cpp</code>)	22
5.2.2	B. Object Detection (<code>ObjectDetector.cpp</code>)	23
5.3	3. Geolocation Engine (<code>Geolocate.hpp</code>)	23
5.4	4. Inputs, Outputs, and Limitations	24
5.4.1	Inputs	24
5.4.2	Outputs	24
5.4.3	Operational Limitations	24
6	Path Planning Subsystem	24
6.1	1. Algorithmic Architecture	24
6.2	2. Global Terrain Planning: <code>GeoPlanner</code>	25
6.2.1	A. 2.5D Costmap Generation	25
6.2.2	B. Weighted A* with Kinematic Constraints	25
6.2.3	C. Tile Management and Caching	25
6.3	3. Localized Search Patterns (<code>SearchPattern.hpp</code>)	25
6.4	4. Path Splicing and Dynamic Recovery (<code>StuckState.cpp</code>)	26
6.5	5. Inputs, Outputs, and Known Constraints	27
6.5.1	Inputs	27
6.5.2	Outputs	27
6.5.3	Operational Constraints	27
7	Control and Actuation Subsystem	27
7.1	1. Algorithmic Architecture	27
7.2	2. Lateral Tracking Controllers	28
7.2.1	1. PID Controller (<code>PIDController.cpp</code>)	28
7.2.2	2. Predictive Stanley Controller (<code>PredictiveStanleyController.cpp</code>)	28
7.2.3	3. Pure Pursuit Controller (<code>PurePursuitController.cpp</code>)	28
7.3	3. Differential Drive Kinematics (<code>DifferentialDrive.hpp</code>)	28
7.3.1	A. Deadband and Input Scaling	28
7.3.2	B. Arcade Drive Kinematics	29
7.3.3	C. Curvature Drive Kinematics	29
7.3.4	D. Squared Input Sensitivity	29
7.4	4. Drive Board Safety and Terrain Multipliers (<code>DriveBoard.cpp</code>)	29
7.4.1	Inclinometer Slope Damping	29
7.4.2	Master Throttle Control	29
7.4.3	Hard Safety Power Ceiling	29

7.5	5. Inputs, Outputs, and Network Actuation	30
7.5.1	Inputs	30
7.5.2	Outputs	30
8	PID Controller	30
8.1	1. Primary Use Cases	30
8.2	2. Mathematical Formulation	30
8.2.1	Component Breakdown	30
8.3	3. Specialized Robotics Features	31
8.3.1	Continuous Input Wraparound	31
8.3.2	Integral Windup Prevention	31
8.3.3	Output Slew Rate Limiting (Ramp Rate)	31
8.3.4	Output Low-Pass Filter	31
8.4	4. Tuning Parameters in <code>AutonomyConstants.cpp</code>	31
9	Predictive Stanley Controller	32
9.1	1. Motivation: Stanley vs Pure Heading Control	32
9.2	2. Mathematical Formulation	32
9.2.1	Standard Stanley Control Law	32
9.2.2	Predictive Horizon with Unicycle Kinematics	33
9.3	3. Output Data Structure: <code>DriveVector</code>	33
9.4	4. Tuning Constants in <code>AutonomyConstants.cpp</code>	33
9.5	5. Implementation Safeguards	34
9.6	6. Usage Example	34
10	Pure Pursuit Controller	34
10.1	1. Overview and Theory	34
10.1.1	Core Principles	35
10.2	2. Mathematical Implementation	35
10.3	3. Configuration Parameters	35
10.4	4. Output: <code>DriveVector</code>	36
10.5	5. Implementation Safeguards	36
10.6	6. Usage Example	36
11	Cameras	36
11.1	1. ZED Stereoscopic Camera Interface (<code>ZEDCamera.hpp</code> & <code>ZEDCam.cpp</code>)	36
11.1.1	Capabilities and Outputs	37
11.1.2	Hardware vs Simulation Architecture	37
11.2	2. Asynchronous Frame Retrieval Architecture	37
11.3	3. Basic Camera Interface (<code>BasicCamera.hpp</code> & <code>BasicCam.cpp</code>)	37
11.4	4. Key Configuration Parameters in <code>AutonomyConstants.cpp</code>	38
12	ArUco Tag Detection	38
12.1	1. The <code>TagDetector</code> Pipeline (<code>TagDetector.cpp</code>)	38
12.2	2. Detection Methods	39
12.2.1	1. Classical OpenCV ArUco	39
12.2.2	2. LibTorch YOLO Fallback	39
12.2.3	3. Temporal Validation and Tracking	39
12.3	3. Pose Estimation (<code>TagDetectionUtility.hpp</code>)	39
12.4	4. Usage in State Machine	40
13	Object Detection	40
13.1	1. Deep Learning Pipeline: LibTorch YOLO	40
13.1.1	Supported TorchScript Models	41
13.2	2. Target Classification and Parsing	41

13.3	3. 3D Geolocation Integration	41
13.4	4. Usage in State Machine	41
14	Vision Utilities	42
14.1	1. YOLOModel.hpp	42
14.1.1	Key Features	42
14.2	2. BoundingBoxTracking.h & BoundingBoxTracking.cpp	42
14.2.1	Pipeline	42
14.3	3. Geolocate.hpp	42
14.4	4. TagDetectionUtility.hpp & ObjectDetectionUtility.hpp	43
14.5	5. ImageOperations.hpp & FetchContainers.hpp	43
15	State Machine Handler	43
15.1	1. Primary Responsibilities	43
15.2	2. Event Handling and State Transition Sequence	44
15.3	3. Sensor Fusion: SmartRetrieveRoverPose()	44
15.4	4. Concurrency and Thread Safety	45
15.5	5. Public Interface Summary	45
16	Camera Handler	45
16.1	1. Primary Responsibilities	45
16.2	2. Managed Cameras	46
16.3	3. Concurrency and Integration	46
16.4	4. Usage Example	46
17	Tag Detection Handler	47
17.1	1. Primary Responsibilities	47
17.2	2. Managed Detectors	47
17.3	3. Concurrency and Integration	47
17.4	4. Usage Example	47
18	Object Detection Handler	48
18.1	1. Primary Responsibilities	48
18.2	2. Managed Detectors	48
18.3	3. Concurrency and Integration	48
18.4	4. Usage Example	48
19	LiDAR Handler	49
19.1	1. Primary Responsibilities	49
19.2	2. Point Data Architecture	49
19.2.1	Filtering via PointFilter	49
19.3	3. Database Engine: DuckDB	49
19.4	4. Usage Example	50
19.5	5. LiDAR Data Sources & Web Inspection Tools	50
19.5.1	A. USGS LiDAR Point Cloud Storage Repository	50
19.5.2	B. Online LiDAR Visualizer Tool	50
20	Waypoint Handler	50
20.1	1. Primary Responsibilities	51
20.2	2. RoveComm Network Callbacks	51
20.2.1	A. Position Leg (ADDPOSITIONLEG)	51
20.2.2	B. Marker Leg (ADDMARKERLEG)	51
20.2.3	C. Object Leg (ADDOBJECTLEG)	51
20.2.4	D. Clear Queue (CLEARWAYPOINTS)	51
20.3	3. Thread Safety and Concurrency	51

20.4	4. Public Interface Summary	52
20.5	5. Standardized Path Caching Keys	52
21	Recording Handler	53
21.1	1. Primary Responsibilities	53
21.2	2. Operational Modes (RecordingMode)	53
21.2.1	Modes Explained	54
21.3	3. Concurrency and Output Formatting	54
21.4	4. Configuration Parameters in AutonomyConstants.cpp	54
22	Drive Board Driver	54
22.1	1. Primary Responsibilities	55
22.2	2. Kinematics Pipeline (CalculateMove)	55
22.3	3. Inclinator Safety Damping (VariableDriveEffort)	55
22.4	4. Public Interface Summary	55
23	Navigation Board Driver	56
23.1	1. Primary Responsibilities	56
23.2	2. Ingested Data Streams	56
23.3	3. Data Freshness Guard (IsOutOfDate)	56
23.4	4. Concurrency and Thread Safety	57
23.5	5. Public Interface Summary	57
24	Multimedia Board Driver	57
24.1	1. Primary Responsibilities	57
24.2	2. Operational Lighting States	58
24.3	3. Network Transmission Protocol	58
24.4	4. Public Interface Summary	58
25	AutonomyThread Interface	58
25.1	1. Architectural Purpose	59
25.2	2. Core Methods and Overrides	59
25.2.1	A. Pure Virtual Worker Methods	59
25.2.2	B. Lifecycle and Control	59
25.3	3. Thread Priority System (AutonomyThreadPriority)	59
25.4	4. Usage Example	59
26	Thread Pools	60
26.1	1. Engine: BS::thread_pool	60
26.1.1	Benefits Over Dynamic Thread Creation	60
26.2	2. Integrated Pool Methods in AutonomyThread	61
26.2.1	A. Batch Execution	61
26.2.2	B. Dynamic Task Submission	61
26.2.3	C. Loop Parallelization (ParallelizeLoop)	61
26.3	3. Real-World Application: Multi-Subscriber Frame Copying	61
27	RoveComm Networking Protocol	61
27.1	1. Transport Protocols: UDP vs TCP	62
27.1.1	A. RoveComm UDP (User Datagram Protocol)	62
27.1.2	B. RoveComm TCP (Transmission Control Protocol)	62
27.2	2. The RoveComm Manifest	62
27.3	3. Packet Structure: RoveCommPacket<T>	62
27.4	4. Sending Telemetry and Commands	63
27.5	5. Asynchronous Callbacks	63

28 Log Files & The Quill Logging Engine	64
28.1 1. Asynchronous Logging Architecture	64
28.2 2. Configured Sinks and Loggers	64
28.2.1 Sinks	64
28.2.2 Logger Instances	65
28.3 3. Formatting Patterns	65
28.4 4. Logging Levels and Priorities	65
28.5 5. Backtrace Ring Buffer	66
28.6 6. Dynamic Runtime Log Level Adjustment	66
28.6.1 Safety Rules:	67
28.7 7. Logging in C++ Source Code	67
29 Camera Feeds & Video Recording	67
29.1 1. Asynchronous Recording Architecture	67
29.2 2. Recording Modes and Stream Types	68
29.2.1 1. eCameraHandler	68
29.2.2 2. eTagDetectionHandler	68
29.2.3 3. eObjectDetectionHandler	68
29.3 3. Output Storage Structure	69
29.4 4. Configuration Constants	69
29.5 5. Lifecycle Management	70
30 Path Plots, Analytics, and Post-Mortem Graphing	70
30.1 1. The log_playback.py Visualization Suite	70
30.1.1 Script Capabilities and Parsed Telemetry	70
30.1.2 Usage	71
30.2 2. Real-Time Path Tracking via VisualizationHandler	71
30.3 3. Matplotlib++ Library Integration	72
31 3D Interactive Visualization & The Visualization Engine	72
31.1 1. System Architecture	72
31.2 2. Local Coordinate Origin Anchoring	73
31.3 3. Core Data Structures	73
31.4 4. HTTP API Endpoints	74
31.5 5. Web Client Features	75
31.6 6. Shutdown Environment Exports	75
31.7 7. Hosted MRDT Web Visualizer Suite	75
32 Autonomy Constants Reference & Tuning Guide	76
32.1 1. General & System Constants	76
32.2 2. Drive & Kinematics Constants	78
32.3 3. Inclinator Damping Multipliers	79
32.4 4. Video Recording Handler Constants	80
32.5 5. Camera & Perception Hardware Constants	81
32.5.1 ZED Camera SDK Parameters	81
32.5.2 Physical Extrinsic Offsets	81
32.6 6. Vision Detection & Tracking Constants	81
32.7 7. State Machine Execution Constants	82
32.8 8. Path Planning & Controller Constants	84
32.9 9. Navigation Board Driver Constants	85
33 CMake Build Configuration & Options	86
33.1 1. Compiler and Toolchain Constraints	86
33.1.1 Mandatory GCC 10 Enforcement	86
33.1.2 C++ and CUDA Standards	87

33.2	2. Performance & Compilation Optimizations	87
33.3	3. Build Types	87
33.4	4. Configurable CMake Options	87
33.5	5. Subsystem Dependencies	89
34	Autonomy Troubleshooting & Field Diagnostics Guide	89
34.1	1. Live Terminal Diagnostic Hotkeys	89
34.2	2. Common Symptom-Based Solutions	90
34.2.1	1. Drivetrain Jitters or Oscillates While Driving	90
34.2.2	2. Rover Stalls When Attempting Point Turns	91
34.2.3	3. “RoveComm UDP/TCP Node Failed to Initialize”	91
34.2.4	4. ZED Camera Connection Failures or Framerate Dropping to Zero	91
34.2.5	5. Autonomy Unexpectedly Enters IdleState	91
34.2.6	6. Visual Odometry Drift and Discontinuous Jumps	92
34.2.7	7. LiDAR Database Opening Failure	92
35	Autonomy Pre-Flight & Operations Checklist	92
35.1	1. On-Rover Jetson Setup & Verification	92
35.1.1	Codebase and Toolchain Verification	92
35.1.2	Compilation	92
35.1.3	Bench Sensor Verification (Prior to Mounting)	93
35.2	2. Basestation Operations & Deployment	93
35.2.1	Remote Session Launch	93
35.3	3. Mission Waypoint Injection & Leg Types	93
35.3.1	Leg Type Configurations	93
35.4	4. Autonomous Mission Execution Workflow	94
35.5	5. Post-Mission Data Archival	94

1 Quick Start and Operations Manual

This guide covers the exact steps required to set up, build, test, and run the Autonomy Software, whether developing inside the Dev Container, simulating with Unreal Engine RoveSoSimulator, or deploying directly to physical rover hardware.

1.1 1. Setting up the Dev Container Environment

The Autonomy Software project uses Docker and Visual Studio Code Dev Containers to ensure consistent toolchains and dependencies across development machines and the onboard Jetson computers.

1.1.1 Prerequisites

- Visual Studio Code with the **Dev Containers** extension (`ms-vscode-remote.remote-containers`).
- Docker Engine (Linux) or Docker Desktop (Windows/macOS).
- NVIDIA Container Toolkit (required on Linux/Jetson hosts for GPU pass-through, CUDA, and TensorRT acceleration).

1.1.2 Building and Attaching

1. Clone the repository recursively to populate submodules (including RoveComm):

```
git clone --recursive https://github.com/MissouriMRDT/Autonomy_Software.git
```

2. Open the repository root in VSCode:

```
code Autonomy_Software
```

3. When prompted in the lower-right notification, select **Reopen in Container**.

- If the prompt does not appear, open the Command Palette (`Ctrl + Shift + P` or `Cmd + Shift + P`), type **Dev Containers: Rebuild and Reopen in Container**, and press Enter.

4. Docker pulls the preconfigured image and initializes the workspace. First-time initialization may require several minutes to complete.

1.1.3 Container Environment Details

The container environment provides: - **Compilers and Toolchain:** GCC 10.0 (strictly enforced by `CMakeLists.txt`), CMake 3.24.3+, LLD linker (`-fuse-ld=lld`), C++20 standard, CUDA 12 / 11 matching the ZED SDK. - **Machine Learning:** LibTorch (PyTorch C++ frontend) with CUDA acceleration for YOLO object and tag detection. - **Computer Vision:** OpenCV 4.x with CUDA modules, Stereolabs ZED SDK 4.x. - **Geospatial and Scientific:** GeographicLib (geodesic and UTM projections), Eigen3, Point Cloud Library (PCL), OpenMPI, OpenMP. - **Storage and Networking:** DuckDB (USGS LiDAR point cloud queries), RoveComm (in-house UDP/TCP protocol), LibDataChannel (WebRTC for simulator camera feeds), nlhmann-json. - **Logging:** Quill asynchronous logging engine.

[!TIP] Comprehensive Installation & Contribution Documentation - **Full Installation Guide:** For native Linux configuration, bare-metal toolchains, and NVIDIA Jetson deployment steps, refer to [INSTALL.md](#). - **Contributing & Git Workflow:** For branch naming conventions, pull request workflows, review procedures, and C++ code style requirements, see [CONTRIBUTING.md](#). - **Team Documentation:** Broad team-wide architecture guides and subsystem documentation are hosted on the [MRDT Documentation Portal](#) and mirrored in the [MissouriMRDT/RoveSoDocs repository](#). Internal packages and datasets are hosted on the [MRDT GitLab Organization](#).

1.2 2. Building the Code

The build system is orchestrated using CMake.

[!IMPORTANT] GCC 10.0 is strictly enforced by `CMakeLists.txt` via `gcc -dumpversion`. Building with other GCC versions will result in a fatal configuration error.

1.2.1 Standard Build (Release Mode)

Always use Release mode on the rover and during competitive runs for maximum compiler optimization (`-O3` equivalent) and loop vectorization:

```
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
```

To limit memory usage during parallel compilation, CMake Unity builds are enabled by default (`CMAKE_UNITY_BUILD ON` with batch size 8). You can adjust parallel compile jobs via:

```
make -j4
```

1.2.2 Simulation Mode Build

When testing without physical rover hardware, compile with `BUILD_SIM_MODE=ON`:

```
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DBUILD_SIM_MODE=ON ..
make -j$(nproc)
```

In simulation mode: - The output binary is named `Autonomy_Software_Sim`. - The software connects to Unreal Engine RoveSoSimulator via WebRTC (LibDataChannel) and local RoveComm endpoints rather than physical hardware boards and cameras.

1.2.3 Unit and Integration Tests Build

To compile the Google Test suites:

```
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Debug -DBUILD_TESTS_MODE=ON ..
make -j$(nproc)
ctest --output-on-failure
```

Optional test suites for LiDAR and GeoPlanner require USGS database files and can be enabled with `-DENABLE_LIDAR_GEO_UTESTS=ON`.

1.2.4 Cleaning the Build Directory

If CMake cache issues occur after switching branches or updating submodules:

```
rm -rf build/*
cmake -DCMAKE_BUILD_TYPE=Release -B build/
make -C build/ -j$(nproc)
```

1.3 3. Running Autonomy

Executables are written to the `build/` directory.

1.3.1 Running on Physical Hardware

```
cd build
./Autonomy_Software
```

On launch, the software performs the following initialization sequence: 1. Initializes the Quill asynchronous loggers and prints the ASCII software header. 2. Binds RoveComm UDP and TCP sockets (UDP port from manifest, TCP interface IP from constants). 3. Instantiates board drivers (`DriveBoard`, `MultimediaBoard`, `NavigationBoard`). 4. Instantiates handlers (`WaypointHandler`, `LiDARHandler`, `CameraHandler`, `TagDetectionHandler`, `ObjectDetectionHandler`, `StateMachineHandler`). 5. Loads the USGS DuckDB database from `constants::LIDAR_HANDLER_DB_PATH`. 6. Instantiates `GeoPlanner` and starts `VisualizationHandler` on port 8080. 7. Starts camera background threads and begins detector pipelines. 8. Enters the main execution loop in `main.cpp`.

1.3.2 Running in Simulation

```
cd build
./Autonomy_Software_Sim
```

Ensure that the Unreal RoveSoSimulator environment is executing before launching the simulation binary.

1.4 4. Interactive Terminal Controls

While `Autonomy_Software` is executing, the terminal is configured into non-canonical mode, enabling single-keypress diagnostic commands without pressing Enter:

Key	Function	Output Description
h / H	Help	Prints the terminal hotkey command summary.
f / F	FPS / IPS Stats	Prints exact iterations-per-second for all camera, detector, networking, and state machine threads.
p / P	Rover Pose	Prints current Easting, Northing, Altitude (UTM), and fused compass heading.
s / S	Sensor Data	Queries the ZED IMU and prints linear acceleration (X, Y, Z), angular velocity, and Euler orientation.
d / D	Drive Powers	Prints current left and right drive power commands (-1.0 to 1.0).
t / T	Tag Detections	Dumps total detected ArUco markers, best OpenCV tag ID/distance/yaw, and best Torch tag detection.
m / M	Object Detections	Dumps total detected props (mallets, bottles, rock picks) with bounding box dimensions, confidence, and classes.

Key	Function	Output Description
q / Q	Graceful Quit	Triggers clean shutdown: stops drives, signals state machine abort, saves 3D visualization, joins threads, and exits.

1.5 5. Log Management

Text logging is powered by the **Quill** asynchronous engine, ensuring logging operations never block time-critical control threads.

- **Storage Location:** Logs are written to timestamped directories under `logs/` (e.g., `logs/YYYY-MM-DD_HH-MM-SS/`).
- **Sinks Configured:**
 - **Console Sink:** Colorized terminal output for immediate visibility.
 - **Rotating File Sink:** Written to disk with automatic size rotation.
 - **RoveComm Sink:** Broadcasts log records over UDP to the Basestation GUI console.
- **Log Levels:** Default levels are defined in `AutonomyConstants.cpp` (`CONSOLE_DEFAULT_LEVEL`, `FILE_DEFAULT_LEVEL`, `ROVECOMM_DEFAULT_LEVEL`) and can be modified at runtime via RoveComm `SETLOGGINGLEVELS` packets.

1.6 6. Quick Commands Reference

Action	Command
Clean Build Tree	<code>rm -rf build/*</code>
Configure Release	<code>cmake -DCMAKE_BUILD_TYPE=Release -B build/</code>
Compile All Targets	<code>make -C build/ -j\$(nproc)</code>
Run Onboard	<code>./build/Autonomy_Software</code>
Run Simulation	<code>./build/Autonomy_Software_Sim</code>
Run Unit Tests	<code>cd build && ctest --output-on-failure</code>

ewpage

2 Architecture Overview

This section describes the high-level software architecture of the Autonomy Software, detailing data pathways, inter-thread synchronization, external hardware communication, and coordinate transformations across navigation frames.

2.1 1. System Data Flow

The software operates as a modular, multithreaded architecture organized around a centralized state machine. The primary data pipeline proceeds through five stages:

```
[Sensors & Hardware]
|
v (RoveComm UDP / USB / CUDA)
[Drivers & Handlers] (NavBoard, DriveBoard, CameraHandler, LiDARHandler)
```

```

|
v (Thread-safe Queues / Shared Memory)
[Perception & Geolocation] (TagDetector, ObjectDetector, GeolocateBox)
|
v (UTM Waypoints & Sensor Fusion)
[State Machine] (StateMachineHandler -> Active State)
|
v (Goal Coordinates & Costmap Lookups)
[Path Planning] (GeoPlanner / A* / Search Patterns)
|
v (Target Waypoints & Speed Profile)
[Controllers & Kinematics] (Predictive Stanley / Pure Pursuit / PID / DifferentialDrive)
|
v (RoveComm UDP)
[Core & Motor Controllers]

```

2.1.1 Data Pipeline Breakdown

1. Sensors to Drivers and Handlers:

- Navigation telemetry (GPS latitude, longitude, altitude, and IMU compass heading) arrives over UDP via RoveComm from the physical Navigation Board.
- Stereoscopic camera frames (RGB color matrices and 32-bit floating point depth/point-cloud matrices) are captured asynchronously on GPU streams via the Stereolabs ZED SDK (`ZEDCam.cpp`) or virtual WebRTC pipes (`SIMZEDCam.cpp`).
- Spatial elevation and obstacle data are retrieved through DuckDB spatial queries against pre-indexed USGS LAS 1.4 point cloud databases (`LiDARHandler.cpp`).

2. Handlers to Perception Pipelines:

- Image frames are passed to `TagDetectionHandler` and `ObjectDetectionHandler`.
- `TagDetector` runs OpenCV ArUco dictionary matching (`DICT_4X4_50`) and an optional LibTorch YOLO model in parallel.
- `ObjectDetector` runs custom LibTorch YOLO models (`.pt` TorchScript) with CUDA acceleration, followed by non-maximum suppression (NMS) and OpenCV CSRT/KCF bounding-box tracking.
- When an object or marker is detected in pixel coordinates, `GeolocateBox()` queries the corresponding 3D camera point cloud (`CV_32FC4`), applies statistical depth filtering (20th percentile surface extraction), and projects the vector into global UTM space using the fused rover pose.

3. Perception and Pose to State Machine:

- `StateMachineHandler` executes at a controlled frequency (`STATEMACHINE_MAX_IPS`, typically 60 Hz).
- The handler runs `SmartRetrieveRoverPose()`, fusing low-frequency GPS heading and position with high-frequency ZED IMU angular rates and dynamic drift correction.
- The active state evaluates target proximity, visual locks, stuck criteria, and mission timeouts to determine whether to transition states.

4. State Machine to Path Planning:

- During `eNavigating`, the state machine passes the fused rover UTM position and the target destination to `GeoPlanner`.
- `GeoPlanner` calculates a 2.5D costmap using USGS terrain data from `LiDARHandler`, applies obstacle inflation, and computes a collision-free path using Kinematically Constrained Weighted A*.
- During `eSearchPattern`, the system mathematically constructs expanding spiral, zigzag, or snake patterns around the search radius.

5. Path Planning to Kinematics and Motor Actuation:

- The planned path is passed to either the `PredictiveStanleyController`, `PurePursuitController`, or the heading PID controller.
- The controller outputs a steering angle and linear velocity setpoint.

- Kinematics routines in `DifferentialDrive.hpp` (Arcade Drive or Curvature Drive) translate linear and angular requests into normalized left and right track power commands (-1.0 to 1.0).
 - Inclinometer slope damping is applied based on rover pitch and roll.
 - The resulting powers are packaged into a RoveComm UDP `DRIVELEFTRIGHT` packet and transmitted to the Core motor controller board.
-

2.2 2. Concurrency and Threading Model

To avoid latency in control loops, compute-intensive processes are separated into dedicated asynchronous worker threads:

2.2.1 Thread Architecture

- **AutonomyThread<T> Interface:** Fundamental base class for long-running worker loops (`CameraHandler`, `TagDetector`, `ObjectDetector`, `StateMachineHandler`, `VisualizationHandler`, `RoveCommUDP`). Each instance runs an independent OS thread, manages thread states (`eStarting`, `eRunning`, `eStopping`, `eStopped`), enforces iteration-per-second (IPS) rate limits, and safely cleans up in its destructor.
 - **Thread Pools (`BS::thread_pool`):** Each `AutonomyThread` instantiates an internal Barak Shoshany thread pool. This is used for bursting tasks without thread allocation overhead, such as dispatching asynchronous frame copies across multiple subscriber buffers (`RequestFrameCopy()`) or evaluating parallelized search loops.
 - **Mutex Strategies:**
 - `std::shared_mutex` is used throughout the codebase for read-heavy resources (such as `WaypointHandler` lists and `LiDARHandler` database connections). Multiple consumers can acquire shared read locks (`std::shared_lock`) simultaneously, while mutations require an exclusive unique lock (`std::unique_lock`).
 - Atomic flags (`std::atomic<bool>`) are utilized for fast state toggles and lifecycle management without mutex lock contention.
-

2.3 3. Communication Architecture (RoveComm)

Communication between the Jetson computing platform and distributed rover subsystems uses the MRDT RoveComm protocol.

2.3.1 Transport Protocols

- **UDP (User Datagram Protocol):**
 - Used for real-time, high-rate, loss-tolerant telemetry.
 - Examples: motor power commands (`DRIVELEFTRIGHT`), GPS telemetry (`GPSLATLON`), IMU heading packets (`IMUDATA`), lighting controls (`STATEDISPLAY`, `LEDRGB`), and console log streaming.
 - Sockets do not block; dropped packets are superseded by subsequent iterations.
- **TCP (Transmission Control Protocol):**
 - Used for state-critical, ordered, guaranteed commands.
 - Examples: mission waypoint injection (`ADDPPOSITIONLEG`, `ADDMARKERLEG`, `ADDOBJECTLEG`), queue clears (`CLEARWAYPOINTS`), and logging level reconfigurations (`SETLOGGINGLEVELS`).

2.3.2 Manifest Binding

All communication relies on `RoveCommManifest.h`. Packet headers define: - `unDataId`: Unique 16-bit identifier for the command or telemetry stream. - `unDataCount`: Number of elements contained in the payload array. - `eDataType`: Primitive type (`UINT8_T`, `INT32_T`, `FLOAT_T`, `DOUBLE_T`). Payload bytes are converted to and from network byte order using endianness helper macros (`hton11`, `ntoh11`).

2.4 4. Coordinate Reference Frames

Navigational calculations span three distinct reference frames. Maintaining mathematical consistency across these frames is essential for accurate geolocation and tracking.

2.4.1 1. Global / World Reference Frame (UTM / NWU)

- **Coordinate System:** Universal Transverse Mercator (UTM).
- **Units:** Meters along a 2D Cartesian grid, with separate altitude in meters above sea level.
- **Orientation Standard:** North-West-Up (NWU):
 - **+X:** North
 - **+Y:** West
 - **+Z:** Up
- **Compass Heading:** Measured clockwise from true North (0° = North, 90° = East, 180° = South, 270° = West).

2.4.2 2. Rover / Kinematics Body Frame

- **Origin:** Physical center of rotation of the rover chassis on the ground plane.
- **Axes Convention:**
 - **+X:** Forward (longitudinal direction of travel)
 - **+Y:** Left (lateral port side)
 - **+Z:** Up (vertical dorsal direction)
- **Rotations:** Follow standard right-hand rule about the +Z axis (counter-clockwise yaw is positive).

2.4.3 3. Camera / Perception Optical Frame

- **Camera Standard:** Stereolabs ZED SDK native coordinate system: `sl::COORDINATE_SYSTEM::LEFT_HANDED_Y_UP`.
- **Axes Convention:**
 - **+X:** Right (across sensor face)
 - **+Y:** Up (vertical sensor plane)
 - **+Z:** Forward (optical axis into the scene)
- **Frame Transformation:** When an object is detected at 3D camera coordinates (X_c, Y_c, Z_c) , its position in global UTM coordinates (E_o, N_o, U_o) is computed relative to camera pose (E_c, N_c, U_c) and compass heading θ (converted to standard Cartesian angle $\alpha = 90^\circ - \theta$):

$$E_o = E_c + (Z_c \cos \alpha + X_c \sin \alpha)$$

$$N_o = N_c + (Z_c \sin \alpha - X_c \cos \alpha)$$

$$U_o = U_c + Y_c$$

ewpage

3 The State Machine

The State Machine forms the decision-making core of the Autonomy Software. It dictates rover behavior by evaluating sensory perception, navigation waypoints, failsafes, and mission timers in a continuous loop.

3.1 1. Architectural Philosophy

The software employs a state-based architecture where navigation tasks are decoupled into explicit, self-contained states derived from `statemachine::State` (`src/interfaces/State.hpp`).

3.1.1 Design Principles

- **Strict Behavioral Isolation:** Each state executes only the logic relevant to its immediate objective. For example, `eApproachingMarker` focuses strictly on visual tracking and approach heuristics without running the global A* path planner.
 - **Event-Driven Transitions:** State changes occur when events are dispatched via `globals::g_pStateMachineHandler->bSaveCurrentState()`.
 - **State Preservation and Recall:** When a temporary disruption occurs (such as getting stuck or needing a reversing maneuver), the calling state can be preserved in `m_umSavedStates` via `bSaveCurrentState = true`. Once recovery finishes, the previous state is recalled without losing internal progress (such as an in-progress search spiral).
 - **Dedicated Execution Thread:** The state machine runs on an `AutonomyThread` in `StateMachineHandler.cpp`, decoupled from camera capture and network I/O. Its tick rate is throttled by `constants::STATEMACHINE_MAX_IPS` (typically 60 Hz).
-

3.2 2. Enumerated State Definitions

The autonomy system defines 10 discrete states (`statemachine::States`):

State Enum	Name	Primary Objective and Behavior
<code>eIdle</code>	Idle	Default standby state. Drives are halted (<code>SendStop()</code>), and lighting is set to off or teleop. The system listens for RoveComm start commands (<code>eStart</code>) or waypoint assignments.
<code>eNavigating</code>	Navigating	Global transit state. The rover queries <code>GeoPlanner / A*</code> to navigate through waypoints toward the target coordinate while steering with predictive or PID controllers.
<code>eSearchPattern</code>	Search Pattern	Executed when the rover arrives at the vicinity of a marker or object but cannot visually identify it. Drives systematic geometric patterns (Spiral, ZigZag, Snake) around the waypoint coordinate.
<code>eApproachingMarker</code>	Approaching Marker	Visual servoing state. Activates once an ArUco tag is detected. Uses trigonometric pose estimation from <code>TagDetectionUtility.hpp</code> to drive directly toward the tag face.
<code>eApproachingObject</code>	Approaching Object	Object-tracking servoing state. Activates once a target prop (mallet, bottle, rock pick) is detected by YOLO. Tracks the bounding box and approaches the geolocated coordinate.

State Enum	Name	Primary Objective and Behavior
eVerifyingPosition	Verifying Position	Stop-and-sample state for GNSS-only waypoints. The rover remains stationary for <code>NAVIGATING_VERIFY_SAMPLE_TIME</code> (10.0 seconds) to average GPS coordinates and confirm arrival within tolerance.
eVerifyingMarker	Verifying Marker	Stationary confirmation state for AR tags. The rover stops in front of the tag for <code>APPROACH_MARKER_VERIFY_TIME</code> seconds, confirming tag visibility before declaring completion.
eVerifyingObject	Verifying Object	Stationary confirmation state for ground objects. The rover halts and samples the YOLO detector across <code>APPROACH_OBJECT_VERIFY_TIME</code> seconds, verifying a minimum detection hit-rate.
eReversing	Reversing	Fallback driving state. Drives backward for <code>constants::REVERSE_DISTANCE</code> at <code>constants::REVERSE_MOTOR_POWER</code> to back away from an obstruction or overshoot.
eStuck	Stuck	Multi-phase recovery state. Triggered when motors are commanded but position/heading do not change. Executes sequential reversing and realignment attempts, followed by path splicing upon recovery.

3.3 3. Enumerated Events

State transitions are driven by discrete triggers defined in `statemachine::Event`:

```
enum class Event
{
    eStart, // Operator commanded autonomy to begin
    eReachedGpsCoordinate, // Rover entered goal radius of target coordinate
    eReachedMarker, // Rover closed distance to ArUco marker within proximity threshold
    eReachedObject, // Rover closed distance to target object within proximity threshold
    eMarkerSeen, // Target ArUco tag detected with sufficient confidence/age
    eObjectSeen, // Target ground object detected by YOLO model
    eMarkerUnseen, // Visual lock on ArUco tag lost beyond timeout buffer
    eObjectUnseen, // Visual lock on object lost beyond timeout buffer
    eVerifyingComplete, // Verification window succeeded (confirmed target)
    eVerifyingFailed, // Verification window failed (false positive or lost sight)
```

```

eAbort,           // Operator commanded immediate emergency stop
eRestart,         // Command to reset and restart mission
eNoWaypoint,      // Waypoint queue is empty
eNewWaypoint,     // New waypoint added to queue
eReverse,         // Triggered to back out of a deadlock
eReverseComplete, // Reversing distance has been achieved
eSearchFailed,    // Search pattern exhausted without detecting target
eStuck,           // Motion sensors confirm drive stall
eUnstuck          // Motion confirmed; rover free to resume prior state
};

```

3.4 4. State Transition Matrix

The table below details typical state transitions, their triggering events, and the resulting target state:

Current State	Event Trigger	Next State	Context / Action
eIdle	eStart	eNavigating	Operator begins mission; first waypoint popped from queue.
eNavigating	eReachedGpsCoordinate (Nav Leg)	eVerifyingPosition	Reached GNSS waypoint within NAVIGATING_REACHED_GOAL_RADIUS.
eNavigating	eReachedGpsCoordinate (Marker/Obj Leg)	eSearchPattern	Arrived at vicinity coordinates without visual detection.
eNavigating	eMarkerSeen	eApproachingMarker	Tag detected en route; global path planning aborted for visual approach.
eNavigating	eObjectSeen	eApproachingObject	Object detected en route; visual approach begins.
eNavigating	eNoWaypoint	eIdle	Mission queue completed.
eSearchPattern	eMarkerSeen	eApproachingMarker	Tag spotted during search spiral; switches to visual servoing.
eSearchPattern	eObjectSeen	eApproachingObject	Object spotted during search spiral; switches to visual approach.
eSearchPattern	Outward Leg Complete	eSearchPattern	Switches to SearchPatternType::END, loads "GeoPlannerPathReverse" to sweep back inward to center.
eSearchPattern	eSearchFailed	eIdle / Next Leg	Inward and outward search legs exhausted (TargetIndex > size - 4); logs warning and proceeds.

Current State	Event Trigger	Next State	Context / Action
eApproachingMarker	eReachedMarker	eVerifyingMarker	Rover within APPROACH_MARKER_PROXIMITY_THRESHOLD (e.g., 2.0 m).
eApproachingMarker	eMarkerUnseen	eSearchPattern	Tag tracking lost for longer than buffer time; resumes search pattern.
eApproachingObject	eReachedObject	eVerifyingObject	Rover within APPROACH_OBJECT_PROXIMITY_THRESHOLD
eApproachingObject	eObjectUnseen	eSearchPattern	Object lost from view; returns to localized search.
eVerifyingMarker	eVerifyingComplete	eNavigating / eIdle	Goal confirmed; flashes green LED, signals basestation, loads next leg.
eVerifyingMarker	eVerifyingFailed	eSearchPattern	Tag could not be re-verified; falls back to search pattern.
eVerifyingObject	eVerifyingComplete	eNavigating / eIdle	Object confirmed; flashes green LED, signals basestation, loads next leg.
eVerifyingPosition	eVerifyingComplete	eNavigating / eIdle	Position confirmed within GPS error radius; loads next leg.
<i>Any Moving State</i>	eStuck	eStuck	Motion detector confirmed motor stall; state preserved for recovery.
eStuck	eUnstuck	<i>Previous State</i>	Rover escaped stall; ModifyPath() splices around obstacle before resuming.
<i>Any State</i>	eAbort	eIdle	Emergency abort commanded; motors stopped immediately.

3.5 5. Recovery and Failsafe Subsystems

3.5.1 Stuck State Recovery Machine (StuckState.cpp)

Stuck detection is handled by `TimeIntervalBasedStuckDetector` (`src/util/states/StuckDetection.hpp`). If linear velocity is below `constants::STUCK_CHECK_VEL_THRESH` and angular velocity is below `constants::STUCK_CHECK_ROT_THRESH` while motors are commanding power for multiple consecutive intervals, `Event::eStuck` is dispatched.

Upon entering `StuckState`:

- Obstacle Declaration:** Immediately calls `DeclareObstacle()` to record a permanent circular obstacle of radius `constants::STUCK_OBSTACLE_RADIUS` (2.0 m) projected `constants::STUCK_OBSTACLE_DISTANCE` (1.0 m) along the rover's heading.
- Sequential Recovery Routine:**
 - **AttemptType::eReverseCurrentHeading:** Preserves active state in `m_umSavedStates`, maintains heading, and dispatches `Event::eReverse` to back up by `constants::REVERSE_DISTANCE`.

AttemptType::eReverseLeft: If still stationary after reversing, point-turns to `m_dOriginalHeading + constants::STUCK_ALIGN_DEGREES` and dispatches a second reversing attempt. - **AttemptType::eReverseRight:** If still stuck, point-turns to `m_dOriginalHeading - constants::STUCK_ALIGN_DEGREES` and reverses a third time. - **AttemptType::eGiveUp:** If all three directional reversals fail to extricate the rover beyond `constants::STUCK_SAME_POINT_PROXIMITY` (0.5 m), it logs a warning and dispatches `Event::eAbort` to return to `eIdle`. 3. **Dynamic Path Modification on Recovery (Event::eUnstuck):** Once physical displacement from the stuck location exceeds `constants::STUCK_SAME_POINT_PROXIMITY`, the state fires `Event::eUnstuck`, invoking `ModifyPath()`: - Queries the obstacle record at index `GetObstaclesCount()` - 1. - Executes `SplicePath()` on "GeoPlannerPath" (and "GeoPlannerPathReverse" if the triggering state was `eSearchPattern`). - Excises trapped waypoints while preserving the final goal node (`it != std::prev(vPath.end())`). - Connects the detour using `GeoPlanner::PlanPath()`, falling back to the current rover UTM pose if the initial waypoint was deleted. - Resumes the saved triggering state (`m_eTriggeringState`) with the updated obstacle-free path.

3.5.2 Battery Protection Failsafe

The state machine monitors battery metrics via RoveComm PMS telemetry. If `BATTERY_CHECKS_ENABLED` is true and any cell drops below `constants::BATTERY_MINIMUM_CELL_VOLTAGE` (default 3.2V), the state machine forcefully dispatches `Event::eAbort` to transition to `eIdle` and halt motor output, preventing battery degradation.

3.5.3 Heading and Odometry Dynamic Realignment

In `StateMachineHandler::SmartRetrieveRoverPose()`, the system monitors the drift between the ZED visual-inertial odometry and absolute GPS/magnetometer heading. - When the rover drives forward at speeds exceeding `constants::ZED_REALIGN_VEL_THRESH` with angular rates below `constants::ZED_REALIGN_ROT_THRESH`, or while resting in `eIdle`, `RealignZEDHeading()` computes the offset:

$$\text{Offset} = \text{Heading}_{\text{Actual}} - \text{Heading}_{\text{Raw ZED}}$$

- During high-rate point-turns or evasive maneuvers where magnetic interference spikes, the system uses the high-frequency ZED IMU fused with this calibrated offset, avoiding erratic steering from compass distortion.

ewpage

4 URC Autonomous Navigation Mission Rules

This section contains the official University Rover Challenge (URC) specifications and rules for the Autonomous Navigation Mission, providing a reference for tuning state machine constraints, tolerance radiuses, and detection thresholds.

4.1 1. General Mission Constraints

- **Course Duration and Length:** The total mission window is **30 minutes**, with a cumulative course distance up to **2.0 km**.
 - **Mission Turnaround:** Teams may be required to begin the Autonomous Navigation Mission as soon as 10 minutes following the completion of the Equipment Servicing Mission, operating from the same Command and Control (C2) station.
 - **Reference Start Gate:** Teams are provided with a reference GNSS coordinate at a designated start gate. Differential GNSS (RTK) is permitted and strongly recommended for base station and rover ground-truth positioning.
-

4.2 2. Mission Target Specifications (7 Total Targets)

The course features **7 distinct targets** distributed across desert terrain. Teams are permitted to attempt and clear targets in any operational sequence.

4.2.1 A. GNSS-Only Locations (2 Targets)

- **Description:** Target coordinates provided as pure GNSS coordinates with no physical visual markers or posts at the site.
- **Objective:** Autonomously navigate to the designated coordinate and come to a complete stop.
- **Success Criteria:** The rover must come to a complete halt within **3.0 meters** of the target GNSS coordinate.
- **Search Radius:** Target coordinates are provided with high precision. No expanded search pattern is required.

4.2.2 B. AR Tag Posts (2 Targets)

- **Description:** Vertical posts equipped with three-sided visual markers displaying black and white ArUco tags.
- **Physical Specifications:**
 - Markers have 20 cm x 20 cm faces mounted 0.5 to 1.5 meters above ground level.
 - ArUco dictionary: **DICT_4X4_50**.
 - Marker grid: 4x4 data cells with a 1-cell wide white border. Each individual cell measures **2.5 cm**.
 - All three sides of the post display identical marker IDs to ensure 360-degree visibility.
- **Objective:** Navigate to vicinity coordinates, locate the post using optical cameras, and approach.
- **Success Criteria:** The rover must autonomously halt within **2.0 meters** of the marker post.
- **GNSS Offset and Search Radiuses:**
 - **Post 1:** Provided vicinity GNSS coordinate is **5 to 10 meters** from the physical post.
 - **Post 2:** Provided vicinity GNSS coordinate is **10 to 20 meters** from the physical post.
 - Software must trigger **eSearchPattern** upon entering the vicinity to locate the marker visually.

4.2.3 C. Ground Objects (3 Targets)

- **Description:** Loose objects placed on terrain requiring autonomous optical identification. Objects may be located near obstacles (such as rock gardens or berms) requiring autonomous obstacle avoidance.
 - **Target Prop Specifications:**
 1. An orange rubber mallet.
 2. A rock pick hammer.
 3. A standard 1-liter wide-mouthed plastic water bottle (approximately 21.5 cm height, 9.0 cm diameter, unconstrained color/labeling).
 - **Objective:** Detect the target prop visually and highlight it on the operator C2 display. Physical contact or manipulation is not required.
 - **Success Criteria:**
 - The rover must come to a complete stop at any distance with the target object in clear optical view.
 - The C2 operator display must autonomously and distinctly highlight exactly one bounding box around the target object to verify computer vision recognition to the judging panel.
 - **GNSS Offset and Search Radiuses:**
 - **Objects 1 and 2:** Provided vicinity coordinates have an error offset of **< 3.0 meters**.
 - **Object 3:** Provided vicinity coordinates have an error offset of **< 10.0 meters**.
-

4.3 3. Communication, Telemetry, and Lighting

4.3.1 Visual Status Indicators

The rover must carry an externally visible status LED assembly (controlled via the `MultimediaBoard` driver) meeting the following operational conventions: - **Solid Red**: Autonomous mode active (state machine executing). - **Solid Blue**: Manual teleoperation active (operator joystick override). - **Flashing Green**: Objective arrival confirmed (state machine completed target verification).

4.3.2 Arrival Confirmation Protocol

Upon reaching a target location, the rover must autonomously: 1. Cease all drive motor commands (`SendStop()`). 2. Activate the flashing green LED indicator. 3. Broadcast telemetry to the C2 display over `RoveComm`, triggering an unambiguous visual prompt for judges.

4.4 4. Operational Protocols: Aborts, Returns, and Reprogramming

4.4.1 Aborts and Location Returns

Operators may command an autonomous abort at any point: - The rover may autonomously return to any previously visited target location or reference coordinate. - The rover must halt within **5.0 meters** of the prior target coordinate. - **Autonomous Return Penalty**: 0% penalty. - **Teleoperated Return Penalty**: If operators manually pilot the rover back to a previously visited target, a **20% penalty** is assessed against the maximum points available for that target. Teleoperation must follow the most direct navigable route back without scouting unvisited areas.

4.4.2 Reprogramming Conditions

Operators may only transmit waypoint coordinates, modify configuration parameters, or alter software logic under strict mission conditions: - While halted following a successful target arrival confirmation. - While halted following an abort return to a previously visited location.

No manual waypoint input or parameter changes are permitted while the rover is in motion or stopped at unverified vicinity coordinates.

4.5 5. Technical Summary Cheat Sheet

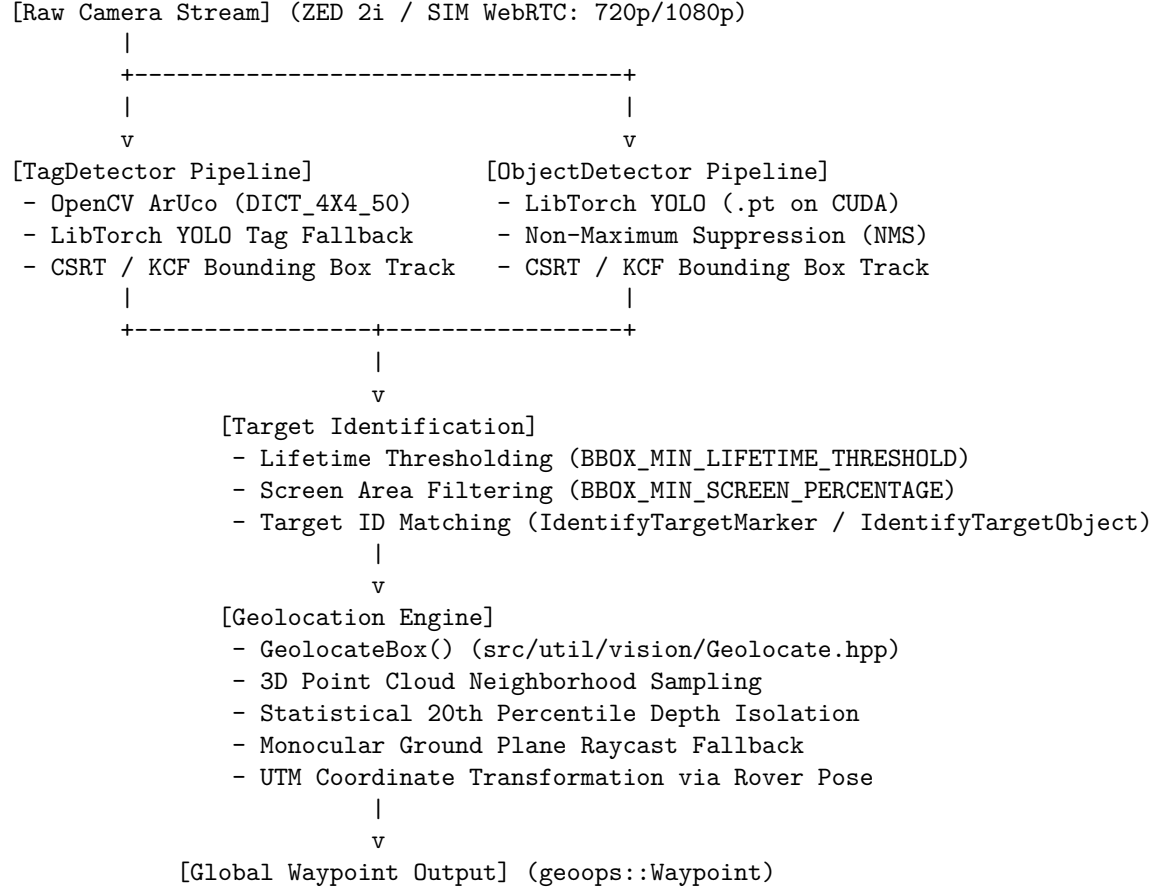
Target Type	Required Stop Distance	Initial Coordinate Offset (Search Radius)	Visual Specification	State Machine Event Trigger
GNSS 1 & 2	≤ 3.0 meters	0 meters (exact)	None	<code>eReachedGpsCoordinate</code>
AR Post 1	≤ 2.0 meters	5 to 10 meters	DICT_4X4_50, 20x20 cm	<code>eMarkerSeen</code> → <code>eReachedMarker</code>
AR Post 2	≤ 2.0 meters	10 to 20 meters	DICT_4X4_50, 20x20 cm	<code>eMarkerSeen</code> → <code>eReachedMarker</code>
Object 1 & 2	Any (optical lock)	≤ 3.0 meters	Mallet / Rock Pick	<code>eObjectSeen</code> → <code>eReachedObject</code>
Object 3	Any (optical lock)	≤ 10.0 meters	1-Liter Water Bottle	<code>eObjectSeen</code> → <code>eReachedObject</code>

5 Perception Subsystem

The Perception subsystem processes optical and depth imagery to identify mission targets (ArUco fiducial tags, competition props) and map physical obstacles across the terrain.

5.1 1. Algorithmic Architecture

The perception pipeline integrates deep learning models, classical computer vision, stereoscopic depth mapping, and geometric projection:



5.2 2. Detection Subsystems

5.2.1 A. AR Tag Detection (TagDetector.cpp)

Fiducial marker detection operates through a dual-path pipeline: 1. **Classical OpenCV ArUco:** - Uses `cv::aruco::detectMarkers` with dictionary `DICT_4X4_50`. - Employs sub-pixel corner refinement (`cv::aruco::CORNER_REFINE_SUBPIX` or `CORNER_REFINE_CONTOUR`) up to `TAGDETECT_MAINCAM_CORNER_REFINE_MAX_ITER`. - Inverted marker detection can be enabled via `TAGDETECT_MAINCAM_DETECT_INVERTED_MARKER` for handling low-contrast lighting conditions. 2. **LibTorch YOLO Tag Fallback:** - When distance or glare degrades high-frequency corner contrast, classical ArUco fails. - An optional YOLO model (`TAGDETECT_TORCH_MODEL`) trained on marker silhouettes runs asynchronously via LibTorch on the GPU, outputting a candidate bounding box and confidence score (`TAGDETECT_MAINCAM_TORCH_CONFIDENCE`). - The bounding box is tracked until the rover closes distance, allowing OpenCV to resolve the

marker ID. 3. **Temporal Tracking:** - Bounding boxes are tracked across intermediate frames using OpenCV KCF/CSRT trackers (`src/util/vision/BoundingBoxTracking.cpp`). - Tags must persist for longer than `constants::BBOX_MIN_LIFETIME_THRESHOLD` (e.g., 0.5 seconds) and occupy at least `constants::BBOX_MIN_SCREEN_PERCENTAGE` of the camera frame to be considered valid targets.

5.2.2 B. Object Detection (`ObjectDetector.cpp`)

Target props (mallet, rock pick, water bottle) lack distinct geometric fiducials and are detected using deep neural networks: 1. **LibTorch Inference:** - Custom YOLO models (`OBJECTDETECT_TORCH_MODEL`) are loaded as TorchScript (.pt) files via `yolomodel::pytorch::PyTorchInterpreter`. - Image tensors are formatted (640x640 letterboxed, RGB, normalized [0.0, 1.0]) and executed on CUDA. 2. **Non-Maximum Suppression (NMS):** - Output tensors containing bounding boxes $[x, y, w, h]$, class IDs, and class confidences are filtered by `OBJECTDETECT_MAINCAM_TORCH_CONFIDENCE` (e.g., 0.60) and merged using `cv::dnn::NMSBoxes` with an IoU threshold (`OBJECTDETECT_MAINCAM_TORCH_NMS_THRESH`, e.g., 0.45). 3. **Multi-Object Tracking:** - Active detections are registered into `tracking::MultiTracker`, maintaining bounding box state during turns or temporary frame drops.

5.3 3. Geolocation Engine (`Geolocate.hpp`)

Converting 2D pixel coordinates (u, v) into 3D global UTM waypoints is performed by `geoloc::GeolocateBox()`:

1. Neighborhood Depth Sampling:

- Extracts a window of size $N \times N$ (default 5×5) centered at (u, v) from the ZED camera's point cloud matrix `cvPointcloud (CV_32FC4)`.
- Invalid coordinates ($Z \leq 0$, NaNs, and infinities) are discarded.

2. Statistical Depth Isolation:

- Rather than computing a simple mean of all depth values (which skews toward background terrain), the Z-depth array is sorted.
- The algorithm selects the 20th percentile depth value:

$$\text{Target } Z = \text{Raw } Z[\lfloor 0.20 \times \text{count} \rfloor]$$

- This isolates the front-facing surface of the object. Points within ± 0.5 meters of this target depth are averaged to determine the camera-relative centroid (X_c, Y_c, Z_c) .

3. Monocular Ground Plane Raycast Fallback:

- If stereovision fails (due to intense specular reflection or uniform texture), the algorithm triggers a geometric pinhole raycast:

$$\begin{aligned} \theta_{\text{ray}} &= \text{atan2}(v_{\text{bottom}} - c_y, f_y) \\ Z_c &= \frac{h_{\text{camera}}}{\tan(\theta_{\text{ray}})} \\ X_c &= \frac{u - c_x}{f_y} \times Z_c, \quad Y_c = -h_{\text{camera}} \end{aligned}$$

4. Global UTM Projection:

- Rover compass heading is converted to standard mathematical radians:

$$\alpha = ((-\theta_{\text{compass}} + 90.0) \pmod{360}) \times \frac{\pi}{180}$$

- Rotates the horizontal vector (X_c, Z_c) by α and translates by camera UTM position (E_c, N_c) :

$$E_{\text{object}} = E_c + (Z_c \cos \alpha + X_c \sin \alpha)$$

$$N_{\text{object}} = N_c + (Z_c \sin \alpha - X_c \cos \alpha)$$

$$\text{Alt}_{\text{object}} = \text{Alt}_c + Y_c$$

5. Radius Estimation:

- Computes Euclidean distance from each filtered point to the centroid, extracts the median distance, and applies an empirical scaling factor of 1.5 to calculate the object's clearance radius.
-

5.4 4. Inputs, Outputs, and Limitations

5.4.1 Inputs

- RGB color frames (`cv::Mat` or `cv::cuda::GpuMat`) from ZED 2i or WebRTC simulation.
- 3D spatial point cloud (`cv::Mat` formatted as `CV_32FC4`).
- Rover pose (`geoops::RoverPose`) including Easting, Northing, Altitude, and fused compass heading.

5.4.2 Outputs

- `geoops::Waypoint`: Complete global target coordinate with UTM position, target classification type (`WaypointType`), and clearance radius.
- Pixel centroid, bounding box, relative distance, and yaw offset angle.

5.4.3 Operational Limitations

- **Stereo Depth Degradation**: Stereoscopic depth error scales quadratically with distance. Objects beyond 15 to 20 meters produce higher geolocation uncertainty.
- **Direct Sunlight Specular Glare**: Can wash out ArUco marker contrast, requiring the rover to rely on YOLO bounding-box visual servoing until close enough for corner extraction.
- **Rotational Blur**: High angular turn rates cause pixel smearing across the CMOS sensor. The drive kinematics damp turning rates during active tracking to preserve frame sharpness.

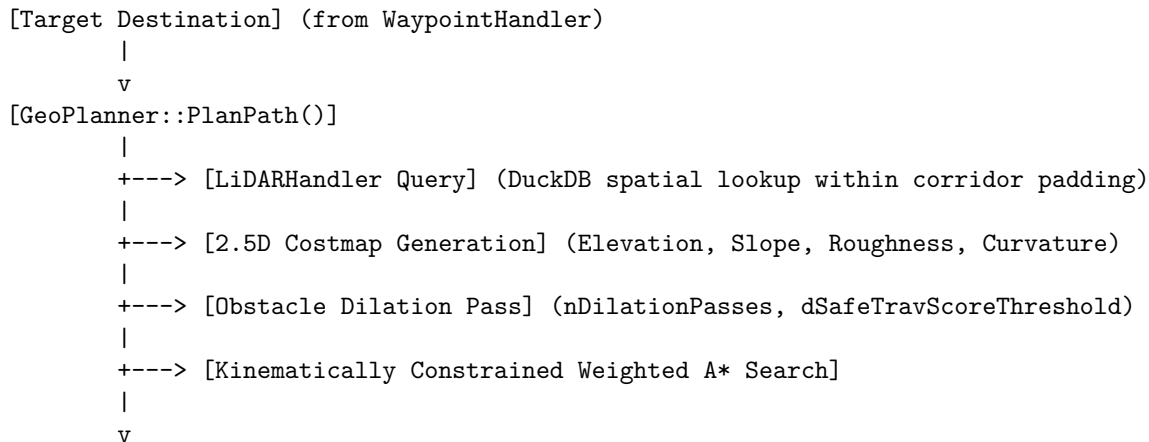
ewpage

6 Path Planning Subsystem

The Path Planning subsystem determines collision-free, kinematically viable trajectories from the rover's current global position to target waypoints across complex terrain.

6.1 1. Algorithmic Architecture

Path planning is orchestrated through two primary components: the **GeoPlanner** (for global terrain traversal) and **SearchPattern** (for localized target search).



[Path Post-Processing] (SplicePath, Waypoint Tolerance Pruning)
 |
 v
 [Ordered Waypoint Path] (std::vector<geoops::Waypoint>)

6.2 2. Global Terrain Planning: GeoPlanner

The `GeoPlanner` (`src/algorithms/planners/GeoPlanner.cpp`) is a specialized geospatial path planner designed for rough natural environments:

6.2.1 A. 2.5D Costmap Generation

Rather than assuming a flat 2D plane with binary open/closed cells, `GeoPlanner` constructs a continuous 2.5D costmap using preprocessed USGS LiDAR data from `LiDARHandler` (sourced from the team's [USGS Data repository](#)):

- **Terrain Metrics:** Each spatial cell evaluates local surface normal vectors (N_x, N_y, N_z), slope gradient, surface roughness, and curvature.
- **Traversal Score:** A composite traversal score (0.0 = impassable cliff/boulder, 1.0 = flat open ground) is assigned to each cell. Cells with scores below `dMinTravScore` are marked non-traversable.
- **Obstacle Dilation:** To prevent the rover chassis from clipping edges, non-traversable cells undergo multiple morphological dilation passes (`nDilationPasses`, default 2), expanding obstacles by an inflation margin.

6.2.2 B. Weighted A* with Kinematic Constraints

- **Search Grid Resolution:** Evaluated on discrete grid tiles (`dGridResolution` = 0.5 meters, `dTileSize` = 50.0 meters).
- **Corridor Padding:** To keep compute times bounded, DuckDB queries only pull points within a corridor (`dCorridorPadding` = 100.0 meters) along the direct line between start and goal.
- **Cost Function:**

$$f(n) = g(n) + w_h \cdot h(n) + \beta \cdot \text{Cost}_{\text{terrain}}(n)$$

- $g(n)$: Distance traveled from start.
- $h(n)$: Euclidean distance heuristic to the goal, scaled by heuristic weight ($w_h = 1.5$) for faster convergence.
- $\text{Cost}_{\text{terrain}}(n)$: Non-linear penalty term scaled by `dPenaltyScalingFactor` and `dPenaltyPower` to actively penalize rough ground even when traversable.
- β (`dBetaBias`): Tuning weight balancing shortest path distance against terrain smoothness.
- **Kinematic Constraints:** The planner checks turning angle delta between sequential nodes, penalizing sharp turns that exceed the skid-steer chassis's lateral turning dynamics.

6.2.3 C. Tile Management and Caching

To maintain high runtime performance:

- `GeoPlanner` caches evaluated grid tiles in memory.
- When traversing long distances, distant tiles can be cleared using `UnloadLiDARTiles()` or `ClearGeoCache()`.

[!TIP] Route Pre-Planning & Inspection Mission routes, waypoint sequences, and A* navigation splines can be validated and previewed using the hosted [Autonomy Task Visualizer](#). Underlying point cloud terrain tiles and slope hazards can be inspected in 3D using the [LiDAR Tool](#), both part of the hosted [MRDT Visualizer Suite](#).

6.3 3. Localized Search Patterns (`SearchPattern.hpp`)

When the rover reaches the vicinity coordinate of an ArUco post or ground object but does not detect it, the state machine enters `eSearchPattern`. `SearchPattern` mathematically constructs structured search paths:

1. **Two-Phase Archimedean Spiral (CalculateSpiralPatternWaypoints):**

- **Heading Initialization:** The starting angle is aligned with the rover's current compass heading:

$$\theta_0 = -\text{Heading}_{\text{degrees}} \times \frac{\pi}{180}$$

- **Phase 1: Outward Spiral (Expansion):** Generates an expanding Archimedean spiral around origin (E_0, N_0) :

$$r(\theta) = \frac{d_{\text{spacing}}}{2\pi} \cdot (\theta - \theta_0)$$

$$E(\theta) = E_0 + d_{\text{windup}} \cos \theta, \quad N(\theta) = N_0 + d_{\text{windup}} \sin \theta$$

Angular step size is governed by `constants::SEARCH_ANGULAR_STEP_DEGREES` (typically 15.0°), with radial arm separation controlled by `constants::SEARCH_SPIRAL_SPACING` (typically 2.0 m). Outward generation continues until reaching the designated search radius R .

- **Phase 2: Inward Spiral (Return Sweep):** Upon reaching the outer boundary R , the algorithm immediately generates an inward spiral winding back toward the origin until $r \geq 0.5$ m and radial spacing wind-up reaches 0.0:

$$d_{\text{windup}} \leftarrow d_{\text{windup}} - d_{\text{spacing}}$$

This inward sweep provides a continuous second-chance search pass and guides the rover back to the vicinity center without leaving it stranded at the outer perimeter.

- **Dual-Path Splitting in SearchPatternState:** After filtering red-zone terrain and passing through `GeoPlanSearchPattern()`, the planned trajectory is split into two halves:
 - **Forward Spiral (vFirstHalf):** Stored in `WaypointHandler` as "GeoPlannerPath", assigned to `PurePursuitController`.
 - **Reverse Return Spiral (vSecondHalf):** Cached in `WaypointHandler` as "GeoPlannerPathReverse". If the outward leg completes without acquiring the target, the state machine transitions `m_eCurrentSearchPatternType` to `SearchPatternType::END`, retrieves "GeoPlannerPathReverse", promotes it to "GeoPlannerPath", and navigates back to center.
- **Completion Safeguard:** To prevent false search pattern completion (which can occur if the rover's start position passes within the completion radius of the origin early in the maneuver), `bReachedFinalTarget` is guarded by target index verification:

$$\text{TargetIndex} > \text{size}(v_{\text{SearchPath}}) - 4$$

Only when the lookahead tracker has actively traversed through to the final segments of the path is `eSearchFailed` permitted to trigger.

2. **ZigZag / Lawnmower Pattern:**

- Generates alternating parallel transects spaced by `constants::SEARCH_ZIGZAG_SPACING`.
- Used in directional terrain features (e.g., canyon floors or ridgelines).

3. **Snake Pattern:**

- Curved sinusoidal sweep pattern controlled by `constants::SEARCH_SNAKE_SLITHERS`.

6.4 4. Path Splicing and Dynamic Recovery (StuckState.cpp)

If the rover encounters an unmapped obstruction or becomes stuck during transit: - **Obstacle Injection (DeclareObstacle):** When `StuckState::Start()` initiates, it computes an obstacle position projected `constants::STUCK_OBSTACLE_DISTANCE` (default 1.0 m) ahead along the rover's current heading:

$$E_{\text{obs}} = E_{\text{rover}} + d_{\text{obs}} \cos(\theta), \quad N_{\text{obs}} = N_{\text{rover}} + d_{\text{obs}} \sin(\theta)$$

This obstacle is permanently recorded in `WaypointHandler` with radius `constants::STUCK_OBSTACLE_RADIUS` (default 2.0 m). - **Recovery Maneuvers:** The rover executes staged directional reversals (`eReverseCurrentHeading`, `eReverseLeft`, `eReverseRight`). Once displacement from the stuck origin exceeds `constants::STUCK_SAME_POINT_PROXIMITY`

(default 0.5 m), the state machine dispatches `Event::eUnstuck`, invoking `ModifyPath()`. - **Dynamic Path Splicing (SplicePath):** 1. **Direct Cache Modification:** Splicing directly modifies "GeoPlannerPath" in place (and also splices "GeoPlannerPathReverse" if recovering during `SearchPatternState`), eliminating legacy intermediate path keys ("stuckPath", "unstuckPath", "RevSpiralPath"). 2. **Boundary Safe-guards:** - Verifies `GetObstaclesCount() > 0` before querying obstacle records. - Retrieves the most recently added obstacle at index `GetObstaclesCount() - 1`. - Strictly preserves the final destination waypoint: `it != std::prev(vPath.end())` prevents goal point excision. 3. **Node Removal and GeoPlanner Re-route:** - Waypoint nodes falling within $(E - E_{\text{obs}})^2 + (N - N_{\text{obs}})^2 \leq R_{\text{obs}}^2$ are excised via `vPath.erase()`. - When leaving the obstacle zone, `GeoPlanner::PlanPath()` generates a connecting detour between the last valid waypoint before the obstacle and the first valid waypoint beyond it. - **Head-Deletion Handling:** If the very first node of the path is within the obstacle radius, `stStartCoordinate` automatically falls back to `stCurrentRoverPose.GetUTMCoordinate()`. - Iterator advancement correctly skips over newly inserted detour nodes (`vSplicePathCoordinates.size() - 2`), preventing duplicate processing or iterator invalidation.

6.5 5. Inputs, Outputs, and Known Constraints

6.5.1 Inputs

- Start UTM Coordinate (`geoops::UTMCoordinate`).
- Goal Waypoint Coordinate (`geoops::Waypoint`).
- USGS LAS DuckDB runtime handler pointer (`LiDARHandler*`).
- Search radius and corridor boundaries.

6.5.2 Outputs

- An ordered `std::vector<geoops::Waypoint>` representing the sequential navigation points.

6.5.3 Operational Constraints

- **Search Timeouts:** Global path calculation is bounded by `dMaxSearchTimeSeconds` (default 120.0 seconds). If terrain geometry creates an impenetrable barrier, the planner aborts rather than freezing the application.
- **Grid Resolution vs Compute:** Reducing `dGridResolution` below 0.25 meters dramatically increases open-set node evaluations. A resolution of 0.5 meters provides optimal balance between path fidelity and real-time responsiveness.

ewpage

7 Control and Actuation Subsystem

The Control and Actuation subsystem translates spatial navigation targets and steering setpoints into physical motor commands, accounting for skid-steer kinematics, ground friction, and chassis pitch/roll limits.

7.1 1. Algorithmic Architecture

The control pipeline translates target headings and velocities into dual-track motor commands:

```
[Path / Steering Setpoint]
    |
    v
[Lateral Controller Selection]
- PIDController (Heading tracking)
```

- PredictiveStanleyController (Cross-track & heading tracking with Unicycle prediction)
- PurePursuitController (Lookahead circle-arc pursuit)

|
v

[Inverse Kinematics] (DifferentialDrive.hpp)

- Deadband Filtering (0.02 threshold, rescaled to full dynamic range)
- Arcade Drive or Curvature Drive Models
- Input Squaring (DRIVE_SQUARE_CONTROL_INPUTS)

|
v

[Safety & Terrain Multipliers] (DriveBoard.cpp)

- Inclinometer Slope Damping (Pitch & Roll weighting)
- Master Throttle Scaling (SETMAXSPEED callback)
- Maximum Safe Power Clamp (DRIVE_MAX_SAFE_POWER)

|
v

[Network Actuation]

- RoveComm UDP Packet (DRIVELEFTRIGHT) -> Core Board Microcontroller

7.2 2. Lateral Tracking Controllers

The autonomy software includes three lateral controllers:

7.2.1 1. PID Controller (PIDController.cpp)

Used for orienting the rover toward single waypoints, search legs, or during visual servoing: - Evaluates heading error: $e_\theta = \theta_{\text{goal}} - \theta_{\text{actual}}$. - Continuous angle wraparound (0° to 360°) prevents $350^\circ \rightarrow 10^\circ$ boundary wraps from triggering full reverse spins. - Output produces a normalized turn effort ($u \in [-1.0, 1.0]$).

7.2.2 2. Predictive Stanley Controller (PredictiveStanleyController.cpp)

Used for following continuous, multi-waypoint paths generated by GeoPlanner: - Simultaneously minimizes **heading error** (θ_e) and **cross-track error** (e_{ct} , perpendicular distance to the reference path line). - Implements a unicycle kinematic prediction model (UnicycleModel.hpp) projecting the rover state N timesteps into the future to compensate for chassis mass and actuation lag.

7.2.3 3. Pure Pursuit Controller (PurePursuitController.cpp)

Alternative geometric path follower: - Identifies a lookahead waypoint at a defined lookahead distance (L_d) along the path. - Calculates the constant curvature arc required to reach that point from the rover's current pose.

7.3 3. Differential Drive Kinematics (DifferentialDrive.hpp)

Because the rover uses skid-steer (tank-style) drive rather than Ackermann steering, turning requires independent velocity control of the left and right wheel sets.

7.3.1 A. Deadband and Input Scaling

Inputs with absolute magnitude < 0.02 are zeroed. Values above the deadband are rescaled so the remaining interval maps smoothly across $[0.0, 1.0]$, preventing sudden motor jump at low throttle.

7.3.2 B. Arcade Drive Kinematics

Maps forward speed v and rotation rate ω :

$$\text{Left Power} = v + \omega$$

$$\text{Right Power} = v - \omega$$

Powers are normalized if either exceeds ± 1.0 :

$$\text{Power}_{\max} = \max(|\text{Left}|, |\text{Right}|, 1.0)$$
$$\text{Left} = \frac{\text{Left}}{\text{Power}_{\max}}, \quad \text{Right} = \frac{\text{Right}}{\text{Power}_{\max}}$$

7.3.3 C. Curvature Drive Kinematics

Controls the radius of curvature rather than raw turning rate: - At higher speeds, steering sensitivity is dynamically scaled down to prevent violent rollovers. - When forward velocity is near zero, point-turning is permitted if `DRIVE_CURVATURE_KINEMATICS_ALLOW_TURN_WHILE_STOPPED` is true.

7.3.4 D. Squared Input Sensitivity

When `DRIVE_SQUARE_CONTROL_INPUTS` is enabled, input magnitudes are squared while preserving sign:

$$u_{\text{squared}} = \text{sgn}(u) \cdot u^2$$

This provides fine, granular steering control at low speeds while retaining full power at maximum deflection.

7.4 4. Drive Board Safety and Terrain Multipliers (`DriveBoard.cpp`)

Raw kinematic commands pass through multi-layered safety conditioning before transmission:

7.4.1 Inclinometer Slope Damping

The `DriveBoard` subscribes to telemetry from the rover's onboard inclinometer: - Computes effective slope angle ϕ as a weighted combination of roll and pitch:

$$\phi = w_{\text{roll}} \cdot |\text{Roll}| + w_{\text{pitch}} \cdot |\text{Pitch}|$$

(Roll is weighted higher because skid-steer rovers are more susceptible to lateral roll-overs). - If $\phi < \text{constants::DRIVE_BOARD_MIN_SLOPE}$ (e.g., 10°), damping multiplier is 1.0 (no attenuation). - If $\phi \geq \text{constants::DRIVE_BOARD_MAX_SLOPE}$ (e.g., 30°), multiplier clamps to `constants::DRIVE\_BOARD\_MIN\_DAMP` (e.g., 0.50). - Between these bounds, a linear interpolation scales the maximum allowed drive power down smoothly.

7.4.2 Master Throttle Control

A `RoveComm` UDP callback listens for `SETMAXSPEED` packets from the Basestation GUI, allowing operators to scale rover velocity across $[0.0, 1.0]$ in real time.

7.4.3 Hard Safety Power Ceiling

All final track outputs are clamped to `constants::DRIVE\_MAX\_SAFE\_POWER` (e.g., 0.90) to prevent motor stall overcurrent and fuse trips.

7.5 5. Inputs, Outputs, and Network Actuation

7.5.1 Inputs

- Target heading and speed setpoints from active state or path follower.
- Actual compass heading and velocity from `NavigationBoard`.
- Inclinometer pitch/roll packets over `RoveComm`.

7.5.2 Outputs

- `manifest::Core::COMMANDS["DRIVELEFTRIGHT"]`: `RoveComm` UDP packet containing two 32-bit floats $[P_{\text{left}}, P_{\text{right}}] \in [-1.0, 1.0]$.

ewpage

8 PID Controller

The `PIDController` class (`src/algorithms/controllers/PIDController.h`) implements a Proportional-Integral-Derivative controller with Feedforward support, anti-windup limits, continuous input wraparound, output slew rate limiting, and output low-pass filtering.

8.1 1. Primary Use Cases

The primary application in Autonomy Software is **Heading and Steering Control**: - When turning the rover toward a goal waypoint or orienting the chassis toward an ArUco marker, the difference between goal heading and current heading is evaluated as an error signal. - The PID controller outputs a normalized rotational effort $u \in [-1.0, 1.0]$ passed to the differential drive kinematics.

8.2 2. Mathematical Formulation

At discrete timestep k with time delta $\Delta t = t_k - t_{k-1}$, the control signal $u(k)$ is computed as:

$$u(k) = u_P(k) + u_I(k) + u_D(k) + u_{FF}(k)$$

8.2.1 Component Breakdown

1. Proportional Term (u_P):

$$u_P(k) = K_p \cdot e(k)$$

Provides immediate corrective action proportional to instantaneous error $e(k) = r(k) - y(k)$ (where r is the setpoint and y is the process variable).

2. Integral Term (u_I):

$$u_I(k) = u_I(k-1) + K_i \cdot e(k) \cdot \Delta t$$

Accumulates steady-state error over time. This term is critical for overcoming static ground friction in skid-steer systems, where small proportional errors fail to produce enough torque to initiate turning.

3. Derivative Term (u_D):

$$u_D(k) = K_d \cdot \frac{e(k) - e(k-1)}{\Delta t}$$

Measures error rate of change to provide damping as the error approaches zero, counteracting overshoot and oscillation.

4. Feedforward Term (u_{FF}):

$$u_{FF}(k) = K_{ff} \cdot r(k)$$

Provides baseline output effort driven directly by the setpoint value rather than the error signal.

8.3 3. Specialized Robotics Features

The `PIDController` class includes several features designed for physical ground robots:

8.3.1 Continuous Input Wraparound

Compass headings wrap from 360° to 0° . Without handling, navigating from 355° to 5° would compute an error of -350° , causing a full counter-clockwise rotation instead of a 10° clockwise turn. - Calling `EnableContinuousInput(0.0, 360.0)` automatically detects the shortest angular distance across the boundary.

8.3.2 Integral Windup Prevention

If the rover is physically obstructed, the integral term can accumulate unbounded error, causing massive overshoot or violent motor spin once the obstacle clears. - `SetMaxIntegralEffort(double dMaxEffort)` clamps the maximum contribution of u_I :

$$|u_I(k)| \leq \text{constants::DRIVE_PID_MAX_INTEGRAL_TERM}$$

8.3.3 Output Slew Rate Limiting (Ramp Rate)

Instantaneous step changes from 0.0 to 1.0 effort can strip motor gearbox teeth or trigger overcurrent cutoffs. - `SetOutputRampRate(double dMaxRatePerSecond)` limits the rate of change of the output:

$$|u(k) - u(k-1)| \leq \text{constants::DRIVE_PID_MAX_RAMP_RATE} \cdot \Delta t$$

8.3.4 Output Low-Pass Filter

Noisy IMU data can cause high-frequency derivative chatter. - `SetOutputFilter(double dFilterAlpha)` applies an exponential moving average to smooth output signals before passing them to motor drivers:

$$u_{\text{filtered}}(k) = \alpha \cdot u(k) + (1 - \alpha) \cdot u_{\text{filtered}}(k-1)$$

8.4 4. Tuning Parameters in `AutonomyConstants.cpp`

Constant Name	Type	Purpose	Tuning Directive
<code>DRIVE_PID_PROPORTIONAL</code>	<code>double</code>	K_p gain	Increase for faster heading response; decrease if the rover oscillates around the setpoint.
<code>DRIVE_PID_INTEGRAL</code>	<code>double</code>	K_i gain	Increase if the rover stalls before finishing a turn; decrease if slow hunting oscillations occur.

Constant Name	Type	Purpose	Tuning Directive
DRIVE_PID_DERIVATIVE	double	K_d gain	Increase to damp overshoot; decrease if high-frequency jitter occurs due to network/actuation delay.
DRIVE_PID_FEEDFORWARD	double	K_{ff} gain	Baseline effort scaling; typically 0.0 for pure heading tracking.
DRIVE_PID_MAX_INTEGRAL_TERM	double	Ceiling on u_I	Clamps integral effort to prevent windup during extended stalls.
DRIVE_PID_MAX_RAMP_RATE	double	Output slew limit	Caps maximum acceleration of commanded effort per second.
DRIVE_PID_OUTPUT_FILTER	double	Filter factor α	Controls output smoothing against IMU noise.
DRIVE_PID_TOLERANCE	double	Deadband tolerance	Error threshold within which the controller declares alignment achieved.

ewpage

9 Predictive Stanley Controller

The `PredictiveStanleyController` (`src/algorithms/controllers/PredictiveStanleyController.h`) implements an advanced lateral path-tracking controller based on the Stanley method, augmented with a kinematic prediction model to compensate for rover mass, skid-steer slip, and actuation latency.

9.1 1. Motivation: Stanley vs Pure Heading Control

Standard heading PID controllers orient the rover toward a target point, but cannot independently regulate the lateral offset from a curving reference path. If a rover deviates laterally, a pure heading controller only steers toward the next node, often resulting in path-cutting, corner-clipping, and lateral drift.

The Stanley Controller simultaneously minimizes two independent error terms: 1. **Heading Error** (θ_e): The angular difference between the rover's heading and the tangent of the nearest path segment. 2. **Cross-Track Error** (e_{ct}): The perpendicular distance from the center of the rover to the reference path.

9.2 2. Mathematical Formulation

9.2.1 Standard Stanley Control Law

For a front-steered vehicle at forward velocity v , the classic Stanley steering angle $\delta(t)$ is defined as:

$$\delta(t) = \theta_e(t) + \arctan\left(\frac{k \cdot e_{ct}(t)}{v(t) + v_{\text{soft}}}\right)$$

- k (`dControlGain`): Cross-track control gain determining correction aggressiveness.
- v_{soft} (`STANLEY_MIN_STABLE_SPEED`): Softening velocity parameter preventing numerical divergence or erratic steering at near-zero forward speeds.

9.2.2 Predictive Horizon with Unicycle Kinematics

Because a 50 kg skid-steer rover cannot instantaneously translate or rotate, applying the steering law strictly to the rover's *current* coordinates results in overshoot and oscillation around the path.

The `PredictiveStanleyController` integrates a **Unicycle Kinematic Model** (`src/algorithms/kinematics/UnicycleModel`) to evaluate error over a prediction horizon: 1. **Forward State Prediction**: Using the rover's current linear velocity v and angular velocity ω , the Unicycle model simulates state forward across N steps (`STANLEY_PREDICTION_HORIZON`, default 5) with time step Δt (`STANLEY_PREDICTION_TIME_STEP`, default 0.01 seconds):

$$\begin{aligned}x_{k+1} &= x_k + v \cos(\theta_k) \cdot \Delta t \\y_{k+1} &= y_k + v \sin(\theta_k) \cdot \Delta t \\\theta_{k+1} &= \theta_k + \omega \cdot \Delta t\end{aligned}$$

2. **Projected Error Evaluation**: The cross-track error e_{ct} and heading error θ_e are evaluated against the predicted future pose (x_N, y_N, θ_N) rather than the present pose. 3. **Angular Velocity Clamping**: The commanded steering rate is clamped by `STANLEY_ANGULAR_VELOCITY_LIMIT` (e.g., 90.0°/s) to prevent skid-steer track slip from exceeding the adhesion limit of the terrain.

9.3 3. Output Data Structure: DriveVector

The controller returns a `PredictiveStanleyController::DriveVector` struct:

```
struct DriveVector
{
    double dThetaHeading; // Target absolute compass heading setpoint
    double dVelocity;      // Target linear velocity
};
```

This output is fed directly into `DriveBoard::CalculateMove()`, which uses the heading PID controller and differential drive kinematics to actuate the left and right tracks.

9.4 4. Tuning Constants in `AutonomyConstants.cpp`

Constant Name	Value	Purpose and Tuning Directive
<code>STANLEY_CROSSTRACK_CONTROL_GAIN</code>	0.1	Cross-track error scaling (k). Higher values pull the rover toward the path more aggressively but can induce weave oscillations.
<code>STANLEY_ANGULAR_VELOCITY_LIMIT</code>	90.0	Maximum turning rate allowed (deg/s). Prevents track slip on loose dirt or sand.
<code>STANLEY_PREDICTION_HORIZON</code>	5	Number of discrete forward simulation steps evaluated by the Unicycle model.
<code>STANLEY_PREDICTION_TIME_STEP</code>	0.01	Integration timestep (seconds) per prediction step.

Constant Name	Value	Purpose and Tuning Directive
STANLEY_MIN_STABLE_SPEED	0.1	Softening constant v_{soft} in denominator (m/s). Prevents division by zero when stopped.
STANLEY_WHEELBASE	0.8	Distance between front and rear axle centers in meters.

9.5 5. Implementation Safeguards

- **Minimum Path Node Requirement:** The `Calculate()` method enforces that the reference path contains at least 2 points (`m_vReferencePath.size() < 2`). If the reference path has 0 or 1 waypoint, `Calculate()` logs a warning:

`PredictiveStanleyController::Calculate: Reference path has fewer than 2 points. Cannot calculate drive vector`
and returns `DriveVector{0.0, 0.0}`. This prevents segmentation faults and undefined behavior when evaluating line segment tangents or cross-track projections near the terminal end of a path.

9.6 6. Usage Example

```
// Set the reference path generated by GeoPlanner
m_StanleyController.SetReferencePath(vGeoPlannerPath);

// Inside the navigation loop:
geoops::RoverPose stPose = globals::g_pStateMachineHandler->SmartRetrieveRoverPose();
PredictiveStanleyController::DriveVector stVector = m_StanleyController.Calculate(stPose, constants::NAV_STANLEY_MAX_STEERING_ANGLE);

// Pass resulting heading setpoint and speed to DriveBoard kinematics
globals::g_pDriveBoard->CalculateMove(stVector.dVelocity, stVector.dThetaHeading, stPose.GetCompassHeading());
globals::g_pDriveBoard->SendDrive();
```

ewpage

10 Pure Pursuit Controller

The `PurePursuitController` class (`src/algorithms/controllers/PurePursuitController.h`) provides an alternative geometric path-tracking controller that calculates the steering angle required to pursue a lookahead waypoint located a set distance down the reference path.

10.1 1. Overview and Theory

Pure Pursuit is a widely established geometric tracking algorithm. Unlike PID heading control (which aims at the nearest node) or Stanley control (which evaluates perpendicular cross-track error to the nearest line segment), Pure Pursuit fits a circular arc between the rover's current pose and a dynamic "lookahead" point on the reference trajectory.

10.1.1 Core Principles

- **Lookahead Anchor:** The controller searches ahead along the path for a target coordinate positioned at a distance L_d (the lookahead distance) from the rover.
- **Curvature Fitting:** It calculates the radius of curvature R of the circular arc connecting the center of the rover to the lookahead point.
- **Steering Setpoint:** The heading setpoint points tangent to this circular arc, smoothing sharp corners and naturally anticipating upcoming bends in the path.

10.2 2. Mathematical Implementation

1. **Closest Waypoint Search:** The controller first determines the nearest waypoint index to the rover's current UTM position using Euclidean distance:

$$i_{\text{closest}} = \arg \min_i \sqrt{(E_{\text{rover}} - E_i)^2 + (N_{\text{rover}} - N_i)^2}$$

2. **Lookahead Waypoint Selection:** Starting from i_{closest} , the algorithm iterates forward along the path segments until it locates the first point where distance from the rover exceeds the lookahead threshold:

$$\text{dist}(\text{Rover}, \text{Waypoint}_j) \geq L_d$$

If discrete waypoint spacing is large, it interpolates along the segment between waypoints to locate the exact intersection with the circle of radius L_d centered at the rover.

3. **Heading Calculation:** The target heading θ_{target} is the bearing from the rover's current UTM position to the lookahead coordinate $(E_{\text{lookahead}}, N_{\text{lookahead}})$:

$$\theta_{\text{target}} = \text{atan2}(E_{\text{lookahead}} - E_{\text{rover}}, N_{\text{lookahead}} - N_{\text{rover}}) \times \frac{180}{\pi}$$

(adjusted to standard clockwise compass degrees where North is 0°).

4. **End-of-Path Deceleration and Stop:** When approaching the terminal path segment ($i \geq \text{size} - 2$), the controller projects the rover onto the final segment vector. If the normalized projection reaches or exceeds 1.0, or if the Euclidean distance to the final node is < 0.5 meters, the controller commands zero velocity to cleanly halt the rover.

10.3 3. Configuration Parameters

The controller exposes constructor arguments and runtime mutators:

Parameter	Default	Purpose and Impact
dLookaheadDistance	2.0 meters	Distance along the path where the target point is selected. Increasing L_d results in smoother trajectories but cuts corners. Decreasing L_d tracks the path tighter but can induce lateral oscillations.
nLookaheadIndex	5	Fallback index offset when distance-based lookahead search reaches path limits.
dMaxSpeed	constants::NAVIGATING_MOTOR_POWER	Maximum linear velocity ceiling commanded by the controller.

10.4 4. Output: DriveVector

The controller returns a `PurePursuitController::DriveVector` struct:

```
struct DriveVector
{
    double dThetaHeading; // Target absolute compass heading setpoint (degrees)
    double dVelocity;      // Target forward velocity (-1.0 to 1.0)
};
```

10.5 5. Implementation Safeguards

- **Minimum Path Node Requirement:** The `Calculate()` method enforces that the reference path contains at least 2 points (`m_vReferencePath.size() < 2`). If the reference path has 0 or 1 waypoint, `Calculate()` logs a warning:

`PurePursuitController::Calculate: Reference path has fewer than 2 points. Cannot calculate drive p`

and returns `DriveVector{0.0, 0.0}`. This prevents undefined behavior or segmentation faults when evaluating terminal path segments, calculating projection vectors (`stLastPoint` and `stSecondToLastPoint`), or computing lookahead intersections on degenerate paths.

10.6 6. Usage Example

```
// Instantiate with a 2.5 meter lookahead distance
controllers::PurePursuitController controller(2.5, 5);

// Set reference path from GeoPlanner
controller.SetReferencePath(vGeoPlannerPath);

// Execute within state machine loop
geoops::RoverPose stPose = globals::g_pStateMachineHandler->SmartRetrieveRoverPose();
controllers::PurePursuitController::DriveVector stVector = controller.Calculate(stPose, constants::NAVI

// Pass setpoints to DriveBoard kinematics
globals::g_pDriveBoard->CalculateMove(stVector.dVelocity, stVector.dThetaHeading, stPose.GetCompassHead
globals::g_pDriveBoard->SendDrive();
```

ewpage

11 Cameras

The Autonomy Software relies on the `CameraHandler` to manage physical and virtual video streams. The vision system supports **Stereolabs ZED Stereoscopic Cameras** and **Basic USB Webcams**, with dedicated simulation interfaces for offline testing.

11.1 1. ZED Stereoscopic Camera Interface (`ZEDCamera.hpp` & `ZEDCam.cpp`)

The primary optical sensors mounted on the rover are Stereolabs ZED 2i cameras (Head Main Camera and optional Rear Camera). The `ZEDCam` class wraps the native Stereolabs C++ SDK (`sl::Camera`).

11.1.1 Capabilities and Outputs

1. **High-Definition RGB Frames:** Standard color imagery captured at configurable resolutions (HD720, HD1080) and framerates (typically 30 or 60 FPS).
2. **Dense Depth Maps:** CV_32FC4 or half-precision floating point matrices where each pixel corresponds to an (X, Y, Z) coordinate relative to the optical center in meters.
3. **Spatial Mapping:** Continuous 3D voxel mesh reconstruction of the terrain (`sl::Mesh`), exported as .ply files upon shutdown if enabled.
4. **Visual-Inertial Positional Tracking:** Tracks 6-DoF chassis movement using fused visual odometry and internal IMU measurements.
5. **Sensor Telemetry:** Real-time extraction of linear acceleration, angular velocity, and magnetic heading via `sl::SensorsData`.

11.1.2 Hardware vs Simulation Architecture

- **Hardware Mode (ZEDCam.cpp):** Communicates with physical ZED 2i cameras via USB 3.0. Supports zero-copy GPU memory sharing via `cv::cuda::GpuMat` to feed CUDA-based PyTorch/YOLO inference without round-tripping to CPU RAM.
- **Simulation Mode (SIMZEDCam.cpp):** When `BUILD_SIM_MODE` is enabled, `CameraHandler` instantiates `SIMZEDCam` instead of `ZEDCam`. It connects to Unreal Engine RoveSoSimulator via WebRTC video tracks (`LibDataChannel`) and decodes H.264 video streams into RGB and depth matrices, matching physical camera APIs.

11.2 2. Asynchronous Frame Retrieval Architecture

To prevent high-latency operations (such as deep learning inference or GUI streaming) from blocking the high-frequency camera capture loop, all frame retrieval methods are asynchronous and return `std::future<bool>`:

```
// 1. Request an RGB color frame into a local buffer
cv::Mat cvColorFrame;
std::future<bool> fuFrameReady = pMainCam->RequestFrameCopy(cvColorFrame);

// 2. Request a 3D Point Cloud matrix
cv::Mat cvPointCloud;
std::future<bool> fuPointcloudReady = pMainCam->RequestPointCloudCopy(cvPointCloud);

// 3. Request IMU sensor telemetry
sl::SensorsData slSensors;
std::future<bool> fuSensorsReady = pMainCam->RequestSensorsCopy(slSensors);

// 4. Await completion before reading buffers
if (fuFrameReady.get() && fuPointcloudReady.get())
{
    // Process cvColorFrame and cvPointCloud safely
}
```

Behind the scenes: - `m_qFrameCopySchedule` queues incoming subscriber requests. - An internal thread pool (`BS::thread_pool`) processes the queue, copying data into destination buffers in parallel.

11.3 3. Basic Camera Interface (BasicCamera.hpp & BasicCam.cpp)

For non-stereoscopic tasks (such as inspecting ground clearance, verifying robotic arm end-effectors, or streaming auxiliary web feeds), the software uses `BasicCam`: - Wraps OpenCV's `cv::VideoCapture` for

standard V4L2 USB cameras on Linux. - In simulation mode, `SIMBasicCam` receives virtual feeds via WebRTC channels. - Employs identical asynchronous `RequestFrameCopy()` semantics, ensuring consistent consumer APIs across all camera types.

11.4 4. Key Configuration Parameters in `AutonomyConstants.cpp`

Constant Name	Default	Purpose
<code>ZED_MAINCAM_RESOLUTIONX / Y</code>	1280 / 720	Native camera resolution (HD720).
<code>ZED_MAINCAM_FPS</code>	30	Capture framerate target.
<code>ZED_COORD_SYSTEM</code>	<code>LEFT_HANDED_Y_UP</code>	Native coordinate convention (+X Right, +Y Up, +Z Forward).
<code>ZED_DEPTH_MODE</code>	<code>ULTRA / NEURAL</code>	Depth reconstruction algorithm. <code>NEURAL</code> is higher accuracy; <code>ULTRA</code> consumes less GPU power.
<code>ZED_MAINCAM_USE_GPU_MAT</code>	<code>false</code>	When true, buffers are maintained in CUDA memory (<code>cv::cuda::GpuMat</code>).
<code>ZED_MAINCAM_FRAME_RETRIEVAL_THREADS</code>		Thread pool worker count for servicing parallel frame copy requests.
<code>ZED_MAINCAM_SERIAL</code>	0	Hardware serial number to differentiate head and rear cameras on USB bus.

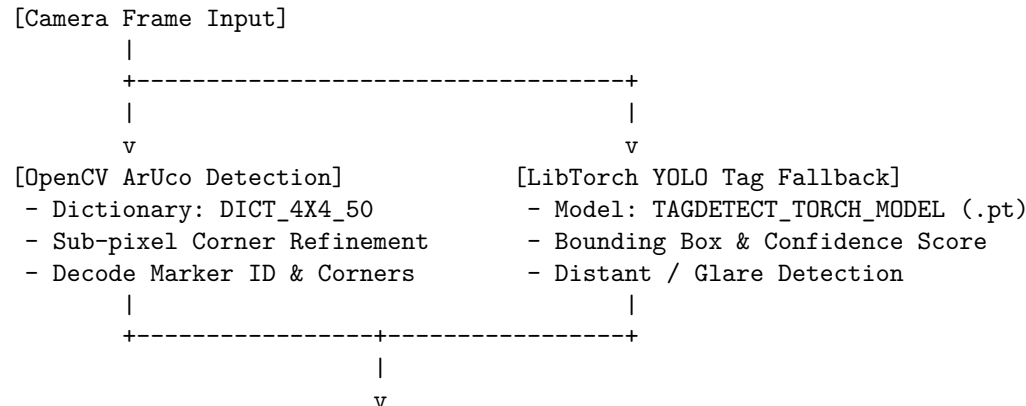
ewpage

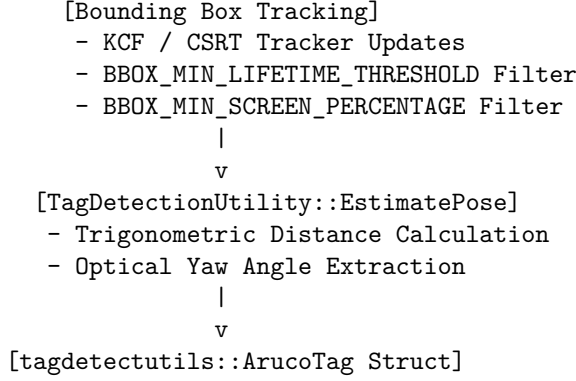
12 ArUco Tag Detection

During competitive missions, the rover must autonomously locate, identify, and drive toward fiducial markers (AR Tags) mounted on target posts across the course.

12.1 1. The `TagDetector` Pipeline (`TagDetector.cpp`)

The `TagDetector` class inherits from `AutonomyThread<void>` and executes continuous image processing on incoming frames from its assigned camera:





12.2 2. Detection Methods

12.2.1 1. Classical OpenCV ArUco

- **Dictionary:** `cv::aruco::DICT_4X4_50` matching official URC specifications.
- **Corner Refinement:** Uses `cv::aruco::CORNER_REFINE_SUBPIX` to pinpoint tag corner vertices at sub-pixel accuracy.
- **Inverted Marker Detection:** Toggled by `constants::TAGDETECT_MAINCAM_DETECT_INVERTED_MARKER` to detect markers in harsh shadow or backlighting.

12.2.2 2. LibTorch YOLO Fallback

When distance exceeds 10 meters, dust occludes corners, or direct sunlight washes out the tag face, classical ArUco fails to detect the geometric square. - A custom YOLO neural network trained on marker silhouettes runs via LibTorch (`yolomodel::pytorch::PyTorchInterpreter`). - If YOLO detects a tag bounding box with confidence $\geq$ `constants::TAGDETECT_MAINCAM_TORCH_CONFIDENCE`, the rover begins approaching the candidate blob using visual servoing until close enough for OpenCV to decode the exact integer ID.

12.2.3 3. Temporal Validation and Tracking

Visual noise and random terrain patterns can produce instantaneous false positive detections. - Before a tag is marked valid by `TagDetectionChecker::IdentifyTargetMarker()`, its bounding box must occupy at least `constants::BBOX_MIN_SCREEN_PERCENTAGE` of the camera image and persist for at least `constants::BBOX_MIN_LIFETIME_THRESHOLD` (typically 0.5 seconds). - Active locks are tracked between neural network inferences using OpenCV KCF or CSRT trackers.

12.3 3. Pose Estimation (`TagDetectionUtility.hpp`)

Knowing a tag exists in frame is insufficient; the control system requires the straight-line distance and the horizontal yaw angle between the camera optical axis and the marker:

1. **Tag Corner Geometry:** The physical width of the marker is known: `constants::ARUCO_TAG_SIDE_LENGTH` (default 0.20 meters).
2. **Trigonometric Distance Calculation:** Given horizontal camera field of view FOV_h , image width W , and pixel width of the detected marker w_{px} :

$$\text{Apparent Width} = \frac{w_{px}}{W}$$

$$d_{\text{straight}} = \frac{\text{ARUCO_TAG_SIDE_LENGTH}}{2 \cdot \tan\left(\frac{\text{FOV}_h \cdot \text{Apparent Width}}{2}\right)}$$

3. **Yaw Offset Angle:** Given tag bounding box center x_{center} :

$$\text{Offset Ratio} = \frac{x_{\text{center}} - \frac{W}{2}}{\frac{W}{2}}$$

$$\theta_{\text{yaw}} = \text{Offset Ratio} \times \frac{\text{FOV}_h}{2}$$

- If $\theta_{\text{yaw}} > 0$, the tag is to the right of the optical axis.
- If $\theta_{\text{yaw}} < 0$, the tag is to the left of the optical axis.

12.4 4. Usage in State Machine

Inside **ApproachingMarkerState**: - Visual servoing feeds θ_{yaw} into the heading PID controller, commanding point-turns or curved approaches to center the tag in the frame. - Forward speed is modulated based on d_{straight} . - When $d_{\text{straight}} \leq \text{constants::APPROACH_MARKER_PROXIMITY_THRESHOLD}$ (e.g., 2.0 meters), the state machine triggers **Event::eReachedMarker** to transition to **eVerifyingMarker**.

ewpage

13 Object Detection

The **ObjectDetector** class (`src/vision/objects/ObjectDetector.cpp`) detects and tracks non-fiducial competition props, including mallets, rock picks, and water bottles.

13.1 1. Deep Learning Pipeline: LibTorch YOLO

Unlike fiducial markers with geometric patterns, natural ground props require convolutional neural networks for robust classification under variable desert lighting.

```
[Raw Camera Image] (cv::Mat, 1280x720)
|
v
[Preprocessing] (yolomodel::pytorch::PyTorchInterpreter)
- Resize / Letterbox to 640x640
- Normalize channels to [0.0, 1.0]
- Convert to CUDA FloatTensor [1, 3, 640, 640]
|
v
[Inference on GPU] (LibTorch torch::jit::load)
- Model: OBJECTDETECT_TORCH_MODEL (.torchscript)
- BMP v6 (Baseline): v8s_x640_150epochs_augment/best.torchscript
- BMP v7 (Tucumcari): v8s_x640_100epochs_augment/best_tucumcari_arugmented_model.torchscript
|
v
[Post-Processing]
- Confidence Filter (OBJECTDETECT_MAINCAM_TORCH_CONFIDENCE)
- Non-Maximum Suppression (cv::dnn::NMSBoxes)
|
v
```

```

[Tracking & Temporal Validation]
- OpenCV CSRT / KCF MultiTracker
- BBOX_MIN_LIFETIME_THRESHOLD Filter
- BBOX_MIN_SCREEN_PERCENTAGE Filter
  |
  v
[3D Point Cloud Geolocation]
- GeolocateBox() against ZED Point Cloud
  |
  v
[objectdetectutils::Object Struct]

```

13.1.1 Supported TorchScript Models

- **BMP v6 Baseline** (`bmp_v6/v8s_x640_150epochs_augment/best.torchscript`): YOLOv8s trained for 150 epochs with standard photometric augmentation.
- **BMP v7 Tucumcari Augmented** (`bmp_v7/v8s_x640_100epochs_augment/best_tucumcari_arugmented_model.to`): YOLOv8s trained for 100 epochs with specialized desert terrain data augmentation specifically captured for the Tucumcari competition site, optimizing detection under extreme midday sunlight and shadows.

13.2 2. Target Classification and Parsing

The system detects three primary competition classes: - **Mallet**: Orange rubber mallet (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::MALLETT`). - **Water Bottle**: 1-liter plastic bottle (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::WATERBOTTLE`). - **Rock Pick**: Geologist rock hammer (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::ROCKPICK`).

When evaluating detections in `ObjectDetectionChecker::IdentifyTargetObject()`: 1. Active detections are matched against the target class requested by the current waypoint leg. 2. If multiple instances appear, the candidate with the highest screen area percentage is selected. 3. Candidate objects must exceed `constants::BBOX_MIN_LIFETIME_THRESHOLD` to eliminate transient false positives.

13.3 3. 3D Geolocation Integration

Because competition props vary in dimensions and orientation, estimating distance via 2D pinhole trigonometry is prone to error. The `ObjectDetector` resolves physical location by pairing 2D bounding boxes with the ZED 3D point cloud:

1. Bounding box center coordinates (u_c, v_c) are extracted from the detection.
2. The coordinate is passed to `geoloc::GeolocateBox()` along with the synchronized `CV_32FC4` point cloud matrix and the fused rover pose.
3. `GeolocateBox()` queries a 5x5 neighborhood around (u_c, v_c), sorts the depth values, and computes the 20th percentile surface depth to isolate the object face from the desert ground behind it.
4. The localized 3D point (X_c, Y_c, Z_c) is rotated by the rover compass heading and translated by the rover UTM position, generating an absolute `geoops::Waypoint`.

13.4 4. Usage in State Machine

During mission execution: - In `eNavigating` or `eSearchPattern`, `ObjectDetectionChecker` monitors for target detections. - Upon confirming a valid object, the state machine triggers `Event::eObjectSeen` and transitions to `eApproachingObject`. - The rover visual-servos toward the object until distance drops below `constants::APPROACH_OBJECT_PROXIMITY_THRESHOLD`. - The state machine triggers

`Event::eReachedObject`, transitioning to `eVerifyingObject` to halt, confirm the detection hit-rate over time, and signal the C2 station.

ewpage

14 Vision Utilities

The `src/util/vision/` directory contains specialized mathematical, image-processing, and neural-network helper utilities supporting the computer vision subsystems.

14.1 1. YOLOModel.hpp

This utility encapsulates the LibTorch C++ API, providing high-level loading, tensor conversion, and inference execution for YOLO models (`.pt` TorchScript).

14.1.1 Key Features

- **Device Management:** Automatically selects between CUDA hardware acceleration (`HardwareDevices::eCUDA`) and host CPU execution (`HardwareDevices::eCPU`).
 - **Tensor Formatting:** Converts OpenCV image matrices (`cv::Mat`) to normalized floating-point PyTorch tensors with shape $[1, 3, H, W]$, handling color space transformation (BGR to RGB) and memory alignment.
 - **Output Parsing:** Translates multi-dimensional output tensors into structured `yolomodel::Detection` objects containing class indices, confidence scores, and `cv::Rect` bounding boxes.
 - **Non-Maximum Suppression:** Wraps OpenCV's `cv::dnn::NMSBoxes` to eliminate redundant bounding boxes based on IoU overlap.
-

14.2 2. BoundingBoxTracking.h & BoundingBoxTracking.cpp

Neural network inference on high-resolution frames requires significant GPU cycles. To maintain high tracking rates while keeping compute loads manageable, the system employs **OpenCV Multi-Object Tracking**.

14.2.1 Pipeline

1. When YOLO detects an object, a tracker instance (KCF or CSRT) is initialized on the detected bounding box.
 2. On subsequent camera frames, the tracker follows the visual features within the bounding box without executing full neural network inference.
 3. Trackers are maintained until:
 - The object leaves the camera field of view.
 - Tracking is lost for longer than `constants::BBOX_TRACKER_LOST_TIMEOUT`.
 - Continuous tracking exceeds `constants::BBOX_TRACKER_MAX_TRACK_TIME`, forcing a neural network re-evaluation.
 4. When a new neural network inference completes, overlapping tracker boxes are reconciled using Intersection-over-Union (IoU) matching (`BBOX_TRACKER_IOU_MATCH_THRESHOLD`).
-

14.3 3. Geolocate.hpp

Provides the `geoloc::GeolocateBox()` function, which bridges the 2D optical frame and the 3D UTM global frame: - **Neighborhood Depth Sampling:** Evaluates an $N \times N$ pixel window around a detected object

centroid within the ZED camera's CV_32FC4 point cloud. - **20th Percentile Depth Isolation:** Filters background terrain points to measure the distance to the front surface of the object. - **Monocular Ground Plane Raycast Fallback:** If depth data is missing (due to glare or occlusion), it executes a pinhole geometric raycast using known camera mounting height and pitch angle. - **UTM Frame Projection:** Rotates the camera-relative vector by the rover's compass heading and adds the camera's current UTM position to produce a `geoops::Waypoint`.

14.4 4. TagDetectionUtility.hpp & ObjectDetectionUtility.hpp

- **TagDetectionUtility.hpp:**
 - Provides `EstimatePoseFromCameraFrame()`, which computes straight-line distance and optical yaw angle from tag pixel dimensions, camera resolution, and horizontal field of view.
 - Generates debug visualization overlays with corner outlines, marker IDs, and distance text.
 - **ObjectDetectionUtility.hpp:**
 - Provides drawing and debug overlay routines for YOLO detections.
 - Implements helper routines for isolating specific target classes (mallet, rock pick, water bottle) from raw multi-class detection vectors.
-

14.5 5. ImageOperations.hpp & FetchContainers.hpp

- **ImageOperations.hpp:**
 - Provides fast image manipulation routines, including letterboxing (preserving aspect ratio during resizing to 640x640), matrix cropping, colorspace conversions, and CPU-to-GPU matrix transfers (`cv::cuda::GpuMat`).
- **FetchContainers.hpp:**
 - Defines thread-safe template wrappers (`containers::FrameFetchContainer<T>`) pairing image matrices with `std::promise<bool>` and `std::future<bool>`.
 - Enables asynchronous frame retrieval pipelines across threads without blocking capture loops.

ewpage

15 State Machine Handler

The `StateMachineHandler` (`src/handlers/StateMachineHandler.h` & `StateMachineHandler.cpp`) manages active state execution, evaluates event-driven transitions, coordinates recovery fallbacks, and executes sensor fusion for the Autonomy Software.

15.1 1. Primary Responsibilities

1. **State Lifecycle Execution:** Instantiates, runs, and terminates concrete `statemachine::State` objects derived from `src/interfaces/State.hpp`.
2. **Event Dispatch and Transitions:** Handles incoming `statemachine::Event` triggers, halts motor outputs for safety, queries the active state for the subsequent state, and manages transitions.
3. **State Preservation and Recall:** Maintains `m_umSavedStates` (`std::unordered_map<statemachine::States, std::shared_ptr<statemachine::State>>`). When an interruption occurs (such as getting stuck or initiating a reverse maneuver), the active state can be saved and restored once recovery completes.
4. **Sensor Fusion Engine:** Implements `SmartRetrieveRoverPose()`, combining low-rate absolute GPS/magnetometer heading from `NavigationBoard` with high-rate visual-inertial odometry from `ZEDCamera`.

5. **RoveComm Telemetry:** Broadcasts `manifest::Autonomy::TELEMETRY["CURRENTSTATE"]` UDP packets on every state change to keep the Basestation GUI synchronized.
-

15.2 2. Event Handling and State Transition Sequence

When a state transition is commanded via `HandleEvent(statemachine::Event eEvent, bool bSaveCurrentState):`

```
[Event Triggered] (HandleEvent called)
|
v
[Lock Event Mutex] (m_muEventMutex)
|
v
[Safety Stop Command] (globals::g_pDriveBoard->SendStop())
|
v
[Query Active State] (eNextState = m_pCurrentState->TriggerEvent(eEvent))
|
v
[ChangeState(eNextState, bSaveCurrentState)]
|
+----> [Lock State Mutex] (m_muStateMutex)
+----> [Set Switching Flag] (m_bSwitchingStates = true)
+----> [Save Current State if requested] (m_umSavedStates[state] = m_pCurrentState)
+----> [Check Saved States Map]
|       |
|       +---> Found: Restore preserved state object
|       +---> Not Found: Create fresh state via CreateState()
|
+----> [Clear Switching Flag] (m_bSwitchingStates = false)
+----> [Broadcast Telemetry] (RoveComm CURRENTSTATE packet)
```

15.3 3. Sensor Fusion: SmartRetrieveRoverPose()

Compass magnetometers on electric rovers are susceptible to magnetic interference caused by high motor currents. Conversely, pure visual odometry drifts over time. `SmartRetrieveRoverPose()` fuses both sources:

1. **Direct GPS and Magnetometer Ingestion:** Reads latitude, longitude, and compass heading from `globals::g_pNavigationBoard`.
2. **Dynamic Realignment Conditions:** When the rover is in `eIdle`, or when driving straight at steady speed ($|v| > \text{constants::ZED_REALIGN_VEL_THRESH}$ and $|\omega| < \text{constants::ZED_REALIGN_ROT_THRESH}$):
 - Queries ZED IMU Euler yaw via `m_pMainCam->RequestSensorsCopy()`.
 - Computes offset:

$$\text{Offset} = \text{Heading}_{\text{GPS}} - \text{Heading}_{\text{Raw ZED}}$$

3. **Fused Heading Output:** During turns or high motor throttle, the system applies the calibrated offset to high-frequency ZED IMU readings:

$$\text{Heading}_{\text{Fused}} = (\text{Heading}_{\text{Raw ZED}} + \text{Offset}) \pmod{360}$$

This eliminates heading jumps caused by magnetic spikes from drive motors.

15.4 4. Concurrency and Thread Safety

- **AutonomyThread Base:** Runs continuously in `ThreadedContinuousCode()` at a rate capped by `constants::STATEMACHINE_MAX_IPS` (default 60 Hz).
 - **Double Mutex Protection:**
 - `m_muEventMutex`: Serializes event processing so multiple asynchronous events (e.g., vision detection and GPS arrival) do not race.
 - `m_muStateMutex`: Prevents state logic from executing while `m_pCurrentState` pointers are being swapped.
 - **Atomic State Guard:** `m_bSwitchingStates` is set to true during transitions, preventing `ThreadedContinuousCode()` from executing methods on half-constructed state objects.
-

15.5 5. Public Interface Summary

```
// Lifecycle
void StartStateMachine();
void StopStateMachine();

// Transitions
void HandleEvent(statemachine::Event eEvent, const bool bSaveCurrentState = false);

// State Inspection & Cache
statemachine::States GetCurrentState() const;
statemachine::States GetPreviousState() const;
void ClearSavedStates();
void ClearSavedState(statemachine::States eState);

// Pose & Telemetry Fusion
geoops::RoverPose SmartRetrieveRoverPose(bool bIMUHeading = true);
double SmartRetrieveVelocity();
double SmartRetrieveAngularVelocity();
void RealignZEDHeading(const double dNewActualHeading, const double dCurrentZEDHeading);
```

ewpage

16 Camera Handler

The `CameraHandler` class (`src/handlers/CameraHandler.h` & `CameraHandler.cpp`) manages camera hardware lifecycle, initializes camera worker threads, provides centralized access to video feeds, and controls asynchronous video recording.

16.1 1. Primary Responsibilities

1. **Hardware Detection and Configuration:** Detects, instantiates, and starts camera objects based on compilation flags (`BUILD_SIM_MODE`) and configuration settings (`MODE_REAR_ZED`).
2. **Global Feed Registry:** Exposes thread-safe getters (`GetZED()`, `GetBasicCam()`) accessible via `globals::g_pCameraHandler` to allow detector handlers to retrieve shared pointers to active camera streams.
3. **Simulation Abstraction:** Automatically instantiates `SIMZEDCam` (WebRTC / `LibDataChannel`) when `BUILD_SIM_MODE` is enabled, or physical `ZEDCam` (ZED SDK 4.x) when running on physical hardware.

4. **Recording Coordination:** Spawns an internal `RecordingHandler` instance to write raw camera feeds to disk without blocking computer vision processing.
-

16.2 2. Managed Cameras

The handler manages cameras designated by `ZEDCamName` and `BasicCamName` enumerations:

- **`ZEDCamName::eHeadMainCam`:** The forward-facing ZED 2i stereoscopic camera mounted on the rover mast. Used as the primary feed for ArUco tag detection, YOLO object detection, visual odometry, and 3D geolocation.
 - **`ZEDCamName::eRearCam`:** An optional rear-facing ZED stereoscopic camera (enabled when `constants::MODE_REAR_ZED` is true). Used for reversing maneuvers and rear situational awareness.
 - **`BasicCamName`:** Extensible interface for standard V4L2 USB cameras or virtual simulation webcams (`BasicCam` / `SIMBasicCam`).
-

16.3 3. Concurrency and Integration

- Each camera managed by `CameraHandler` inherits from `AutonomyThread<void>`.
 - Frame acquisition runs on an independent background thread at the hardware framerate (30 or 60 FPS).
 - Downstream modules request data using future-based asynchronous methods (`RequestFrameCopy()`, `RequestDepthCopy()`, `RequestPointCloudCopy()`, `RequestSensorsCopy()`), preventing downstream inference latency from stalling hardware capture.
-

16.4 4. Usage Example

```
// Startup sequence (in main.cpp)
globals::g_pCameraHandler = new CameraHandler();
globals::g_pCameraHandler->StartAllCameras();
globals::g_pCameraHandler->StartRecording();

// Accessing cameras in downstream modules:
std::shared_ptr<ZEDCamera> pMainCam = globals::g_pCameraHandler->GetZED(CameraHandler::ZEDCamName::eHeadMainCam);

// Asynchronously request color frame and point cloud
cv::Mat cvFrame;
cv::Mat cvPointCloud;
std::future<bool> fuFrame = pMainCam->RequestFrameCopy(cvFrame);
std::future<bool> fuCloud = pMainCam->RequestPointCloudCopy(cvPointCloud);

if (fuFrame.get() && fuCloud.get())
{
    // Execute computer vision and 3D geolocation
}
```

ewpage

17 Tag Detection Handler

The `TagDetectionHandler` (`src/handlers/TagDetectionHandler.h` & `TagDetectionHandler.cpp`) orchestrates all ArUco marker detection pipelines across active camera feeds.

17.1 1. Primary Responsibilities

1. **Detector Lifecycle Management:** Instantiates and initializes `TagDetector` worker threads for assigned cameras (`eHeadMainCam`, `eRearCam`).
 2. **Dual-Model Fusion:** Configures detectors to run classical OpenCV ArUco decoding in parallel with LibTorch YOLO marker candidate detection.
 3. **Debug Overlay Streaming:** Generates annotated image frames (`RequestDetectionOverlayFrame()`) containing marker bounding boxes, coordinate axes, IDs, and estimated distances for transmission to the Basestation GUI or WebRTC stream.
 4. **Synchronized Video Recording:** Houses an internal `RecordingHandler` configured in `RecordingType::eTagDetectionHandler` mode to save annotated detection feeds to disk.
-

17.2 2. Managed Detectors

The handler provides access to detectors via the `TagDetectors` enumeration: - `TagDetectors::eHeadMainCam`: Primary detector analyzing frames from the forward mast camera. - `TagDetectors::eRearCam`: Secondary detector monitoring the rear camera feed when enabled.

17.3 3. Concurrency and Integration

- Each `TagDetector` runs as an independent `AutonomyThread<void>`.
 - Frame pulling from `CameraHandler` is asynchronous.
 - Detections are cached thread-safely in `tagdetectutils::ArucoTag` structs with creation timestamps, enabling `TagDetectionChecker` to evaluate lifetime persistence before triggering state machine transitions.
-

17.4 4. Usage Example

```
// Initialization in main.cpp
globals::g_pTagDetectionHandler = new TagDetectionHandler();
globals::g_pTagDetectionHandler->StartAllDetectors();
globals::g_pTagDetectionHandler->StartRecording();

// Querying detection overlay for UI streaming:
cv::Mat cvAnnotatedFrame = globals::g_pTagDetectionHandler->RequestDetectionOverlayFrame(
    TagDetectionHandler::TagDetectors::eHeadMainCam
);
```

ewpage

18 Object Detection Handler

The `ObjectDetectionHandler` (`src/handlers/ObjectDetectionHandler.h` & `ObjectDetectionHandler.cpp`) orchestrates all deep-learning prop and obstacle detection pipelines across active camera feeds.

18.1 1. Primary Responsibilities

1. **Model Loading and Management:** Loads custom LibTorch YOLO models (`OBJECTDETECT_TORCH_MODEL`) onto GPU memory via CUDA (supporting both baseline `bmp_v6` and competition-tuned `bmp_v7` Tucumcari weights).
 2. **Detector Lifecycle Management:** Instantiates and initializes `ObjectDetector` instances for assigned cameras (`eHeadMainCam`, `eRearCam`).
 3. **Bounding Box Tracking Integration:** Coordinates OpenCV CSRT/KCF multi-object trackers between neural network inferences to reduce compute load.
 4. **Debug Overlay Streaming:** Generates annotated frames (`RequestDetectionOverlayFrame()`) displaying bounding boxes, class labels, and confidence scores.
 5. **Video Recording:** Houses an internal `RecordingHandler` configured in `RecordingType::eObjectDetectionHandler` mode to record annotated detection video.
-

18.2 2. Managed Detectors

Access to detector instances is provided via the `ObjectDetectors` enumeration: - `ObjectDetectors::eHeadMainCam`: Primary detector analyzing the forward camera stream. - `ObjectDetectors::eRearCam`: Secondary detector analyzing the rear camera stream when enabled.

18.3 3. Concurrency and Integration

- Each `ObjectDetector` executes on its own `AutonomyThread<void>`.
 - Inferences run asynchronously at rates up to `constants::OBJECTDETECT_MAINCAM_MAX_FPS`.
 - Detected props are cached thread-safely in `objectdetectutils::Object` structs, which `ObjectDetectionChecker` evaluates against requested leg types (Mallet, Water Bottle, Rock Pick).
-

18.4 4. Usage Example

```
// Initialization in main.cpp
globals::g_pObjectDetectionHandler = new ObjectDetectionHandler();
globals::g_pObjectDetectionHandler->StartAllDetectors();
globals::g_pObjectDetectionHandler->StartRecording();

// Querying detection overlay for UI streaming:
cv::Mat cvAnnotatedFrame = globals::g_pObjectDetectionHandler->RequestDetectionOverlayFrame(
    ObjectDetectionHandler::ObjectDetectors::eHeadMainCam
);
```

ewpage

19 LiDAR Handler

The `LiDARHandler` (`src/handlers/LiDARHandler.h` & `LiDARHandler.cpp`) manages runtime spatial queries against preprocessed LiDAR point cloud databases, providing real-time terrain topology and obstacle metrics to `GeoPlanner` and `VisualizationHandler`.

19.1 1. Primary Responsibilities

1. **DuckDB Database Interfacing:** Connects to pre-built DuckDB database files (`constants::LIDAR_HANDLER_DB_PATH`) containing millions of geospatial points derived from USGS 3DEP LAS 1.4 point clouds.
 2. **Radial Spatial Lookups:** Executes fast spatial queries to extract terrain points within a specified radius R of an Easting/Northing coordinate.
 3. **Multi-Property Filtering:** Supports filtering points by classification (e.g., ground vs vegetation vs structures), surface normal vectors, terrain slope, roughness, and curvature.
 4. **Traversal Metrics Provisioning:** Provides precomputed traversal scores to the `GeoPlanner` costmap generator.
-

19.2 2. Point Data Architecture

Points are stored and returned in the `LiDARHandler::PointRow` structure:

```
struct PointRow
{
    int nID; // Unique point identifier
    double dEasting; // UTM Easting coordinate (meters)
    double dNorthing; // UTM Northing coordinate (meters)
    double dAltitude; // Altitude above sea level (meters)
    std::string szZone; // UTM Zone designator
    std::string szClassification; // Point classification (ground, rock, etc.)
    double dNormalX; // X component of local surface normal vector
    double dNormalY; // Y component of local surface normal vector
    double dNormalZ; // Z component of local surface normal vector
    double dSlope; // Surface slope (degrees)
    double dRoughness; // Local terrain roughness metric
    double dCurvature; // Surface curvature metric
    double dTraversalScore; // Composite score [0.0 = impassable, 1.0 = smooth]
};
```

19.2.1 Filtering via PointFilter

Queries can be conditioned using `LiDARHandler::PointFilter`, specifying min/max bounds on slope, roughness, curvature, and normal vectors to isolate specific terrain hazards.

19.3 3. Database Engine: DuckDB

The handler leverages **DuckDB** rather than traditional relational engines: - **Columnar Execution Engine:** Optimized for analytical vectorized queries on large numerical datasets. - **Embedded Operation:** Runs in-process without requiring background server daemons. - **Thread Safety:** Uses `std::shared_mutex` to allow concurrent read queries across `GeoPlanner` and `VisualizationHandler` threads.

19.4 4. Usage Example

```
// Opening the database during initialization (in main.cpp)
globals::g_pLiDARHandler = new LiDARHandler();
if (!globals::g_pLiDARHandler->OpenDB(constants::LIDAR_HANDLER_DB_PATH))
{
    LOG_ERROR(logging::g_qSharedLogger, "Failed to open LiDAR DuckDB database.");
}

// Querying terrain within a 15-meter radius of the rover
std::vector<LiDARHandler::PointRow> vNearbyPoints;
vNearbyPoints = globals::g_pLiDARHandler->GetPointsInRadius(stRoverUTM, 15.0);

// Filtering for steep obstacles (slope > 25 degrees)
LiDARHandler::PointFilter stFilter;
stFilter.dEasting = stRoverUTM.dEasting;
stFilter.dNorthing = stRoverUTM.dNorthing;
stFilter.dRadius = 20.0;
stFilter.dSlope = LiDARHandler::PointFilter::Range<double>{25.0, 90.0};

std::vector<LiDARHandler::PointRow> vSteepObstacles;
vSteepObstacles = globals::g_pLiDARHandler->GetPointsWithFilter(stFilter);
```

19.5 5. LiDAR Data Sources & Web Inspection Tools

19.5.1 A. USGS LiDAR Point Cloud Storage Repository

The spatial elevation and terrain point clouds queried by `LiDARHandler` are sourced from the USGS 3D Elevation Program (3DEP) and processed into indexed DuckDB databases. Raw LAS/LAZ point cloud tiles, pre-generated DuckDB database artifacts, and ingestion scripts are hosted on the team's GitLab server: - **USGS LiDAR Dataset Repository:** [MissouriMRDT/USGS_Data](#) - **MRDT GitLab Organization:** [MissouriMRDT GitLab](#)

Developers running simulations or offline tests requiring local terrain maps should acquire the appropriate regional `.duckdb` tiles from `USGS_Data` and place them at the path configured in `constants::LIDAR_HANDLER_DB_PATH` (`data/LiDAR/` by default).

19.5.2 B. Online LiDAR Visualizer Tool

Terrain point clouds, cross-sectional elevation profiles, and traversability slopes can be visualized interactively in the web browser without launching local DuckDB instances: - **Interactive LiDAR Visualizer:** visualizer.themrdr.org/lidar-tool/

This tool supports inspecting 3D colored point distributions, evaluating elevation gradients, and testing traversability threshold configurations across competition terrains.

ewpage

20 Waypoint Handler

The `WaypointHandler` (`src/handlers/WaypointHandler.h` & `WaypointHandler.cpp`) is a thread-safe registry that stores, sequences, and manages mission waypoints, intermediate planned paths, and permanent obstacle coordinates.

20.1 1. Primary Responsibilities

1. **Mission Queue Management:** Maintains the sequential queue of target waypoints (`std::vector<geops::Waypoint>`), commanding autonomous rover movement.
 2. **RoveComm Command Ingestion:** Registers asynchronous callbacks listening for Basestation mission commands (`ADDPPOSITIONLEG`, `ADDMARKERLEG`, `ADDOBJECTLEG`, `CLEARWAYPOINTS`).
 3. **Trajectory and Path Storage:** Provides key-value path caching (`StorePath()`, `RetrievePath()`) allowing `GeoPlanner` to save generated A* routes.
 4. **Obstacle Memory:** Stores global coordinates of declared obstacles (`m_vPermanentObstacles`), ensuring the path planner retains obstacle awareness across mission legs.
-

20.2 2. RoveComm Network Callbacks

The handler intercepts incoming RoveComm packets transmitted by the Basestation GUI:

20.2.1 A. Position Leg (`ADDPPOSITIONLEG`)

- **Data Payload:** [Latitude, Longitude, LegID]
- **Action:** Constructs a `geops::Waypoint` with type `geops::WaypointType::eNavigationWaypoint` and appends it to the queue.

20.2.2 B. Marker Leg (`ADDMARKERLEG`)

- **Data Payload:** [Latitude, Longitude, MarkerID, SearchRadius]
- **Action:** Clamps the search radius between 0 and 40 meters, assigns type `geops::WaypointType::eTagWaypoint`, and appends the marker waypoint with the specified ArUco ID.

20.2.3 C. Object Leg (`ADDOBJECTLEG`)

- **Data Payload:** [Latitude, Longitude, ObjectID, SearchRadius]
- **Action:** Parses `ObjectID` using `manifest::Autonomy::AUTONOMYWAYPOINTTYPES`:
 - `MALLET` → `geops::WaypointType::eMalletWaypoint`
 - `WATERBOTTLE` → `geops::WaypointType::eWaterBottleWaypoint`
 - `ROCKPICK` → `geops::WaypointType::eRockPickWaypoint`
 - Clamps the search radius between 0 and 40 meters and appends the object waypoint.

20.2.4 D. Clear Queue (`CLEARWAYPOINTS`)

- **Action:** Clears the waypoint queue and signals the state machine if an active navigation leg is running.
-

20.3 3. Thread Safety and Concurrency

- Internal vectors are protected by reader-writer locks using `std::shared_mutex`:
 - `m_muWaypointsMutex` protects the mission queue.
 - `m_muPathMutex` protects stored A* paths.
 - `m_muObstaclesMutex` protects declared obstacle coordinates.
 - Multiple threads (such as `GeoPlanner`, `VisualizationHandler`, and `NavigatingState`) can read waypoints simultaneously using `std::shared_lock`, while incoming RoveComm callbacks acquire exclusive `std::unique_lock`.
-

20.4 4. Public Interface Summary

```
// Queue Manipulation
void AddWaypoint(const geoops::Waypoint& stWaypoint);
geoops::Waypoint PeekNextWaypoint();
geoops::Waypoint PopNextWaypoint();
void ClearWaypoints();
int GetWaypointCount();
const std::vector<geoops::Waypoint> GetAllWaypoints();

// Path Storage (GeoPlanner)
void StorePath(const std::string& szPathName, const std::vector<geoops::Waypoint>& vWaypointPath);
const std::vector<geoops::Waypoint> RetrievePath(const std::string& szPathName);

// Obstacle Management
void AddObstacle(const geoops::Waypoint& stObstacle);
const std::vector<geoops::Waypoint> GetAllObstacles();
```

20.5 5. Standardized Path Caching Keys

The WaypointHandler provides key-value storage for computed navigation paths via StorePath(szPathName, vWaypointPath) and RetrievePath(szPathName). The autonomy system standardizes on two primary path keys to coordinate state machine operations:

Path Key Name	Generating / Owning State	Description & Usage
"GeoPlannerPath"	GeoPlanner / SearchPatternState / StuckState	Primary Active Navigation Path. Stores the global A* path generated between the rover's starting pose and the current target destination waypoint. In SearchPatternState, stores the first half (vFirstHalf) of the Archimedean spiral trajectory representing outward expansion. In StuckState, obstacle avoidance splices are applied directly into this path, replacing previous node sequences with obstacle coordinates and rover pose fallback points.

Path Key Name	Generating / Owning State	Description & Usage
"GeoPlannerPathReverse"	SearchPatternState / StuckState	Inward Spiral Sweep Path. Stores the second half (vSecondHalf) of the Archimedean spiral trajectory generated by SearchPattern::GenerateSpiral(). Once the outward expansion is fully traversed, SearchPatternState loads this trajectory into the controller to navigate the rover back to the search pattern origin center. If an obstacle is detected during the reverse inward sweep, StuckState modifies this path directly.

[!NOTE] Legacy Key Deprecation Previous implementations utilized transient path keys such as "stuckPath", "unstuckPath", and "RevSpiralPath". These have been deprecated and removed. All path tracking, obstacle splicing, and state transitions now operate exclusively and symmetrically on "GeoPlannerPath" and "GeoPlannerPathReverse".

ewpage

21 Recording Handler

The RecordingHandler (src/handlers/RecordingHandler.h & RecordingHandler.cpp) manages the asynchronous recording of camera video feeds and computer vision detection overlays directly to disk.

21.1 1. Primary Responsibilities

1. **Stream Recording:** Connects to active camera and detector streams and encodes frames into video files using OpenCV cv::VideoWriter.
2. **Asynchronous Encoding:** Runs on an independent AutonomyThread<void> to prevent video encoding overhead from degrading computer vision and control loop framerates.
3. **Multi-Mode Operation:** Supports three distinct operational modes depending on which handler instantiates it.
4. **Selective Recording Toggles:** Reads individual recording enable flags from AutonomyConstants.cpp to conserve disk space and CPU resources.

21.2 2. Operational Modes (RecordingMode)

```
enum class RecordingMode
{
    eCameraHandler,           // Records raw, unmodified RGB camera feeds
    eTagDetectionHandler,     // Records video feeds with ArUco and YOLO tag detection overlays
    eObjectDetectionHandler   // Records video feeds with YOLO prop detection overlays
};
```

21.2.1 Modes Explained

- **eCameraHandler**: Instantiated inside **CameraHandler**. Directly captures raw frames from **ZEDCamera** and **BasicCamera** streams for ground-truth review and simulation replay.
 - **eTagDetectionHandler**: Instantiated inside **TagDetectionHandler**. Queries **RequestDetectionOverlayFrame()** from each **TagDetector** to capture video showing detected ArUco marker corners, decoded IDs, and estimated distances.
 - **eObjectDetectionHandler**: Instantiated inside **ObjectDetectionHandler**. Captures video showing YOLO object bounding boxes, class labels (Mallet, Bottle, Rock Pick), and confidence scores.
-

21.3 3. Concurrency and Output Formatting

- **Thread Independence**: Inherits from **AutonomyThread<void>**, executing **ThreadedContinuousCode()** at an iteration rate throttled by **constants::RECORDER_FPS** (typically 15 to 30 FPS).
 - **Asynchronous Frame Pulling**: Pushes requests to cameras and detectors via futures (**std::future<bool>**), awaiting data transfer in the background without blocking the capture pipeline.
 - **File Container and Codec**: Video streams are encoded in H.264 / MP4 format and saved within the timestamped mission directory inside **logs/** (e.g., **logs/YYYY-MM-DD_HH-MM-SS/**).
-

21.4 4. Configuration Parameters in **AutonomyConstants.cpp**

Constant Name	Type	Purpose
RECORDER_FPS	int	Framerate ceiling for disk video encoding.
ZED_MAINCAM_ENABLE_RECORDING	bool	Enables raw recording of the main mast ZED camera.
ZED_REARCAM_ENABLE_RECORDING	bool	Enables raw recording of the rear ZED camera.
TAGDETECT_MAINCAM_ENABLE_RECORDING	bool	Enables recording of ArUco detection overlays on main camera.
TAGDETECT_REARCAM_ENABLE_RECORDING	bool	Enables recording of ArUco detection overlays on rear camera.
OBJECTDETECT_MAINCAM_ENABLE_RECORDING	bool	Enables recording of YOLO prop detection overlays on main camera.
OBJECTDETECT_REARCAM_ENABLE_RECORDING	bool	Enables recording of YOLO prop detection overlays on rear camera.

ewpage

22 Drive Board Driver

The **DriveBoard** class (**src/drivers/DriveBoard.h** & **DriveBoard.cpp**) converts high-level speed and steering requests into physical motor powers and transmits them over **RoveComm** to the Core board microcontroller.

22.1 1. Primary Responsibilities

1. **Kinematics Processing:** Accepts linear speed and heading requests and calculates left and right track power percentages using Differential Drive inverse kinematics.
 2. **Network Transmission:** Formats track powers into RoveComm UDP DRIVELEFTRIGHT packets and transmits them to the Core microcontroller.
 3. **Terrain Slope Damping:** Intercepts pitch and roll telemetry from the rover's inclinometer to attenuate motor power on steep inclines, preventing tip-overs.
 4. **Master Throttle Regulation:** Listens for Basestation SETMAXSPEED commands, scaling output powers across $[0.0, 1.0]$.
 5. **Emergency Stop Command:** Provides SendStop() to immediately command 0.0 power across both tracks.
-

22.2 2. Kinematics Pipeline (CalculateMove)

```
void DriveBoard::CalculateMove(double dSpeed, double dGoalHeading, double dActualHeading);
```

The calculation follows three sequential steps: 1. **Heading Error and PID Effort:** Computes the angular delta:

$$\theta_{\text{error}} = \theta_{\text{goal}} - \theta_{\text{actual}}$$

The internal PID controller (DRIVE_PID_*) calculates a normalized rotational turn effort:

$$\omega = \text{PID.Calculate}(\theta_{\text{error}}) \in [-1.0, 1.0]$$

2. **Differential Drive Inverse Kinematics:** Depending on configuration, the forward speed v and turn effort ω are evaluated using: - **Arcade Drive:**

$$\text{Left} = v + \omega, \quad \text{Right} = v - \omega$$

- **Curvature Drive:** Scales turning sensitivity inversely with forward velocity to prevent dynamic rollovers at high speeds. Point-turning is permitted when forward speed is near zero. - Powers are normalized so neither track exceeds ± 1.0 , with optional input squaring (DRIVE_SQUARE_CONTROL_INPUTS). 3. **Terrain Damping Multiplier:** Multiplies raw track powers by VariableDriveEffort() and the global m_dMaxDriveEffort multiplier:

$$P_{\text{final}} = P_{\text{raw}} \cdot \text{Damp}_{\text{slope}} \cdot \text{Multiplier}_{\text{throttle}}$$

Final outputs are clamped to constants::DRIVE_MAX_SAFE_POWER.

22.3 3. Inclinometer Safety Damping (VariableDriveEffort)

The driver registers a RoveComm callback listening for manifest::Core::TELEMETRY["INCLINOMETERDATA"]:

- Extracts chassis Pitch and Roll in degrees. - Computes effective slope angle ϕ :

$$\phi = w_{\text{roll}} \cdot |\text{Roll}| + w_{\text{pitch}} \cdot |\text{Pitch}|$$

where weights are defined by constants::DRIVE_BOARD_ROLL_WEIGHT and constants::DRIVE_BOARD_PITCH_WEIGHT.

- If $\phi \leq \text{constants::DRIVE_BOARD_MIN_SLOPE}$ (10°), damping factor is 1.0. - If $\phi \geq \text{constants::DRIVE_BOARD_MAX_SLOPE}$ (30°), damping factor clamps to constants::DRIVE_BOARD_MIN_DAMP (0.50). - Between these limits, linear interpolation smoothly decreases drive power.

22.4 4. Public Interface Summary

```

// Kinematics and Movement
void CalculateMove(double dSpeed, double dGoalHeading, double dActualHeading);
void SendDrive();
void SendStop();

// Power Inspection & Setters
diffdrive::DrivePowers GetDrivePowers() const;
void SetMaxDriveEffort(const double dMaxDriveEffort);
double GetMaxDriveEffort() const;

// Differential Drive Mode Selection
void SetDifferentialControlMethod(diffdrive::DifferentialControlMethod eMethod);

```

ewpage

23 Navigation Board Driver

The NavigationBoard driver (src/drivers/NavigationBoard.h & NavigationBoard.cpp) interfaces with the rover's GPS receivers, RTK systems, and IMU compass to maintain ground-truth positioning.

23.1 1. Primary Responsibilities

1. **Telemetry Ingestion:** Subscribes to high-frequency RoveComm UDP streams from the physical Navigation Board microcontroller.
2. **Geodetic Projections:** Automatically converts raw WGS84 GPS latitude/longitude/altitude coordinates into Universal Transverse Mercator (UTM) Cartesian coordinates using GeographicLib.
3. **Chassis Antenna Offsets:** Compensates for the physical offset between the GPS antenna mounting location and the rover center of rotation using `constants::NAVBOARD_EASTING_OFFSET`, `NAVBOARD_NORTHING_OFFSET`, and `NAVBOARD_ALTITUDE_OFFSET`.
4. **Kinematic Velocity and Heading Estimation:** Computes linear velocity from sequential GPS positions and angular velocity from sequential IMU heading differentials.
5. **Data Freshness and Failsafe:** Monitors telemetry latency via `IsOutOfDate()`, alerting the state machine if GPS packets drop for longer than `constants::NAVBOARD_MAX_GPS_DATA_AGE`.

23.2 2. Ingested Data Streams

The driver registers RoveComm callbacks for two primary telemetry packets: - **GPSLATLON:** Contains double-precision latitude, longitude, altitude, and fix accuracy metrics. - **IMUDATA:** Contains double-precision compass heading (0° to 360° clockwise from North) and heading accuracy estimate in degrees.

23.3 3. Data Freshness Guard (IsOutOfDate)

GPS antennas can lose satellite lock, and network lines can experience dropped packets. - Every incoming GPS packet updates `m_tmLastGPSUpdateTime`. - Every incoming compass packet updates `m_tmLastCompassUpdateTime`. - The `IsOutOfDate()` method checks:

$$\Delta t_{\text{GPS}} = t_{\text{current}} - t_{\text{last GPS}}$$

$$\Delta t_{\text{compass}} = t_{\text{current}} - t_{\text{last compass}}$$

If $\Delta t_{\text{GPS}} > \text{constants::NAVBOARD_MAX_GPS_DATA_AGE}$ (default 3.0 seconds) or $\Delta t_{\text{compass}} > \text{constants::NAVBOARD_MAX_COMPASS_DATA_AGE}$, `IsOutOfDate()` returns true. - The `NavigatingState` continuously polls `IsOutOfDate()`. If true, the rover halts and logs critical warnings, preventing blind runaway.

23.4 4. Concurrency and Thread Safety

Telemetry arrives on the `RoveCommUDP` background thread while multiple autonomy threads (`StateMachineHandler`, `GeoPlanner`, `VisualizationHandler`, `DriveBoard`) read navigation state simultaneously. - Thread safety is enforced through granular `std::shared_mutex` instances: - `m_muLocationMutex` - `m_muHeadingMutex` - `m_muVelocityMutex` - `m_muAngularVelocityMutex` - Callbacks acquire exclusive unique locks (`std::unique_lock`), while getter methods acquire shared read locks (`std::shared_lock`), ensuring high throughput without data races.

23.5 5. Public Interface Summary

```
// Coordinate Getters
geoops::GPSCoordinate GetGPSData();
geoops::UTMCoordinate GetUTMData();

// Orientation & Kinematics
double GetHeading();
double GetHeadingAccuracy();
double GetVelocity();
double GetAngularVelocity();

// Freshness & Health
std::chrono::system_clock::duration GetGPSLastUpdateTime();
std::chrono::system_clock::duration GetCompassLastUpdateTime();
bool IsOutOfDate();
```

ewpage

24 Multimedia Board Driver

The `MultimediaBoard` driver (`src/drivers/MultimediaBoard.h` & `MultimediaBoard.cpp`) manages the rover's visual signaling hardware, controlling high-intensity LED light strips and indicators to communicate operational status to judges and operators.

24.1 1. Primary Responsibilities

1. **State-Driven Lighting:** Translates high-level autonomy states into competition-compliant LED colors.
 2. **Dual Telemetry and Command Transmission:** Dispatches both a Basestation telemetry packet (`manifest::Autonomy::TELEMETRY["STATEDISPLAY"]`) and a hardware microcontroller command packet (`manifest::Core::COMMANDS["STATEDISPLAY"]`).
 3. **Direct RGB Control:** Provides low-level interfaces (`SendRGB()`) to command custom hexadecimal or RGB values directly.
-

24.2 2. Operational Lighting States

Lighting behavior is governed by the `MultimediaBoardLightingState` enumeration:

Enum State	Commanded Color	Associated Robot Status
<code>eOff</code>	Black / Off [0, 0, 0]	System shutdown, idle standby, or unpowered LEDs.
<code>eAutonomy</code>	Solid Red	Autonomy state machine active and in control of chassis movement.
<code>eTeleOp</code>	Solid Blue	Manual teleoperation active; operator joystick override.
<code>eReachedGoal</code>	Flashing Green	Target waypoint reached, ArUco post verified, or prop detected.
<code>eCustom</code>	User-defined RGB	Diagnostic test patterns or custom animations.

24.3 3. Network Transmission Protocol

When `SendLightingState()` is called: 1. `stTelemPacket` is constructed with Data ID `manifest::Autonomy::TELEMETRY["ST"]` notifying the Basestation GUI to update on-screen indicators. 2. `stCorePacket` is constructed with Data ID `manifest::Core::COMMANDS["STATEDISPLAY"]`, instructing the physical Core microcontroller to toggle LED driver relays or WS2812B strips. 3. Packets are transmitted over UDP via `network::g_pRoveCommUDPNode`. Because lighting commands represent discrete state changes rather than continuous control loops, transmission occurs only on state transitions to conserve network bandwidth.

24.4 4. Public Interface Summary

```
enum class MultimediaBoardLightingState
{
    eOff,
    eTeleOp,
    eAutonomy,
    eReachedGoal,
    eCustom
};

void SendLightingState(MultimediaBoardLightingState eState);
void SendRGB(const RGB& stRGB);
MultimediaBoardLightingState GetCurrentLightingState() const;
```

ewpage

25 AutonomyThread Interface

The `AutonomyThread` template interface (`src/interfaces/AutonomyThread.hpp`) is the foundational concurrency abstraction across the Autonomy Software. It provides standardized thread lifecycle management, rate-limiting, performance instrumentation, and integrated thread-pooling without exposing raw OS thread primitives to derived classes.

25.1 1. Architectural Purpose

Most autonomy modules (cameras, neural network detectors, the state machine, network listeners) require dedicated background loops. Executing these on the main thread would cause catastrophic latency spikes.

Inheriting from `AutonomyThread<T>` provides:

1. **Managed Background Execution:** Safely spawns and oversees an independent OS thread executing `ThreadedContinuousCode()`.
2. **Deterministic IPS Limiting:** Regulates loop execution frequency via high-precision sleep calculations.
3. **Execution Metrics:** Instruments real-time iterations-per-second (IPS) tracking.
4. **Internal Thread Pooling:** Embeds a `BS::thread_pool` for executing parallelized burst tasks (`PooledLinearCode()`).
5. **Thread Prioritization:** Wraps thread scheduling priorities (`AutonomyThreadPriority`) from lowest to highest.
6. **Destructor Safety:** Enforces clean thread joining upon destruction, preventing segmentation faults from orphaned threads during application shutdown.

25.2 2. Core Methods and Overrides

25.2.1 A. Pure Virtual Worker Methods

- `virtual void ThreadedContinuousCode() = 0`: The payload of the continuous loop. Runs inside a `while(!m_bStopThreads)` loop on the background thread.
- `virtual void PooledLinearCode() = 0`: The payload executed by tasks dispatched to the embedded thread pool.

25.2.2 B. Lifecycle and Control

- `Start()`: Spawns the worker thread, transitions state to `AutonomyThreadState::eRunning`, and begins loop execution.
 - `RequestStop()`: Atomically sets `m_bStopThreads = true` and updates state to `eStopping`. The current iteration will complete before the thread terminates.
 - `Join()`: Blocks the calling thread until the worker thread has completely exited.
 - `SetMainThreadIPSLimit(int nMaxIterationsPerSecond)`: Configures the rate ceiling. After each iteration of `ThreadedContinuousCode()`, the elapsed time is measured; if execution finished ahead of the timestep, the thread sleeps for the remainder of the slice.
 - `GetIPS().GetExactIPS()`: Returns the moving-average iterations per second.
-

25.3 3. Thread Priority System (`AutonomyThreadPriority`)

The interface abstracts thread scheduling priority:

```
enum class AutonomyThreadPriority
{
    eLowest    = BS::pr::lowest,    // Scheduled less frequently; yields to other tasks
    eLow       = BS::pr::low,
    eNormal    = BS::pr::normal,    // Default priority
    eHigh      = BS::pr::high,      // Prioritized under system load
    eHighest   = BS::pr::highest    // Highest scheduling priority
};
```

25.4 4. Usage Example

```

// Template parameter defines the return type of pooled tasks (void if unused)
class SensorWatcher : public AutonomyThread<void>
{
public:
    SensorWatcher()
    {
        // Regulate continuous polling to 20 Hz
        SetMainThreadIPSLimit(20);
    }

protected:
    void ThreadedContinuousCode() override
    {
        // Runs continuously on worker thread
        ReadHardware();
        ProcessTelemetry();
    }

    void PooledLinearCode() override
    {
        // Executed by thread pool workers when dispatched
    }
};

// Application usage (in main.cpp)
SensorWatcher watcher;
watcher.Start();

// Shutdown sequence
watcher.RequestStop();
watcher.Join();

```

ewpage

26 Thread Pools

While `AutonomyThread` oversees single persistent background worker loops, the Autonomy Software also requires mechanisms to execute parallelized burst workloads across multiple CPU cores without thread allocation latency.

26.1 1. Engine: `BS::thread_pool`

The codebase utilizes the **Barak Shoshany C++ Thread Pool library** (`BS::thread_pool`), included under `external/threadpool/include/BS_thread_pool.hpp`.

26.1.1 Benefits Over Dynamic Thread Creation

- **Pre-Allocated Worker Threads:** Spawns worker threads once during object initialization.
- **Zero OS Thread Creation Overhead:** Tasks are submitted into a concurrent priority queue, and idle workers immediately claim and execute them.
- **Priority Scheduling:** Tasks can be assigned priorities (`BS::pr::lowest` to `BS::pr::highest`) to ensure time-critical computations bypass routine jobs.

26.2 2. Integrated Pool Methods in AutonomyThread

Every class derived from `AutonomyThread<T>` contains an embedded `BS::thread_pool`. It exposes several protected methods for dispatching parallel work:

26.2.1 A. Batch Execution

- `RunPool(int nTasks, int nThreads, AutonomyThreadPriority ePriority)`: Submits `nTasks` to execute `PooledLinearCode()`. It returns a vector of `std::future<T>`, allowing the caller to collect return values via `GetPoolResults()`.
- `RunDetachedPool(int nTasks, int nThreads, AutonomyThreadPriority ePriority)`: Submits `nTasks` as fire-and-forget executions, bypassing future synchronization for minimal latency.

26.2.2 B. Dynamic Task Submission

- `SubmitTaskToPool(Func&& task, Args&&... args)`: Queues an arbitrary lambda or function pointer to the pool, returning an `std::future` representing its eventual completion.
- `SubmitDetachedTaskToPool(Func&& task, Args&&... args)`: Queues an arbitrary function without allocating a future object.

26.2.3 C. Loop Parallelization (ParallelizeLoop)

Splits large iterative loops across available CPU cores:

```
// Distributes 10,000 iterations across 4 worker threads
this->ParallelizeLoop(4, 10000, [this](const int nStart, const int nEnd) {
    for (int i = nStart; i < nEnd; ++i)
    {
        ProcessDataPoint(i);
    }
});
```

26.3 3. Real-World Application: Multi-Subscriber Frame Copying

The primary consumer of thread pooling in the software is camera buffer distribution (`ZEDCam.cpp` and `BasicCam.cpp`): 1. A physical camera frame is captured on the camera capture thread. 2. Multiple consumer threads (`TagDetector`, `ObjectDetector`, `SimpleWebServer` video streamer) require independent copies of the frame matrix. 3. If the camera thread copied frames sequentially, a slow consumer would block subsequent hardware frame grabs. 4. Instead, the camera pushes a copy task for each active subscriber into its thread pool. Workers execute the matrix copies simultaneously in parallel, allowing the hardware capture loop to immediately fetch the next frame.

ewpage

27 RoveComm Networking Protocol

`RoveComm` is a custom in-house application-layer communication protocol developed by the Mars Rover Design Team. It connects the Autonomy Software (executing on the Jetson computer), distributed embedded microcontrollers (Drive Board, Navigation Board, Multimedia Board), and the Basestation Command and Control (C2) console.

The protocol is included as a git submodule in `external/rovecomm`.

27.1 1. Transport Protocols: UDP vs TCP

RoveComm provides dual transport layers tailored to different telemetry requirements:

27.1.1 A. RoveComm UDP (User Datagram Protocol)

- **Primary Use:** High-rate, periodic, loss-tolerant sensor and actuator telemetry.
- **Examples:**
 - `manifest::Core::COMMANDS["DRIVELEFTRIGHT"]` transmitted at 60 Hz to the motor controller.
 - `manifest::Nav::TELEMETRY["GPSLATLON"]` and `["IMUDATA"]` streaming from the Navigation Board.
 - Periodic lighting commands (`STATEDISPLAY`, `LEDRGB`).
 - Log message streaming to the Basestation console.
- **Behavior:** Socket transmission is non-blocking. If a wireless frame is dropped, subsequent packets overwrite the dropped data without retransmission delays.
- **Binding:** Handled by `network::g_pRoveCommUDPNode` on port `manifest::General::ETHERNET_UDP_PORT` (default 11000).

27.1.2 B. RoveComm TCP (Transmission Control Protocol)

- **Primary Use:** Low-rate, mission-critical, guaranteed-delivery commands.
- **Examples:**
 - Mission leg injections (`ADDPPOSITIONLEG`, `ADDMARKERLEG`, `ADDOBJECTLEG`).
 - Queue clearing commands (`CLEARWAYPOINTS`).
 - Runtime logging level reconfigurations (`SETLOGGINGLEVELS`).
- **Behavior:** Uses stream-based delivery with kernel-level acknowledgments and ordered sequencing.
- **Binding:** Handled by `network::g_pRoveCommTCPNode` bound to `constants::ROVECOMM_TCP_INTERFACE_IP` and `manifest::General::ETHERNET_TCP_PORT` (default 11000).

27.2 2. The RoveComm Manifest

To maintain compatibility between C++ embedded firmware, C++ autonomy software, and Python base station GUI software, all message definitions are centralized in `RoveCommManifest.h` (generated from `manifest.json`).

Each manifest entry defines three fields: 1. **DATA_ID:** A unique 16-bit unsigned integer identifier. 2. **DATA_COUNT:** Expected number of array elements in the payload. 3. **DATA_TYPE:** Primitive data type identifier: - `UINT8_T`, `INT8_T` - `UINT16_T`, `INT16_T` - `UINT32_T`, `INT32_T` - `FLOAT_T` (32-bit IEEE 754) - `DOUBLE_T` (64-bit IEEE 754) - `CHAR_T`

27.3 3. Packet Structure: RoveCommPacket<T>

Network payloads are encapsulated within the templated `RoveCommPacket<T>` struct:

```
template<typename T>
struct RoveCommPacket
{
    uint16_t unDataId;           // Message ID from manifest
    uint16_t unDataCount;       // Array element count
    manifest::DataTypes eDataType; // Data type enum
}
```

```
std::vector<T> vData;           // Payload vector
};
```

When transmitted, network byte order conversions (`htonll`, `ntohl1`) ensure consistent endianness across x86_64 host machines and ARM64 Jetson architectures.

27.4 4. Sending Telemetry and Commands

To transmit a packet, instantiate `RoveCommPacket<T>`, set the manifest parameters, populate `vData`, and dispatch via the node pointer:

```
// Example: Sending motor powers over UDP (from DriveBoard.cpp)
rovecomm::RoveCommPacket<float> stPacket;
stPacket.unDataId    = manifest::Core::COMMANDS.find("DRIVELEFTRIGHT")->second.DATA_ID;
stPacket.unDataCount = manifest::Core::COMMANDS.find("DRIVELEFTRIGHT")->second.DATA_COUNT;
stPacket.eDataType   = manifest::Core::COMMANDS.find("DRIVELEFTRIGHT")->second.DATA_TYPE;

stPacket.vData.emplace_back(fLeftTrackPower);
stPacket.vData.emplace_back(fRightTrackPower);

// Transmit to the Core board IP
network::g_pRoveCommUDPNode->SendUDPPacket(
    stPacket,
    "192.168.1.130",
    constants::ROVECOMM_OUTGOING_UDP_PORT
);
```

27.5 5. Asynchronous Callbacks

Incoming messages are processed asynchronously using callback handlers registered with the UDP or TCP node:

```
// 1. Define callback lambda
const std::function<void(const rovecomm::RoveCommPacket<double>&, const sockaddr_in&)> AddPositionLegCa
    [this](const rovecomm::RoveCommPacket<double>& stPacket, const sockaddr_in& stdAddr)
    {
        double dLat = stPacket.vData[0];
        double dLon = stPacket.vData[1];
        int nLegID  = static_cast<int>(stPacket.vData[2]);

        this->AddWaypoint(geoops::GPSCoordinate(dLat, dLon), geoops::WaypointType::eNavigationWaypoint, 0.0
    };

// 2. Register callback on the UDP node
network::g_pRoveCommUDPNode->AddUDPCallback<double>(
    AddPositionLegCallback,
    manifest::Autonomy::COMMANDS.find("ADDPPOSITIONLEG")->second.DATA_ID
);
```

Because `RoveCommUDP` runs on its own background thread, incoming socket packets are unpacked, matched to their `DATA_ID`, and dispatched to their registered callbacks automatically without polling.

ewpage

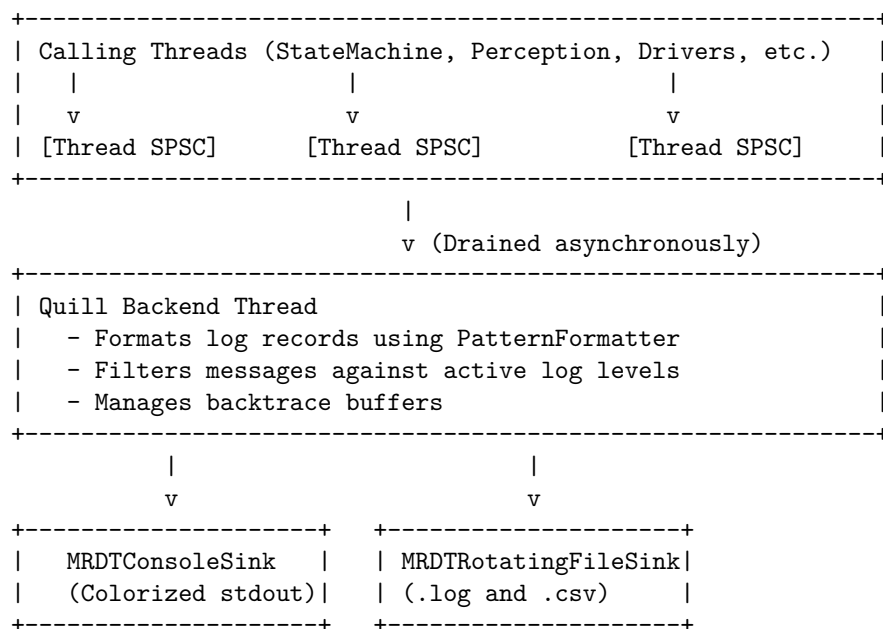
28 Log Files & The Quill Logging Engine

The autonomy software utilizes **Quill**, a low-latency, asynchronous C++ logging library, to manage system telemetry, diagnostics, and debugging outputs across all threads without degrading control-loop execution speed.

28.1 1. Asynchronous Logging Architecture

Traditional synchronous logging libraries format and write log messages directly within the calling thread, introducing unpredictable latency spikes (ranging from microseconds to milliseconds) caused by terminal I/O or filesystem buffer writes. In real-time robotics operating at high frequencies (such as state machine and motor control loops), such delays can cause jitter or missed deadlines.

Quill solves this by decoupling log generation from I/O operations: 1. **Frontend:** When a worker thread invokes a logging macro (e.g., `LOG_INFO`), Quill serializes the arguments into a lock-free, bounded Single-Producer Single-Consumer (SPSC) queue dedicated to that thread. This operation takes mere nanoseconds. 2. **Backend:** A dedicated background logging worker thread (`quill::Backend::start()`) continuously drains these queues, executes the `fmt`-based string formatting, and dispatches the formatted text to the configured sinks.



28.2 2. Configured Sinks and Loggers

In `src/AutonomyLogging.cpp`, initialization creates custom sinks and distinct logger instances:

28.2.1 Sinks

1. `MRDTCConsoleSink`:
 - Subclasses `quill::ConsoleSink`.
 - Directs formatted output to standard output (`stdout`).
 - Automatically detects color support (`quill::ConsoleSinkConfig::ColourMode::Automatic`) and applies ANSI color sequences assigned to each log level (`szDebugColor`, `szInfoColor`, `szWarningColor`, `szErrorColor`, `szCriticalColor`).

- Evaluates `logging::g_eConsoleLogLevel` on every write to selectively output messages that meet or exceed the active console threshold.
2. **MRDTRotatingFileSink:**
- Subclasses `quill::RotatingFileSink`.
 - Configured with open mode 'a' (append).
 - Generates two synchronized files inside `logs/<YYYY-MM-DD_HH-MM-SS>/`:
 - `console_output.log`: Formatted text log for human reading.
 - `console_output.csv`: Tab-delimited CSV log for automated parsing and analytics.
 - Evaluates `logging::g_eFileLogLevel` on every write to record messages that meet or exceed the active file threshold.

28.2.2 Logger Instances

- `logging::g_qFileLogger`: Routes output to both file sinks (`qLogFileSink` and `qCSVFileSink`).
- `logging::g_qConsoleLogger`: Routes output exclusively to `qConsoleSink`.
- `logging::g_qSharedLogger`: Routes output simultaneously to `qLogFileSink`, `qCSVFileSink`, and `qConsoleSink`. This is the primary logger used across the autonomy codebase.

28.3 3. Formatting Patterns

Quill uses pattern strings to structure each log line:

Stream	Pattern String
Log File (.log)	<code>%(time) %(log_level) [% (thread_id)]</code>
CSV File (.csv)	<code>[(file_name):(line_number)] %(message)</code>
Console (stdout)	<code>%(time),\t%(log_level),\t[% (thread_id)],\t[(file_name):%(line_number)] %(message)</code>
Timestamp Format	<code>%(time) %(log_level:9) [% (thread_id)]</code> <code>%Y-%m-%d %H:%M:%S.%Qms (Year-Month-Day Hour:Minute:Second.Milliseconds)</code>

Example log file line:

```
2026-09-08 14:32:05.124 INFO [140234512] [StateMachineHandler.cpp:88] Rover state changed from Idle to I
```

28.4 4. Logging Levels and Priorities

Quill log levels are ordered by increasing severity. When a log level is set, only messages equal to or higher in priority are emitted:

Priority	Quill Log Level	Typical Code Usage
1	<code>quill::LogLevel::TraceL3</code>	Unused in general operation.
2	<code>quill::LogLevel::TraceL2</code>	Deep memory or byte-level debug tracing.
3	<code>quill::LogLevel::TraceL1</code>	High-frequency raw sensor data (e.g., every individual LiDAR point or pixel coordinate).
4	<code>quill::LogLevel::Debug</code>	Intermediate algorithm values, Stanley control vectors, raw GPS/IMU readings.

Priority	Quill Log Level	Typical Code Usage
5	<code>quill::LogLevel::Info</code>	State machine transitions, waypoint completions, camera initialization confirmations.
6	<code>quill::LogLevel::Notice</code>	Main loop interactive diagnostic dumps (hotkeys <code>h</code> , <code>f</code> , <code>p</code> , <code>s</code> , <code>d</code> , <code>t</code> , <code>m</code>).
7	<code>quill::LogLevel::Warning</code>	Non-fatal conditions: tracking bounding box lost, GPS data aging past threshold, out-of-bounds parameters clamped.
8	<code>quill::LogLevel::Error</code>	Recoverable component failure: camera frame retrieval failure, LiDAR DB query failure, obstacle blockages.
9	<code>quill::LogLevel::Critical</code>	Fatal errors causing program shutdown: RoveComm socket binding failure, low battery failsafe trigger.
10	<code>quill::LogLevel::Backtrace</code>	Reserved for backtrace buffer dumps.

28.5 5. Backtrace Ring Buffer

All three loggers configure a backtrace ring buffer:

```
g_qFileLogger->init_backtrace(10, quill::LogLevel::Critical);
g_qConsoleLogger->init_backtrace(10, quill::LogLevel::Critical);
g_qSharedLogger->init_backtrace(10, quill::LogLevel::Critical);
```

When active, Quill keeps a rolling buffer of the last 10 log messages across all levels (even if their priority was lower than the active logging filter). If an event triggers a **CRITICAL** log, Quill automatically flushes the entire backtrace ring buffer to the sinks. This ensures that the events and states directly preceding a crash are captured in the log without requiring verbose logging during normal execution.

28.6 6. Dynamic Runtime Log Level Adjustment

Operators can adjust the logging severity thresholds on the fly without restarting the autonomy software. The autonomy system registers a RoveComm UDP callback for the **SETLOGGINGLEVELS** command (`manifest::Autonomy::COMMANDS.find("SETLOGGINGLEVELS")->second.DATA_ID`):

```
const std::function<void(const rovecomm::RoveCommPacket<uint8_t>&, const sockaddr_in&)> SetLoggingLevel =
[] (const rovecomm::RoveCommPacket<uint8_t>& stPacket, const sockaddr_in& stdAddr)
{
    (void) stdAddr;

    const int nMinConsoleLevel = static_cast<int>(constants::CONSOLE_MIN_LEVEL);
    const int nMinFileLevel    = static_cast<int>(constants::FILE_MIN_LEVEL);

    const int nRequestedConsoleLevel = stPacket.vData[0];
```

```

const int nRequestedFileLevel    = stPacket.vData[1];

bool bConsoleLevelChangePermitted = nRequestedConsoleLevel >= nMinConsoleLevel;
bool bFileLevelChangePermitted    = nRequestedFileLevel >= nMinFileLevel;

logging::g_eConsoleLogLevel = bConsoleLevelChangePermitted ? static_cast<quill::LogLevel>(stPacket.vData[2]) : nMinConsoleLevel;
logging::g_eFileLogLevel    = bFileLevelChangePermitted ? static_cast<quill::LogLevel>(stPacket.vData[3]) : nMinFileLevel;

LOG_INFO(logging::g_qSharedLogger, "Incoming SETLOGGINGLEVELS: [Console: {}, File: {}]", stPacket.vData[2], stPacket.vData[3]);

```

28.6.1 Safety Rules:

- The requested console log level cannot be set lower than `constants::CONSOLE_MIN_LEVEL`.
- The requested file log level cannot be set lower than `constants::FILE_MIN_LEVEL`.
- If an invalid level is sent, the previous logging level is retained.

28.7 7. Logging in C++ Source Code

To emit log statements in any module, include `AutonomyLogging.h` and use Quill's format macros:

```

#include "AutonomyLogging.h"

// Info log with variable formatting
LOG_INFO(logging::g_qSharedLogger, "Rover arrived at waypoint {}. Distance to target: {:.2f} m", nWaypointIndex, nDistanceToTarget);

// Warning log with contextual details
LOG_WARNING(logging::g_qSharedLogger, "NavBoard GPS age exceeds limit: {:.3f} s > {:.3f} s", dAge, constants::GPS_MAX_AGE);

// Error log
LOG_ERROR(logging::g_qSharedLogger, "Camera retrieval timed out on camera index {}", nCamIndex);

// Critical log (will trigger backtrace dump and program abort if fatal)
LOG_CRITICAL(logging::g_qSharedLogger, "Failed to bind RoveComm UDP socket to port {}", nPort);

```

ewpage

29 Camera Feeds & Video Recording

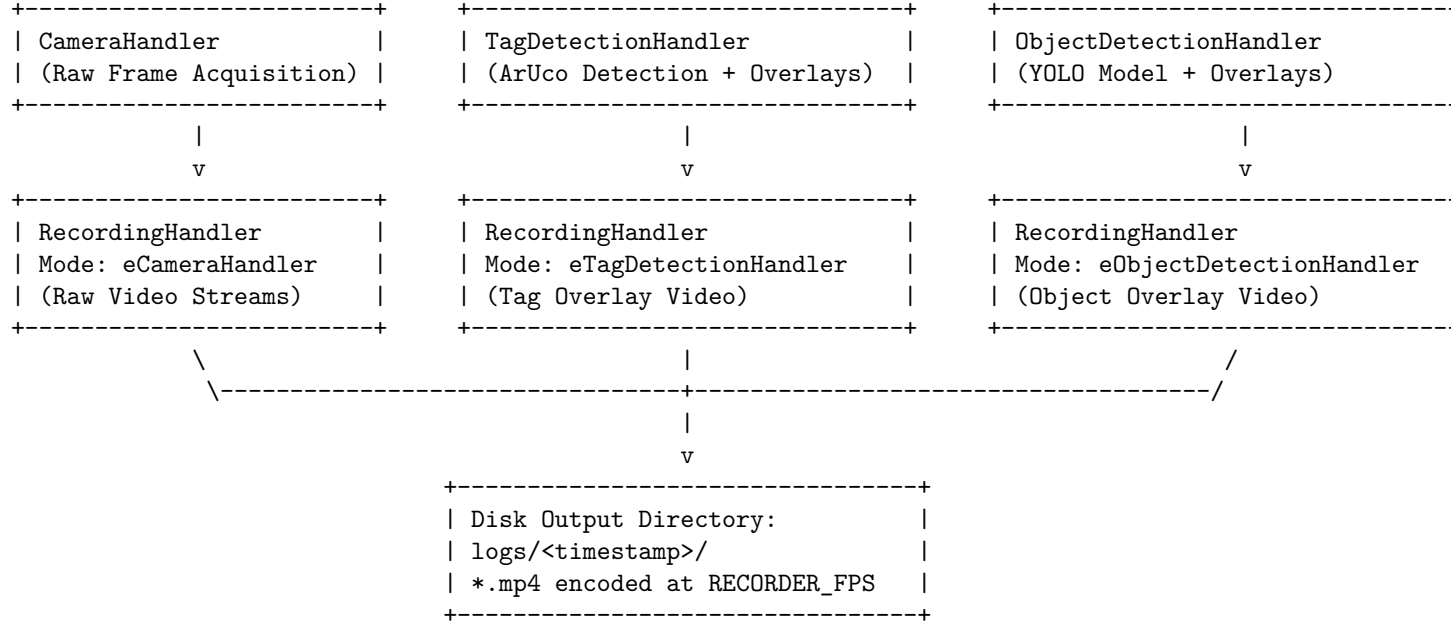
During testing operations and competition runs, reviewing raw camera perspectives alongside real-time neural network and detector inferences is essential for diagnostic analysis. Video recording is managed by the `RecordingHandler`, an asynchronous recording subsystem that runs independently of sensor capture and machine vision inference pipelines.

29.1 1. Asynchronous Recording Architecture

H.264 video compression and disk I/O are computationally heavy operations. If detector threads or camera acquisition threads encoded and wrote video frames synchronously, perception loop frequencies would drop significantly.

To isolate the critical path: 1. `RecordingHandler` runs in a dedicated thread derived from `AutonomyThread<void>`. 2. Dedicated instances are spawned by three parent handlers: `CameraHandler`,

TagDetectionHandler, and ObjectDetectionHandler. 3. In each recording loop iteration, the RecordingHandler requests frames asynchronously from its parent cameras or detectors using non-blocking futures (`std::future<bool>`), leaving the parent processing pipelines completely unhindered. 4. Frames are fed into OpenCV `cv::VideoWriter` pipelines initialized with four-character code H264 or mp4v targeting .mp4 container files.



29.2 2. Recording Modes and Stream Types

The `RecordingHandler::RecordingMode` enum configures the nature of the frames captured:

29.2.1 1. eCameraHandler

- **Source:** Directly captures raw RGB frames from initialized cameras (`ZEDCamera` instances or `BasicCam` USB devices).
- **Content:** Clean, unprocessed video without bounding boxes, telemetry overlays, or artificial markers.
- **Use Case:** Post-session sensor calibration, stereo disparity ground-truth evaluation, and photogrammetry reconstruction.

29.2.2 2. eTagDetectionHandler

- **Source:** Calls `TagDetector::RequestOverlayFrameCopyAsync()`.
- **Content:** The original camera frame augmented with OpenCV visualization overlays:
 - Green/red bounding quad outlines tracing detected ArUco markers.
 - Marker ID numeric tags drawn above the tag center.
 - 3D pose coordinate axes projecting outward from the marker center.
 - Distance (meters) and yaw offset angle (degrees) rendered as diagnostic text.
- **Use Case:** Verifying tag detection range, tracker stability during rover movement, and corner refinement accuracy.

29.2.3 3. eObjectDetectionHandler

- **Source:** Calls `ObjectDetector::RequestOverlayFrameCopyAsync()`.
- **Content:** The original camera frame augmented with LibTorch YOLO inference outputs:

- Bounding boxes color-coded by detected object class (Mallet, Water Bottle, Rock Pick).
- Class name label and confidence percentage score.
- 20th-percentile geolocated distance and relative bearing angles.
- **Use Case:** Evaluating model false positive rates, tracking continuity, occlusion handling, and non-maximum suppression (NMS) thresholds in outdoor sunlight conditions.

29.3 3. Output Storage Structure

At startup, `AutonomyLogging::InitializeLoggers()` establishes a unified run folder based on the session timestamp:

```
logs/
+-- 2026-09-08_15-30-00/
|  |-- console_output.log
|  |-- console_output.csv
|  |-- visualization.html
|  |-- spatial_map.ply
|  |-- MainCam_Raw.mp4
|  |-- RearCam_Raw.mp4
|  |-- MainCam_TagOverlay.mp4
|  |-- RearCam_TagOverlay.mp4
|  |-- MainCam_ObjectOverlay.mp4
+-- RearCam_ObjectOverlay.mp4
```

Videos are written at the resolution established by `constants::ZED_MAINCAM_RESOLUTIONX` and `constants::ZED_MAINCAM_RESOLUTIONY` (typically 1280x720) and throttled to `constants::RECORDER_FPS`.

29.4 4. Configuration Constants

Recording behavior is selectively controlled in `src/AutonomyConstants.cpp`:

Constant Name	Type	Default	Description
<code>RECORDER_FPS</code>	<code>int</code>	<code>15</code>	Target framerate for video encoding. Lower values conserve GPU encoder capacity and disk bandwidth.
<code>ZED_MAINCAM_ENABLE_RECORDING</code>	<code>bool</code>	<code>true</code>	Toggles raw video recording on the forward ZED camera.
<code>ZED_REARCAM_ENABLE_RECORDING</code>	<code>bool</code>	<code>true</code>	Toggles raw video recording on the rear ZED camera.
<code>TAGDETECT_MAINCAM_ENABLE_RECORDING</code>	<code>bool</code>	<code>true</code>	Toggles overlay recording on the forward ArUco tag detector.
<code>TAGDETECT_REARCAM_ENABLE_RECORDING</code>	<code>bool</code>	<code>false</code>	Toggles overlay recording on the rear ArUco tag detector.

Constant Name	Type	Default	Description
OBJECTDETECT_MAINCAM_ENABLE_RECORDING	bool	true	Toggles overlay recording on the forward YOLO object detector.
OBJECTDETECT_REARCAM_ENABLE_RECORDING	bool	false	Toggles overlay recording on the rear YOLO object detector.

29.5 5. Lifecycle Management

1. **Initialization:** Handlers create their internal `RecordingHandler` instances upon construction.
2. **Activation:** In `src/main.cpp`, recording is formally enabled after hardware verification:

```
globals::g_pCameraHandler->StartRecording();
globals::g_pTagDetectionHandler->StartRecording();
globals::g_pObjectDetectionHandler->StartRecording();
```

3. **Shutdown:** When the main loop exits (via signal or Q key), the parent handler stop calls signal the `RecordingHandler` thread to finish writing remaining frames, close the `cv::VideoWriter` streams cleanly, and finalize file containers on disk.

ewpage

30 Path Plots, Analytics, and Post-Mortem Graphing

Trajectory analysis, path planning verification, and control-loop diagnostics are supported by dedicated offline playback tooling and runtime logging analytics. This document details how log data is parsed, visualized, and evaluated to assess rover navigation performance.

30.1 1. The `log_playback.py` Visualization Suite

Located at `tools/logging/log_playback.py`, this Python analysis script parses the tab-delimited CSV log produced by Quill during an autonomy session (`logs/<timestamp>/console_output.csv`) and generates animated, synchronized multi-panel plots using `matplotlib`.

30.1.1 Script Capabilities and Parsed Telemetry

The script uses regular expressions to extract structured metrics from the unstructured and semi-structured Quill log messages:

1. **3D Positional Trajectory:**
 - **GPS Position:** Extracted from GPS Data: `<lat> lat, <lon> lon, <alt> alt`.
 - **Fused Rover Pose:** Extracted from Rover Pose: `<lat> (lat), <lon> (lon), <alt> (alt), <deg> (degrees), GNSS/VIO FUSED? = <bool>`.
 - Plotted as dual 3D trajectory subplots (`Axes3D`) comparing raw GPS against the visual-inertial fused pose.
2. **Heading and Compass Alignment:**
 - **Compass Data:** Extracted from Incoming Compass Data: `<heading>`.

- Plotted alongside the fused pose heading to detect local magnetic anomalies, declination drift, or IMU yaw lag.
3. **GNSS Accuracy and Fix Quality:**
 - Extracted from Incoming Accuracy Data: (2D: <val>, 3D: <val>, Compass: <val>, FIX_TYPE: <fix>).
 - Tracks 2D horizontal accuracy, 3D spatial accuracy, and fix status across the run, highlighting GPS degradation under satellite occlusion.
 4. **Thread Framerate Monitors:**
 - Extracted from periodic `Threads FPS` messages.
 - Tracks the performance of 11 concurrent threads simultaneously:
 - `main_process_fps`
 - `main_cam_fps`
 - `left_cam_fps`
 - `right_cam_fps`
 - `ground_cam_fps`
 - `main_detector_fps`
 - `left_detector_fps`
 - `right_detector_fps`
 - `state_machine_fps`
 - `rovecomm_udp_fps`
 - `rovecomm_tcp_fps`
 5. **Drivetrain Power and State Overlay:**
 - Extracted from Driving at: (<left_power>, <right_power>) and Current State: <state_name>.
 - Displays real-time dynamic bar graphs of left and right motor efforts (scaled between -1.0 and +1.0) correlated directly with active state machine phases (`Navigating`, `ApproachingMarker`, `Stuck`, etc.).
 6. **Waypoint Queue Transitions:**
 - Extracts waypoint additions and queue resets to demarcate leg boundaries visually along the timeline.

30.1.2 Usage

Run the playback script from within the autonomy workspace:

```
python3 tools/logging/log_playback.py path/to/logs/2026-09-08_15-30-00/console_output.csv
```

30.2 2. Real-Time Path Tracking via VisualizationHandler

While `log_playback.py` operates offline after a run, real-time spatial trajectories and planned paths are maintained dynamically by the `VisualizationHandler`:

- **Path History (`m_vPathHistory`):**
 - Accumulates `DisplayPoint` structs containing Easting, Northing, and Altitude relative to the local session origin `m_stOriginUTM`.
 - Captures the traversal score and the active rover state at each recorded point.
 - Rendered as a persistent trail in the Three.js 3D web interface.
 - **Planned Path (`m_vPlannedPath`):**
 - Queried at 1 Hz from `GeoPlanner::GetPlannedPath()`.
 - Transferred over HTTP JSON endpoints (`/api/planned_path`) to display upcoming A* trajectory splines and search pattern geometries.
-

30.3 3. Matplot++ Library Integration

For standalone benchmarking, algorithm evaluation, and C++ plotting routines, the build environment provides pre-compiled packages for **Matplot++** (located in `tools/package-builders/matplotplusplus/`).

Matplot++ provides a C++ syntax mirroring MATLAB plotting functions, allowing developers to: - Export costmap heatmaps and elevation contours directly from DuckDB point queries. - Plot A* search trees, open sets, and closed sets during path planning algorithm tuning. - Save high-resolution vector plots (`.svg` or `.png`) for technical design reports and competition review documentation.

ewpage

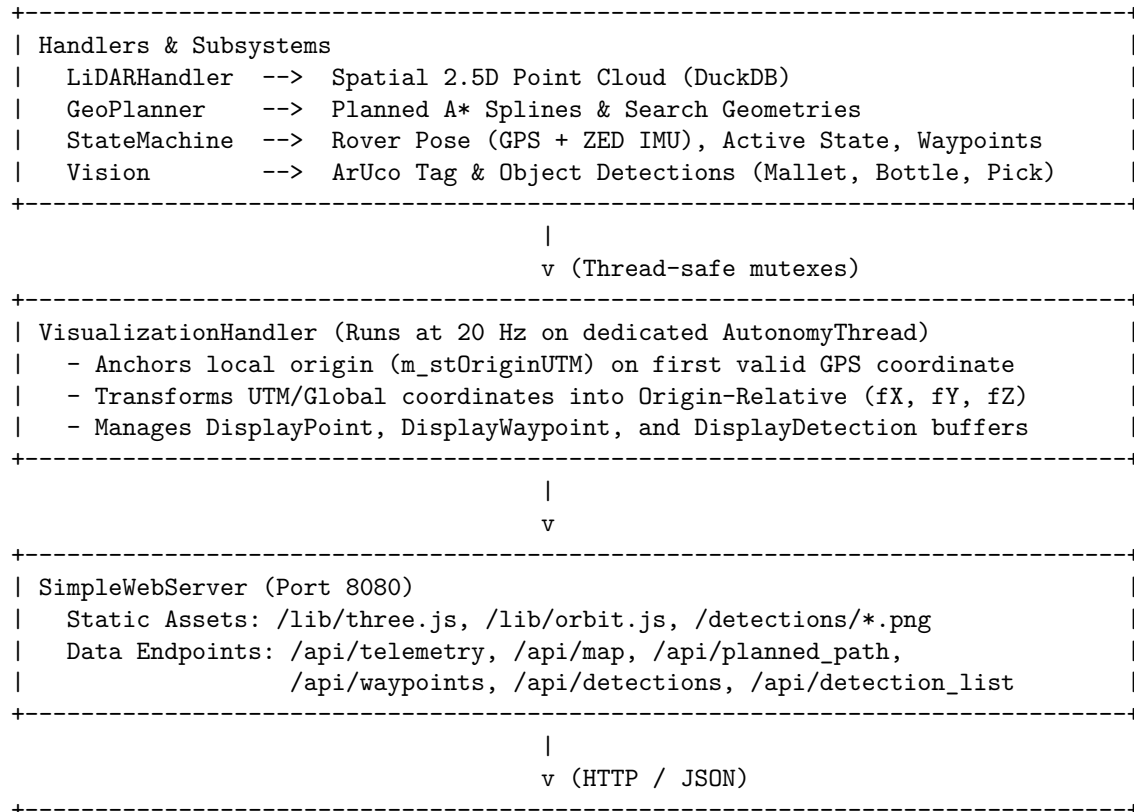
31 3D Interactive Visualization & The Visualization Engine

The autonomy software incorporates a real-time, interactive 3D digital twin engine hosted directly on the rover's Jetson processor. Managed by the **VisualizationHandler**, this subsystem aggregates sensor fusion telemetry, spatial LiDAR point clouds, planned path splines, and neural network detections into a live WebGL 3D scene accessible from any device on the rover's local network.

31.1 1. System Architecture

To avoid burdening the embedded compute platform with heavy native desktop UI dependencies (such as Qt or X11 OpenGL contexts), visualization is implemented via an asynchronous HTTP and WebGL architecture.

The **VisualizationHandler** inherits from **AutonomyThread<void>** and executes at 20 Hz (`SetMainThreadIPSLimit(20)`). It encapsulates a lightweight HTTP server (**SimpleWebServer**) that serves web assets and JSON telemetry endpoints while maintaining thread-safe internal state buffers.



Web Client (Laptop Browser, Basestation GUI, or visualizer.themrdr.org)	
- Three.js WebGL Scene with OrbitControls	
- Interactive camera panning, rotation, and elevation cross-sections	
+-----+	

31.2 2. Local Coordinate Origin Anchoring

Global UTM coordinates contain large Easting and Northing values (e.g., Easting $\sim 500,000$ m, Northing $\sim 4,200,000$ m). Directly feeding these values into single-precision 32-bit floating-point WebGL buffers introduces severe floating-point jitter and vertex distortion.

To eliminate this precision loss, `VisualizationHandler` initializes a session origin (`m_stOriginUTM`) upon receiving the first valid GPS coordinate from `StateMachineHandler::SmartRetrieveRoverPose()`:

```
if (!m_bOriginSet)
{
    if (std::abs(stRoverUTM.dEasting) > 1.0 || std::abs(stRoverUTM.dNorthing) > 1.0)
    {
        m_stOriginUTM = stRoverUTM;
        m_bOriginSet = true;
    }
}
```

All spatial vectors streamed over the web API are projected into this local tangential frame:

$$\Delta X = \text{Easting} - \text{Origin}_{\text{Easting}}$$

$$\Delta Z = \text{Northing} - \text{Origin}_{\text{Northing}}$$

$$\Delta Y = \text{Altitude} - \text{Origin}_{\text{Altitude}}$$

This yields millimeter-level visualization precision centered at (0, 0, 0).

31.3 3. Core Data Structures

The handler packages spatial elements into compact C++ structs protected by dedicated mutexes:

```
// Historical trajectory breadcrumb
struct DisplayPoint
{
    float fX, fY, fZ;    // Coordinates relative to m_stOriginUTM
    float fScore;        // Terrain traversal score from costmap
    int nState;          // Active robot state machine state
};

// Target waypoint or navigation beacon
struct DisplayWaypoint
{
    float fX, fY, fZ;    // Coordinates relative to m_stOriginUTM
    int nType;           // Waypoint type enum
};

// Persistent vision detection
struct DisplayDetection
```

```

{
    float fX, fY, fZ;    // Coordinates relative to m_stOriginUTM
    int nType;           // 10 = ArUco Tag, 11 = Mallet, 12 = Water Bottle, 13 = Rock Pick
};

```

31.4 4. HTTP API Endpoints

The internal `SimpleWebServer` exposes endpoints on port 8080 (configurable via `constants::VISUALIZER_WEBSERVER_PORT`):

Endpoint	Method	Response Format	Purpose
/	GET	HTML (text/html)	Serves the embedded Three.js 3D web application interface.
/lib/three.js	GET	JavaScript	Serves the bundled Three.js library.
/lib/orbit.js	GET	JavaScript	Serves the Three.js OrbitControls camera manipulation library.
/api/telemetry	GET	JSON	Returns current rover pose (relative position, heading, pitch, roll, active state).
/api/map	GET	JSON	Queries <code>LiDARHandler</code> for spatial terrain points within a radius of the rover and returns point positions with traversal costs.
/api/planned_path	GET	JSON	Returns upcoming waypoint coordinates and the active A* trajectory spline.
/api/waypoints	GET	JSON	Returns all waypoints currently queued in <code>WaypointHandler</code> .
/api/detections	GET	JSON	Returns 3D positions and type tags of all confirmed visual detections.
/api/detection_list	GET	JSON	Returns a list of filenames for detection snapshot images captured on disk.
/detections/<file>	GET	PNG Image	Serves static detection snapshot images recorded during the run.

31.5 5. Web Client Features

The frontend application renders the digital twin with the following layers: - **Rover Model & Coordinate Frame**: Indicates current position and orientation in real-time. - **Path History Ribbon**: Color-coded line tracing where the rover has driven, shaded by the traversal cost of the terrain beneath it. - **Planned Path Spline**: Cyan path vector showing the route generated by GeoPlanner. - **LiDAR Point Cloud**: Colored terrain scatter plot reflecting relative elevation and slope hazards. - **Waypoints & Markers**: Cylindrical beacons showing goal locations, labeled by ID and search radius tolerances. - **Detection Markers**: Specialized 3D glyphs highlighting confirmed objects (green for ArUco tags, orange for mallets, blue for water bottles, brown for rock picks).

31.6 6. Shutdown Environment Exports

When `src/main.cpp` executes its shutdown sequence (upon receiving SIGINT or user hotkey Q), persistent exports are saved into `logs/<timestamp>/`:

1. Self-Contained HTML Export (`visualization.html`):

- `pVisualizationHandler->SaveVisualization()` computes the bounding envelope of the entire run, queries all corresponding LiDAR tiles from the DuckDB database, and bakes the Three.js viewer, telemetry history, and 3D terrain points into a standalone `.html` file.
- This file can be opened offline in any standard web browser without network connectivity or external web servers.

2. ZED Spatial Mapping PLY Export (`spatial_map.ply`):

- If ZED spatial mapping was active (`pMainCam->GetSpatialMappingState() == sl::SPATIAL_MAPPING_STATE::ON`), the main routine asynchronously requests the fused 3D mesh:

```
std::future<sl::Mesh> fuSpatialMap;  
pMainCam->ExtractSpatialMapAsync(fuSpatialMap);  
sl::Mesh slSpatialMap = fuSpatialMap.get();  
slSpatialMap.save(szFilePath.c_str(), sl::MESH_FILE_FORMAT::PLY);
```

- The resulting `.ply` mesh can be loaded into CloudCompare, MeshLab, or Blender for detailed geometric inspection of terrain obstacles.
-

31.7 7. Hosted MRDT Web Visualizer Suite

In addition to the onboard lightweight web server running on port 8080, the team maintains an ecosystem of cloud-hosted web applications deployed at visualizer.themrdr.org:

Web Application	Direct URL	Description & Capabilities
Main Visualizer Hub	visualizer.themrdr.org	Central landing portal for MRDT telemetry tools, flight software digital twins, and spatial data tooling.
Autonomy Task & Route Visualizer	visualizer.themrdr.org/autonomy-task/	Pre-mission planning, waypoint layout design, simulated route traversal, and state machine search geometry verification. Enables operators to visualize GPS coordinates, test obstacle clearances, and review planned A* splines.

Web Application	Direct URL	Description & Capabilities
LiDAR Inspection Tool	visualizer.themrtdt.org/lidar-tool/	3D web-based point cloud analyzer for inspecting USGS LAS/LAZ terrain tiles. Features cross-sectional elevation slicing, gradient angle filters, contour mapping, and traversability threshold tuning before importing into DuckDB.

ewpage

32 Autonomy Constants Reference & Tuning Guide

This document serves as the exhaustive engineering reference for configurable constants within the autonomy software. All constants are declared in `src/AutonomyConstants.h` and defined in `src/AutonomyConstants.cpp`. Modifying any value requires recompilation.

32.1 1. General & System Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
<code>MODE_SIM</code>	<code>bool</code>	<code>false</code>	When true, activates simulated camera WebRTC pipelines and simulator network sockets. Set to false for hardware deployment on the physical rover.
<code>SIM_IP_ADDRESS</code>	<code>std::string</code>	<code>"127.0.0.1"</code>	IP address for connecting to the Unreal Engine RoveSoSimulator instance.
<code>SIM_WEBSOCKET_PORT</code>	<code>uint</code>	<code>8080</code>	WebRTC signaling port for simulator pixel streaming.
<code>SIM_WEBRTC_QP</code>	<code>uint</code>	<code>20</code>	Quantization parameter for WebRTC video decompression. Lower values yield higher image fidelity.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
BATTERY_MINIMUM_CELL_VOLTAGE	double	3.2	Minimum allowable LiPo cell voltage (V). If battery voltage falls below this threshold and checks are enabled, the state machine transitions to IdleState .
BATTERY_CHECKS_ENABLED	bool	true	Enables or disables battery monitoring failsafes. Set to false in lab environments lacking PMS telemetry.
LOGGING_OUTPUT_PATH_ABSOLUTE	std::string	"../logs/"	Base directory on the filesystem where session log folders and recordings are written.
CONSOLE_MIN_LEVEL	quill::LogLevel	Debug	Absolute minimum permissible log level for the console sink. Restricts SETLOGGINGLEVELS changes from muting vital diagnostics.
FILE_MIN_LEVEL	quill::LogLevel	Debug	Absolute minimum permissible log level for file sinks (.log and .csv).
CONSOLE_DEFAULT_LEVEL	quill::LogLevel	Notice	Initial console verbosity at program launch. Recommended Notice or Info for competition to prevent terminal saturation.
FILE_DEFAULT_LEVEL	quill::LogLevel	Debug	Initial file verbosity at launch. Captures full diagnostic details to disk.
ROVECOMM_OUTGOING_UDP_PORT	int	11000	Target UDP port for outgoing telemetry packets dispatched across the rover network.
ROVECOMM_OUTGOING_TCP_PORT	int	11000	Target TCP port for reliable packet transmission.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
ROVECOMM_TCP_INTERFACE_IP	std::string	"0.0.0.0"	Network interface IP bound by the local RoveComm TCP listener socket.

32.2 2. Drive & Kinematics Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
DRIVE_MAX_POWER	float	1.0	Absolute software ceiling for motor effort scalar.
DRIVE_MIN_POWER	float	-1.0	Absolute software floor for motor effort scalar.
DRIVE_MAX_SAFE_POWER	float	0.7	Global safety clamp applied across all autonomous states to limit peak speeds during testing.
DRIVE_PID_PROPORTIONAL	double	0.008	Proportional gain K_p for closed-loop heading correction. Increases responsiveness to angular heading errors.
DRIVE_PID_INTEGRAL	double	0.0001	Integral gain K_i for steady-state heading error accumulation to overcome surface scrubbing friction.
DRIVE_PID_DERIVATIVE	double	0.001	Derivative gain K_d to dampen angular velocity and mitigate overshoot when approaching the setpoint heading.
DRIVE_PID_FEEDFORWARD	double	0.0	Feedforward gain K_{ff} for heading control.
DRIVE_PID_MAX_ERROR	double	180.0	Maximum angular error (degrees) fed into the PID controller calculation.
DRIVE_PID_MAX_INTEGRAL_TERM	double	0.2	Anti-windup clamping threshold on the accumulated integral term.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
DRIVE_PID_MAX_RAMP_RATE	double	0.05	Slew rate limiter restricting maximum change in PID output per second to prevent aggressive motor current spikes.
DRIVE_PID_OUTPUT_FILTER	double	0.1	Low-pass filter smoothing coefficient applied to the controller output.
DRIVE_PID_TOLERANCE	double	1.5	Heading error tolerance band (degrees) within which heading error is treated as zero.
DRIVE_PID_OUTPUT_REVERSED	bool	false	Reverses polarity of PID controller output if motor cabling is inverted.
DRIVE_SQUARE_CONTROL_INPUTS	bool	false	Applies parabolic scaling ($x \cdot x $) to throttle commands to enhance fine control at low velocities.
DRIVE_CURVATURE_KINEMATICS_ALLOW_TURN_WHILE_STOPPED	bool	true	Allows zero-radius point turns when forward throttle is zero.

32.3 3. Inclinometer Damping Multipliers

Dynamically down-scales motor throttle as terrain slope steepens to prevent high-speed rollover incidents:

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
DRIVE_BOARD_MIN_SLOPE	float	10.0	Slope angle (degrees) below which no damping is applied ($D = 1.0$).
DRIVE_BOARD_MAX_SLOPE	float	30.0	Slope angle (degrees) at which damping reaches maximum severity ($D = D_{\min}$).
DRIVE_BOARD_MIN_DAMP	float	0.4	Minimum motor power multiplier (40% throttle) permitted at or beyond MAX_SLOPE.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
DRIVE_BOARD_MAX_DAMP	float	1.0	Maximum motor power multiplier (100% throttle) applied when terrain is flat.
DRIVE_BOARD_ROLL_WEIGHT	float	0.6	Weight assigned to roll axis tilt. Weighted higher because lateral rollovers occur at lower angles than pitch rollovers.
DRIVE_BOARD_PITCH_WEIGHT	float	0.4	Weight assigned to pitch axis tilt.
DRIVE_BOARD_YAW_WEIGHT	float	0.0	Weight assigned to yaw axis tilt (typically zero).

32.4 4. Video Recording Handler Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
RECORDER_FPS	int	15	Framerate limit for encoding .mp4 video files to disk.
ZED_MAINCAM_ENABLE_RECORDING	bool	true	Toggles raw video recording from the forward ZED camera.
ZED_REARCAM_ENABLE_RECORDING	bool	true	Toggles raw video recording from the rear ZED camera.
TAGDETECT_MAINCAM_ENABLE_RECORDING	bool	true	Toggles ArUco overlay frame recording from the forward detector.
TAGDETECT_REARCAM_ENABLE_RECORDING	bool	false	Toggles ArUco overlay frame recording from the rear detector.
OBJECTDETECT_MAINCAM_ENABLE_RECORDING	bool	true	Toggles YOLO overlay frame recording from the forward detector.
OBJECTDETECT_REARCAM_ENABLE_RECORDING	bool	false	Toggles YOLO overlay frame recording from the rear detector.

32.5 5. Camera & Perception Hardware Constants

32.5.1 ZED Camera SDK Parameters

- ZED_BASE_RESOLUTION: `s1::RESOLUTION::HD720` (1280x720).
- ZED_MEASURE_UNITS: `s1::UNIT::METER`.
- ZED_COORD_SYSTEM: `s1::COORDINATE_SYSTEM::LEFT_HANDED_Y_UP`.
- ZED_DEPTH_MODE: `s1::DEPTH_MODE::NEURAL` (High accuracy neural stereo matching).
- ZED_DEFAULT_MINIMUM_DISTANCE: 0.3f (Clamps depth below 30 cm to prevent lens distortion artifacts).
- ZED_DEFAULT_MAXIMUM_DISTANCE: 25.0f (Maximum usable range in meters).
- ZED_DEFAULT_FLOOR_PLANE_ERROR: 0.15f (Floor plane detection tolerance in meters).
- ZED_DEPTH_STABILIZATION: 1 (Enables temporal smoothing of depth point clouds).

32.5.2 Physical Extrinsic Offsets

- Forward ZED Camera:
 - ZED_MAINCAM_EASTING_OFFSET: 0.0 m
 - ZED_MAINCAM_NORTHING_OFFSET: 0.35 m (Camera mounted 35 cm forward of chassis center)
 - ZED_MAINCAM_ALTITUDE_OFFSET: 0.65 m (Camera mounted 65 cm above ground level)
 - Quaternion rotation offsets: X = 0.0, Y = 0.0, Z = 0.0, W = 1.0
- Rear ZED Camera:
 - MODE_REAR_ZED: `true`
 - ZED_REARCAM_NORTHING_OFFSET: -0.35 m (Camera mounted 35 cm behind chassis center)
 - ZED_REARCAM_ALTITUDE_OFFSET: 0.65 m
 - Quaternion rotation offsets: X = 0.0, Y = 1.0, Z = 0.0, W = 0.0 (180 degree yaw rotation)

32.6 6. Vision Detection & Tracking Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
BBOX_MIN_LIFETIME_THRESHOLD	double	0.3	Minimum duration (seconds) a detection must persist before confirmation. Filters single-frame visual noise.
BBOX_MIN_SCREEN_PERCENTAGE	double	0.0005	Minimum screen area fraction required to track an object bounding box.
BBOX_TRACKER_LOST_TIMEOUT	double	1.0	Maximum time (seconds) a lost tracker will extrapolate position before deregistration.
BBOX_TRACKER_MAX_TRACK_TIME	double	30.0	Maximum lifespan (seconds) of a continuous bounding box track before mandatory re-detection.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
BBOX_TRACKER_IOU_MATCH_THRESHOLD	double	0.3	Intersection-over-Union threshold for associating new neural inferences with active trackers.
TAGDETECT_TORCH_MODEL	std::string	"../data/models/yolo_models/best_tag_weightscript"	path for YOLO ArUco detection model.
OBJECTDETECT_TORCH_MODEL	std::string	"../data/models/yolo_models/best_script/v8s_x640_150epochs_augment"	path for YOLO competition object model (or Tucumcari model)
TAGDETECT_MAINCAM_TORCH_CONFIDENCE	double	0.55	Confidence score cutoff for ArUco tag neural detections.
TAGDETECT_MAINCAM_TORCH_NMS_THRESHOLD	double	0.45	Non-Maximum Suppression IoU threshold for tag bounding boxes.
OBJECTDETECT_MAINCAM_TORCH_CONFIDENCE	double	0.60	Confidence cutoff for Mallet, Water Bottle, and Rock Pick detections.
OBJECTDETECT_MAINCAM_TORCH_NMS_THRESHOLD	double	0.45	Non-Maximum Suppression IoU threshold for object bounding boxes.
ARUCO_TAG_SIDE_LENGTH	float	0.20	Physical edge length of competition ArUco tags (meters). Set to 0.20 m per URC rules.

32.7 7. State Machine Execution Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
STATEMACHINE_MAX_IPS	int	60	Loop execution rate ceiling (Hz) for the core state machine thread.
STATEMACHINE_ZED_REALIGN_THRESHOLD	double	1.5	Discrepancy (meters) between visual odometry and GPS before triggering visual realignment.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
NAVIGATING_MOTOR_POWER	double	0.6	Base motor power scalar while in NavigatingState.
NAVIGATING_REACHED_GOAL_RADIUS	double	1.5	Arrival tolerance radius (meters) around a target navigation waypoint.
NAVIGATING_VERIFY_POSITION	bool	true	When true, stops rover at waypoint and averages GPS samples to verify arrival.
NAVIGATING_VERIFY_SAMPLE_TIME	double	10.0	Duration (seconds) rover remains stationary to sample and average GPS fixes to confirm arrival at waypoint. Reduced from 30.0s to minimize mission clock penalty.
NAVIGATING_SLOWDOWN_WITHIN_WAYPOINT_RADIUS	bool	true	Toggles linear speed deceleration as rover closes within waypoint arrival radius.
APPROACH_MARKER_MOTOR_POWER	double	0.35	Motor power scalar while actively homing in on an ArUco post.
APPROACH_MARKER_PROXIMITY_THRESHOLD	double	1.0	Target standoff distance (meters) for completing marker approach phase.
APPROACH_MARKER_LOST_GIVE_UP_TIME	double	5.0	Maximum time (seconds) marker can remain lost before falling back to search patterns.
APPROACH_OBJECT_MOTOR_POWER	double	0.30	Motor power scalar while closing distance to a mission object.
APPROACH_OBJECT_PROXIMITY_THRESHOLD	double	0.8	Target standoff distance (meters) for completing object approach phase.
APPROACH_OBJECT_REQUIRED_TIME_UPDATE	double	0.5	Required fraction of detection frames needed to maintain active homing state.
SEARCH_MOTOR_POWER	double	0.40	Motor power scalar while tracing spiral or snake search patterns.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
SEARCH_ANGULAR_STEP_DEGREES	double	15.0	Angular step size (degrees) for computing Archimedean spiral search trajectory waypoints.
SEARCH_SPIRAL_SPACING	double	2.0	Radial distance (meters) between concentric arms of the spiral pattern.
SEARCH_ZIGZAG_SPACING	double	3.0	Track separation distance (meters) for zigzag search geometry.
REVERSE_MOTOR_POWER	double	-0.35	Motor effort scalar applied during ReversingState .
REVERSE_DISTANCE	double	1.5	Total linear distance (meters) traversed backward during recovery maneuvers.
REVERSE_TIMEOUT_PER_METER	double	4.0	Time allowance (seconds/meter) before reversing maneuver aborts due to stall.
STUCK_SAME_POINT_PROXIMITY	double	0.5	Spatial radius (meters) within which the rover is flagged as stuck if progress halts.
STUCK_HEADING_ALIGN_TIMEOUT	double	8.0	Maximum time (seconds) allotted to turn toward recovery headings in StuckState .

32.8 8. Path Planning & Controller Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
GEOPLANNER_TILE_SIZE	double	50.0	Edge length (meters) of spatial tiles cached from DuckDB terrain database.
ASTAR_AVOIDANCE_MULTIPLIER	double	2.5	Multiplier inflating obstacle boundaries in the 2.5D costmap during A* search.

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
ASTAR_MAX_SEARCH_GRID	double	150.0	Maximum dimension (meters) of local search window to cap computational complexity.
ASTAR_MAX_SEARCH_TIME	double	0.5	Maximum execution time (seconds) before A* yields best available partial path.
ASTAR_NODE_SIZE	double	0.25	Spatial grid cell resolution (meters) for A* nodes.
STANLEY_CROSSTRACK_CONTROL_GAIN	double	0.8	Gain coefficient k scaling lateral deviation correction in the Stanley controller.
STANLEY_WHEELBASE	double	1.2	Effective kinematic wheelbase length (meters) between front and rear axle centers.
STANLEY_ANGULAR_VELOCITY_LIMIT	double	1.5	Maximum permissible yaw angular rate (rad/s) computed by the Stanley controller.
STANLEY_PREDICTION_HORIZON	int	5	Lookahead steps N simulated by <code>UnicycleModel</code> forward projection.
STANLEY_PREDICTION_TIME_STEP	double	0.1	Integration time step dt (seconds) for kinematic unicycle trajectory simulation.
STANLEY_MIN_STABLE_SPEED	double	0.15	Minimum velocity threshold (m/s) in Stanley denominator to prevent division by zero.
CLOSE_RANGE_PENALTY	double	0.5	Speed damping scalar applied by Pure Pursuit when within close range of path terminators.

32.9 9. Navigation Board Driver Constants

Constant Name	Type	Typical Value	Engineering Purpose & Tuning Effect
NAVBOARD_MAX_GPS_DATA_AGE	double	2.0	Maximum acceptable age (seconds) of GPS packets before data is marked stale.
NAVBOARD_MAX_COMPASS_DATA_AGE	double	1.0	Maximum acceptable age (seconds) of compass packets before data is marked stale.
NAVBOARD_EASTING_OFFSET	double	0.0	GPS antenna physical mounting offset in Easting axis relative to rover center.
NAVBOARD_NORTHING_OFFSET	double	-0.20	GPS antenna mounting offset in Northing axis (meters).
NAVBOARD_ALTITUDE_OFFSET	double	0.85	GPS antenna mounting offset in Altitude axis above ground plane (meters).

ewpage

33 CMake Build Configuration & Options

The autonomy software utilizes **CMake** (version 3.24.3 or newer) as its meta-build system. The build configuration enforces modern C++20 language standards, compiler constraints, and aggressive compile-time optimizations to support high-performance embedded execution on the NVIDIA Jetson platform.

33.1 1. Compiler and Toolchain Constraints

33.1.1 Mandatory GCC 10 Enforcement

The top-level `CMakeLists.txt` executes a pre-configuration check verifying the host compiler version:

```
execute_process(
    COMMAND gcc -dumpversion
    OUTPUT_VARIABLE GCC_VERSION
    OUTPUT_STRIP_TRAILING_WHITESPACE
)
if(NOT GCC_VERSION VERSION_EQUAL "10.0")
    message(FATAL_ERROR "Only GCC 10.0 is allowed! Detected GCC version ${GCC_VERSION}.")
endif()
```

Rationale: Strict binary compatibility with NVIDIA JetPack 5.x, CUDA 11.4/12.x, and LibTorch runtime libraries requires GCC 10. Attempting to build with GCC 9 or GCC 11+ produces ABI incompatibilities or internal compiler segmentation faults during TorchScript and TensorRT template expansions.

33.1.2 C++ and CUDA Standards

- **CMAKE_CXX_STANDARD 20**: Enables modern C++20 features (concepts, ranges, `std::span`, designated initializers, three-way comparisons, and coroutine primitives).
 - **CMAKE_CUDA_STANDARD 20**: Aligns device CUDA compilation with host C++20 standards.
-

33.2 2. Performance & Compilation Optimizations

1. LLD Linker Integration:

```
if(NOT MSVC)
    add_link_options("-fuse-ld=lld")
endif()
```

Replaces the default GNU `ld` or `gold` linkers with LLVM's `lld`. This reduces final binary link times by up to 70% and substantially curtails peak RAM consumption on RAM-constrained Jetson systems.

2. CMake Unity Builds:

```
set(CMAKE_UNITY_BUILD ON)
set(CMAKE_UNITY_BUILD_BATCH_SIZE 8)
```

Aggregates up to 8 translation units into unified compilation files. This dramatically reduces redundant header parsing overhead (particularly heavy headers like OpenCV, LibTorch, and ZED SDK), speeding up full builds and minimizing compiler process swapping.

33.3 3. Build Types

Specify the build type during CMake generation:

```
cmake -B build -DCMAKE_BUILD_TYPE=<Type>
```

- **Release** (Mandatory for Testing & Competition):
 - Sets optimization level `-O3`.
 - Strips debug symbol tables and enables aggressive inlining and vectorization.
 - Required to hit targeted vision framerates (30 FPS) and maintain sub-millisecond state machine iterations.
 - **Debug** (Local Developer Diagnostics Only):
 - Sets optimization level `-O0` and includes `-g` debug symbols.
 - Useful for debugging segmentation faults with GDB or Valgrind.
 - *Warning*: Do not run on the physical rover in competition; perception pipelines will drop below 5 FPS.
 - **RelWithDebInfo**:
 - Compiles with `-O2 -g`. Provides optimized execution while preserving stack traces for core dump analysis.
-

33.4 4. Configurable CMake Options

Options are toggled via `-D<OPTION>=ON|OFF` during configuration:

Option	Default	Output Executable	Description
BUILD_SIM_MODE	OFF	Autonomy_Software_Sim (when ON)	Defines __AUTONOMY_SIM_MODE__=1. Switches sensor streams to consume WebRTC pixel streaming and local loopback sockets from Unreal Engine RoveSoSimulator.
BUILD_TESTS_MODE	OFF	tests/*	Enables CTest and builds GoogleTest unit and integration test suites in tests/.
ENABLE_LIDAR_GEO_UTESTS	OFF	N/A	Enables specialized LiDAR database query and GeoPlanner unit tests that require physical USGS LiDAR data tiles (available from MissouriM- RDT/USGS_Data).
BUILD_CODE_COVERAGE	OFF	N/A	Injects GCC profiling flags (-O0 -g -fprofile-arcs -ftest-coverage --coverage) to generate gcov/lcov coverage reports in CI pipelines.
BUILD_COVERAGE_WATCH	OFF	N/A	Enables real-time code coverage file-watching mode for development workflows.
BUILD_VERBOSE_MODE	OFF	N/A	Generates verbose Makefiles displaying all raw compiler and linker commands during compilation.
BUILD_EXAMPLES_MODE	OFF	examples/*	Compiles standalone hardware verification examples for isolated subsystem testing.
LINK_SHARED_ZED	ON	N/A	Links dynamically against the ZED SDK shared libraries (sl::Camera). Set to OFF if using custom static ZED builds.

Option	Default	Output Executable	Description
<code>RC_CROSS_COMPILE</code>	OFF	N/A	Cross-compiles the RoveComm communication library for both Linux and Windows environments.
<code>LIST_ALL_VARS</code>	OFF	N/A	Dumps all active internal CMake cache variables and include paths to the console during configuration.

33.5 5. Subsystem Dependencies

The CMake build automatically locates and links the following system packages:

- **CUDA Toolkit** (`CUDA::cudart`, `CUDA::curand`): Accelerates deep learning inferences and stereoscopic processing.
- **LibTorch** (`Torch`): PyTorch C++ front-end for executing YOLO TorchScript models.
- **OpenCV** (`OpenCV`): Computer vision algorithms, image manipulation, ArUco fiducials, and video encoding.
- **ZED SDK** (`sl::Camera`): Stereolabs depth estimation, positional tracking, and spatial mapping.
- **GeographicLib** (`GeographicLib::GeographicLib`): High-accuracy ellipsoidal geodesy and UTM coordinate conversions.
- **DuckDB** (`duckdb`): Embedded analytical spatial SQL engine for querying 2.5D USGS elevation maps.
- **Quill** (`quill::quill`): Low-latency asynchronous multi-threaded logging engine.
- **BS::thread_pool**: Header-only thread pool for concurrent camera and frame dispatching.
- **RoveComm**: Missouri MRDT telemetry transport layer.

ewpage

34 Autonomy Troubleshooting & Field Diagnostics Guide

This document provides systematic diagnostics and rapid-resolution procedures for common hardware, networking, algorithmic, and software anomalies encountered during field testing and competition operations.

34.1 1. Live Terminal Diagnostic Hotkeys

The autonomy main loop runs in non-canonical terminal mode (`SetNonCanonicalTerminalMode()`), enabling real-time keyboard commands without requiring a trailing newline. When investigating system behavior via SSH, press the following single keys:

Key	Diagnostic Action	Information Output
<code>h</code>	Print Help Menu	Displays the catalog of available terminal diagnostic keys.

Key	Diagnostic Action	Information Output
f	Thread Performance Telemetry	Dumps instantaneous execution frequencies (FPS) across all 11 worker threads (Main, Cameras, Detectors, State Machine, RoveComm UDP/TCP, Visualizer).
p	Rover Spatial Pose	Dumps current UTM coordinates (Easting, Northing, Altitude) and compass heading from <code>SmartRetrieveRoverPose()</code> .
s	ZED Inertial Telemetry	Dumps raw linear acceleration (X, Y, Z), angular velocity (X, Y, Z), and Euler gyro orientation from the forward ZED camera.
d	Drive Actuation State	Dumps real-time normalized motor power commands (Left Power, Right Power) currently sent to the drive board.
t	ArUco Marker Telemetry	Dumps total detected tags, Best OpenCV tag ID, distance (m), and yaw angle, alongside Best Torch tag metrics.
m	Object Detection Telemetry	Dumps total identified mission objects, best detected class (Mallet, Water Bottle, Rock Pick), distance, and yaw angle.
q	Graceful System Shutdown	Signals all threads to halt, stops motor output, flushes logs, generates <code>visualization.html</code> , and exports <code>spatial_map.ply</code> .

34.2 2. Common Symptom-Based Solutions

34.2.1 1. Drivetrain Jitters or Oscillates While Driving

- **Underlying Cause:** Closed-loop heading PID controller is over-tuned, or the Navigation Board IMU is subject to electromagnetic interference from motor power cables.
- **Diagnostic Action:** Press **d** to observe drive power fluctuations. If powers rapidly oscillate between positive and negative extremes, the heading controller is unstable.
- **Resolution:**
 1. Decrease `DRIVE_PID_PROPORTIONAL` (K_p) by 20% to 30% in `AutonomyConstants.cpp`.
 2. Increase `DRIVE_PID_DERIVATIVE` (K_d) slightly to dampen overshoot.
 3. Increase `DRIVE_PID_OUTPUT_FILTER` to smooth high-frequency actuation chatter.
 4. Press **p** and verify that the compass heading is stable while driving in a straight line.

34.2.2 2. Rover Stalls When Attempting Point Turns

- **Underlying Cause:** Static ground friction exceeds motor breakaway torque, or integral anti-windup clamping is restricting power build-up.
- **Diagnostic Action:** Observe if drive powers plateau at low values without overcoming carpet or soft soil resistance.
- **Resolution:**
 1. Increase `DRIVE_PID_INTEGRAL` (K_i) slightly to accelerate error integration.
 2. Increase `DRIVE_PID_MAX_INTEGRAL_TERM` in `AutonomyConstants.cpp` to allow higher sustained current during stationary turning.
 3. Verify battery voltage is above 3.5V per cell to prevent voltage sag brownouts under heavy steering load.

34.2.3 3. “RoveComm UDP/TCP Node Failed to Initialize”

- **Underlying Cause:** Socket port 11000 is already bound by a stale background instance, a failed prior run, or an unclosed Python utility.
- **Diagnostic Action:** Inspect terminal logs for `RoveComm did not initialize properly! UDPNode Status: 0, TCPNode Status: 0.`
- **Resolution:**
 1. Check for processes occupying the port:

```
sudo lsof -i :11000
sudo lsof -i :11001
```
 2. Terminate the blocking process:

```
sudo kill -9 <PID>
```
 3. Restart `./Autonomy_Software`.

34.2.4 4. ZED Camera Connection Failures or Framerate Dropping to Zero

- **Underlying Cause:** USB 3.0 power fluctuations, kernel USB autosuspend, or loose USB-C locking screws.
- **Diagnostic Action:** Run `lsusb` in the terminal. Each connected ZED 2i camera should present two separate USB endpoints (one for video, one for internal sensors/IMU):

```
Bus 002 Device 004: ID 2b03:f880 STEREO LABS ZED 2i
Bus 001 Device 007: ID 2b03:f881 STEREO LABS ZED 2i MCU
```
- **Resolution:**
 1. If endpoints are missing, reseal the USB-C locking connector on the Jetson carrier board.
 2. Restart the Jetson USB subsystem or reset the camera device:

```
sudo systemctl restart udev
```
 3. Press `f` in the autonomy terminal to verify camera FPS recovers to 30 FPS.

34.2.5 5. Autonomy Unexpectedly Enters IdleState

- **Underlying Cause:** Battery failsafe triggered, or all recovery attempts in `StuckState` were exhausted.
- **Diagnostic Action:** Inspect `console_output.log` or press `f` to check current state.
- **Resolution:**
 1. Look for log message: `Battery cell voltage below minimum!`. If present, swap the main LiPo battery.
 2. Look for log message: `Exhausted all recovery attempts in StuckState!`. If present, the rover was trapped by insurmountable terrain; inspect physical rover clearance and obstacle map.

34.2.6 6. Visual Odometry Drift and Discontinuous Jumps

- **Underlying Cause:** Stereoscopic feature loss in featureless terrain (e.g., flat sand or monochrome asphalt) causing the ZED SDK visual odometry to diverge from GNSS coordinates.
- **Diagnostic Action:** Check if `STATEMACHINE_ZED_REALIGN_THRESHOLD` warnings appear repeatedly in the log.
- **Resolution:**
 1. Press `p` to inspect current rover pose coordinates.
 2. If visual odometry has drifted, press `q` to halt cleanly and restart autonomy software to force visual odometry to re-anchor to GNSS UTM coordinates.
 3. Ensure camera lenses are clean of dust, mud, and water droplets.

34.2.7 7. LiDAR Database Opening Failure

- **Underlying Cause:** The DuckDB spatial database file is missing, corrupt, or has incorrect file permissions.
- **Diagnostic Action:** Look for `Failed to open LiDAR database.` at launch.
- **Resolution:**
 1. Verify the file path configured in `constants::LIDAR_HANDLER_DB_PATH`.
 2. Check that the `.db` file exists and has read permissions:

```
ls -la data/LiDAR/
```
 3. Ensure DuckDB shared libraries are correctly located in the system library path.

ewpage

35 Autonomy Pre-Flight & Operations Checklist

This checklist defines the required engineering procedures for configuring, verifying, compiling, and operating the autonomy software during field trials and official University Rover Challenge (URC) competition runs.

35.1 1. On-Rover Jetson Setup & Verification

35.1.1 Codebase and Toolchain Verification

- ☐ **Verify Git Branch:**
 - `git status` (Confirm working directory is clean and checked out to the designated deployment branch).
- ☐ **Fetch Latest Commits:**
 - `git fetch origin && git pull`
- ☐ **Verify Build System & Compiler Version:**
 - `gcc --version` (Must strictly report GCC 10.x per CMake configuration rules).
- ☐ **Verify Configuration Constants in `src/AutonomyConstants.cpp`:**
 - `BUILD_SIM_MODE` is set to `OFF` in `CMakeLists.txt`.
 - `LIDAR_HANDLER_DB_PATH` points to the valid DuckDB database file.
 - `TAGDETECT_TORCH_MODEL` points to the verified TorchScript ArUco weights (`data/Models/best_tag.pt`).
 - `OBJECTDETECT_TORCH_MODEL` points to the verified TorchScript mission object weights (`data/Models/best_object.pt`).
 - Extrinsic offsets match the current physical camera and GPS mast placements.

35.1.2 Compilation

- ☐ **Clean Build Configuration:**
 - `rm -rf build && mkdir build`

- `cmake -B build -DCMAKE_BUILD_TYPE=Release`
- Ensure CMake configuration prints `-- [ ]: Sim Mode: Disabled` and generates `Autonomy_Software` (not `Autonomy_Software_Sim`).

□ **Compile:**

- `make -C build -j$(nproc)`
- Confirm binary links without warnings or missing shared library errors.

35.1.3 Bench Sensor Verification (Prior to Mounting)

□ **Check USB Device Tree:**

- Run `lsusb` in the terminal.
- Verify that each connected ZED 2i camera populates two distinct devices:
 - * Camera Video Interface (ID `2b03:f880`)
 - * Sensor / IMU Microcontroller (ID `2b03:f881`)

□ **Execute Standalone Smoke Test:**

- Launch executable: `./build/Autonomy_Software`
- Press `f`: Confirm camera capture rates and detector threads achieve steady 30 FPS.
- Press `s`: Confirm linear acceleration and gyro angular rates update dynamically when moving the camera.
- Press `t` and `m`: Confirm tag and object detector models are loaded onto the CUDA device without memory exhaustion.
- Press `q`: Confirm clean shutdown, database closure, and export of `visualization.html`.

□ **Network Interface Isolation:**

- Put the Jetson internal Wi-Fi adapter into Airplane Mode / Disabled state to prevent wireless interference with the 900 MHz and 5.8 GHz competition radio links.

35.2 2. Basestation Operations & Deployment

35.2.1 Remote Session Launch

□ **Establish Secure Shell Session:**

- Open terminal on the basestation computer: `ssh pigeon@192.168.3.100` (Default password: `nandgate`).

□ **Launch Autonomy Process:**

- `cd ~/Documents/GitHub/Autonomy_Software`
- `./build/Autonomy_Software`

□ **Verify RoveComm Telemetry:**

- Confirm log message reports: `RoveComm UDP and TCP nodes successfully initialized`.
- Verify heartbeat packets populate in the Basestation GUI telemetry dashboard.

35.3 3. Mission Waypoint Injection & Leg Types

Waypoints are queued into the autonomy system via the Basestation GUI or through direct RoveComm packets. Ensure the appropriate leg type and parameters are injected:

35.3.1 Leg Type Configurations

1. **GNSS Position Leg (ADDPPOSITIONLEG):**

- **Command ID:** `11002`
- **Payload:** `[Latitude, Longitude, WaypointID]`
- Used for raw coordinate transit legs. The rover navigates until it enters `constants::NAVIGATING_REACHED_GOAL_RADIUS` (typically 1.5 m).

2. **ArUco Marker Leg (ADDMARKERLEG):**
 - **Command ID:** 11003
 - **Payload:** [Latitude, Longitude, MarkerID, SearchRadius]
 - **MarkerID Parameter:**
 - 0, 1, 2, or 3: Directs the rover to search specifically for that numerical tag ID.
 - -1 (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::ANY`): Rover accepts any detected ArUco marker.
 - **SearchRadius:** Clamped between 0.0 m and 40.0 m. Specifies the Archimedean spiral search envelope upon reaching the GNSS post location.
 3. **Mission Object Leg (ADDOBJECTLEG):**
 - **Command ID:** 11004
 - **Payload:** [Latitude, Longitude, ObjectID, SearchRadius]
 - **ObjectID Parameter:**
 - -2 (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::MALLET`): Orange mallet target.
 - -3 (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::WATERBOTTLE`): Water bottle target.
 - -4 (`manifest::Autonomy::AUTONOMYWAYPOINTTYPES::ROCKPICK`): Geologist rock hammer target.
 - **SearchRadius:** Clamped between 0.0 m and 40.0 m.
-

35.4 4. Autonomous Mission Execution Workflow

- ☐ **Confirm Waypoint Queue Status:**
 - Check terminal output: Confirm the incoming leg appears with correct coordinates, ID, and search radius.
 - ☐ **Initiate Autonomous Navigation:**
 - On the Basestation GUI, click **Start Autonomy** (dispatches `eStartAutonomy` packet).
 - Verify rover state machine transitions from `eIdle` to `eNavigating`.
 - ☐ **Monitor Multimedia Board Status Lights:**
 - **Off:** Autonomy is stopped / idle.
 - **Solid Red:** Autonomy is actively driving.
 - **Flashing Green:** Rover has successfully navigated to the waypoint, detected the target tag, or completed object approach.
 - **Solid Blue:** Teleoperation override active.
 - ☐ **Emergency Abort Protocol:**
 - In the event of an imminent collision or boundary violation, click **Abort** on the Basestation GUI or press the physical wireless E-Stop.
 - The state machine immediately clears motor commands via `DriveBoard::SendStop()` and returns to `eIdle`.
-

35.5 5. Post-Mission Data Archival

- ☐ **Terminate Autonomy Session:**
 - Press `q` in the active SSH terminal.
- ☐ **Archive Run Artifacts:**
 - Navigate to `logs/<YYYY-MM-DD_HH-MM-SS>/`.
 - Verify generation of:
 - * `console_output.log` and `console_output.csv`
 - * `visualization.html` (Open locally in browser to inspect traversed route and 3D point cloud)
 - * `spatial_map.ply` (ZED stereoscopic mesh)
 - * Recorded video streams (`MainCam_Raw.mp4`, `MainCam_TagOverlay.mp4`, `MainCam_ObjectOverlay.mp4`)
- ☐ **Log Trajectory Playback:**

- Run `python3 tools/logging/log_playback.py logs/<timestamp>/console_output.csv` to review controller tracking fidelity and telemetry timelines.

ewpage